Database Systems

Department of Information Technology,

Faculty of Computing,

Sri Lanka Institute of Information Technology.

Course Introduction

- Introduction to Unit
  - Contents
    - Design and development of object-relational databases
    - Understanding of indexing techniques
    - Understanding of query optimization
    - Understanding and application of database tuning techniques.
    - Understanding the concepts and techniques used in distributed databases.

Course Introduction… (contd.)

- Contact hours
  - 2 hours lecture/week
  - 2 hours practical/week
  - 1 hour tutorial/week

- Recommended References

  - Oracle Documentation

  - Ramakrishnan, R. and Gehrke, J., *Database Management Systems*, 3nd Editon, McGraw-Hill,2003.

  - Garcia-Molina, H., Ullman, J.D. and Widom, J., *Database Systems: The Complete Boo*k,Prentice-Hall, 2002.

Course Introduction… (contd.)

- Grading
  - Midterm test - 20%
  - Practical examinations - 20%
  - Final Exam - 60%

Course Introduction… (contd.)

- All related course materials on CourseWeb page
  - http://courseweb.sliit.lk
  - Enrolment key: **IT3020**
- Contact persons:
  - Prasanna S. Haddela (Lecturer-in-charge)
    - Email: prasanna@sliit.lk (preferred for appointments)
    - Office: 7th Floor - Malabe Campus
  - Ms. Thamali Dassanayake
    - Office: 7th Floor, Malabe Campus
    - Email: thamali.d@sliit.lk

Database Management Systems III

Introduction to Object Relational Database Systems

Lecture - 1

Background

- Relational model (70's):
  - Clean and simple representation and access.
    - Tables, primary and foreign keys, SQL.
    - Good foundation of relational structure, algebra, and normal forms.
  - Swept the DB market in the 1980s.
    - Continues to dominate the DB applications even now.
  - Right for business and administrative data.
    - Tables of atomic attributes adequate to represent information.

Background

- Relational model (70's):
  - Not as good for other kinds of data (e.g., multimedia, networks, CAD).
    - Cumbersome to manage such data with Binary Large Objects (BLOBs).
  - RDB has limitations even in its core application areas.
    - Set valued attributes (e.g., academic qualifications, children of employees).
    - Some logical identifiers replaced by artificial keys (e.g., addresses of properties, multiple attribute keys).
    - ISA Hierarchies of entity sets (e.g., student is a person).

Background

- Object-Oriented models (80's):
  - Proposed as an alternative to relational model.
    - To overcome its limitations.
  - Complex objects were supported.
    - nested relations.
  - Added DBMS functionality to OO programming environments.
    - Object ID, inheritance, methods.
  - ODMG standards: Object data and query languages
    - ODL & OQL.
  - Made limited inroads in the 1990s, then faded away.

Background

- Object-relational DBMS (mid-90's):
  - Extends relational model to support a broader class of applications.
  - Bridge between relational and OO models.
  - SQL:99 and SQL:2003 standards extended SQL to support the OO model.
  - DBMS vendors (e.g., Oracle, IBM, Informix) have added OO functionality.
    - Adherence to the standard varies.

Limitations of relational model

- No support for set valued attributes
  - Example:
    - Person (ID: string, Name: string, PhoneN: string, childID: string)
    - Key: (ID, PhoneN, childID)
    - Not in 3NF (FD: ID -> Name)

ID	Name	PhoneN	ChildID
111	Joe	4576	222
111	Joe	6798	222
222	Bob	5162	333

1NF to 3NF

ID	Name	PhoneN	ChildID
111	Joe	4576	222
111	Joe	6798	222
222	Bob	5162	333

ID	Name
111	Joe
111	Joe
222	Bob

ID	PhoneN
111	4576
111	6798
222	5162

ID	ChildID
111	222
111	222
222	333

Limitations of RDB: Example

- 1NF relation:
  - Person (ID: string, Name: string, PhoneN: string, childID: string)
- 3NF relations:
  - Person(ID, Name)
  - Phone (ID, PhoneN)
  - ChildOf (ID, childID)
- Query: Find the phone numbers of all of Joe's grandchildren.
- SQL on both schema need multiple joins.

Set valued attributes

ID	Name	PhoneN	ChildID
111	Joe	{4576, 6798}	{222}
222	Bob	{5162}	{333}

- A more appropriate schema:
  - Person (ID: string, Name: string, PhoneN: {string}, child: {string})
  - Query: Find the phone numbers of all of Joe's grandchildren.
  - Ideally, the query could be written as:
    - Select p.child.child.PhoneN
    - From Person p
    - Where p.Name = 'Joe'
  - Note: Oracle implementation is different from this.

SQL extensions for ORDB

- Add OO features to the type system of SQL.
  - columns can be of new user defined types (UDTs)
  - user-defined methods on UDTs
  - reference types and “deref”
  - inheritance
  - old SQL schemas still work! (backwards compatible)

User Defined Types

- A user-defined type, or UDT, is essentially a class definition, with data fields and methods.
- Two uses:
  1. As a rowtype, that is, the type of a relation.
  2. As the type of an attribute of a relation.

Object types in Oracle

- Uses object types for both columns and rows of relations.
- Example:

```
CREATE TYPE BarType AS OBJECT (
 name CHAR(20),
 addr CHAR(20) )
 /
```

- Note: in Oracle, type definitions must be followed by a slash (/) to store the type.

Object Type

- An object type has 3 components:
  - A name identifies the object type uniquely within the schema.
  - Attributes model the structure and state of the real-world entity.
    - Attributes can be built-in types or object types.
  - Methods, functions or procedures implement operations.
    - (To be covered later.)

Creating Row Objects

- Type declarations do not create tables.
- Object types used in place of attribute lists in CREATE TABLE statements.
  - Example:

```
CREATE TABLE bars OF BarType;
```

- Each row of the table represents an object of given rowtype.

Inserting Values

- As a multi-column table:

```
INSERT INTO bars VALUES ('Sally''s', 'River
    Rd');
```

- As a single column/object value:
  - Each object type (type defined with AS OBJECT) has a type constructor of the same name.
  - Example:

```
INSERT INTO Bars VALUES(
    BarType('Joe''s Bar', 'Maple St.') );
```

Normal select

- Retrieve as a multi-column table:

```
SELECT * FROM bars;
NAME                ADDR
-------------------------------------
Joe's Bar           Maple St.
Sally's             River Rd
```

Select as a Single Column

- Select from bars as a single column table.

```
SELECT VALUE(b) FROM bars b;
VALUE(B)(NAME, ADDR)
--------------------------------------------
BARTYPE('Joe''s Bar  ', 'Maple St.   ')
BARTYPE('Sally''s    ', 'River Rd    ')
```

Column Objects

- Objects that occupy table columns in a relational table.
- Example:

```
CREATE TYPE BeerType AS OBJECT (
 name CHAR(20),
 manf CHAR(20) )
 /
CREATE TABLE menu
 (bar bartype,
  beer beertype,
  price real);
```

  - Menu is a table with two attributes of object types.

Alternative: Object table

```
CREATE TYPE MenuType AS OBJECT (
    bar BarType,
    beer BeerType,
    price real)
/
CREATE TABLE Menu2 OF MenuType;
```

- Menu2 is a table of object type.
- Rows of Menu2 are objects (not so in table Menu).
- Methods can be defined on MenuType and invoked on rows of Menu2.

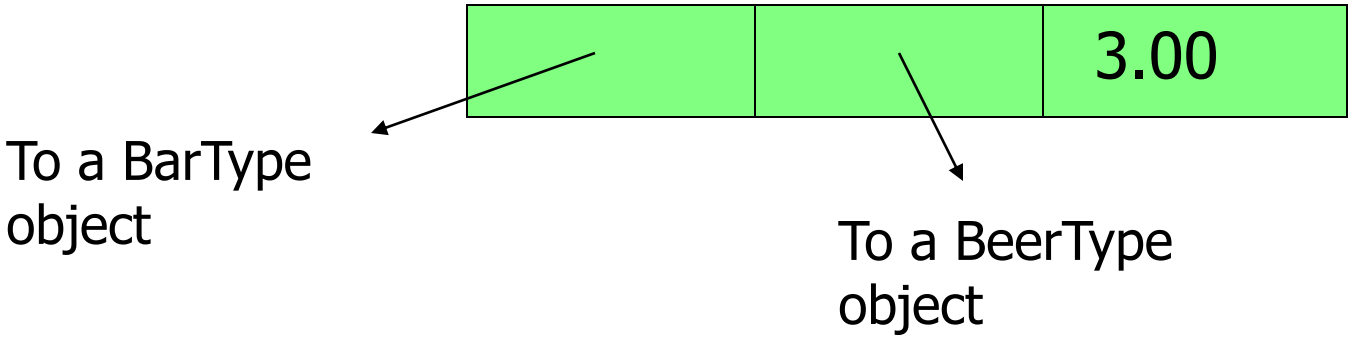
Object References using REF

- If T is a type, then REF T is the type of a reference to T, that is, a pointer to an object of type T.
- Often called an “object ID” in OO systems.
- REF is a built-in data type in Oracle.
- Unlike object ID’s, a REF is visible, although it is usually gibberish.

Example: REF

```
CREATE TYPE MenuType2 AS OBJECT (
    bar         REF BarType,
    beer        REF BeerType,
    price       FLOAT)
/
```

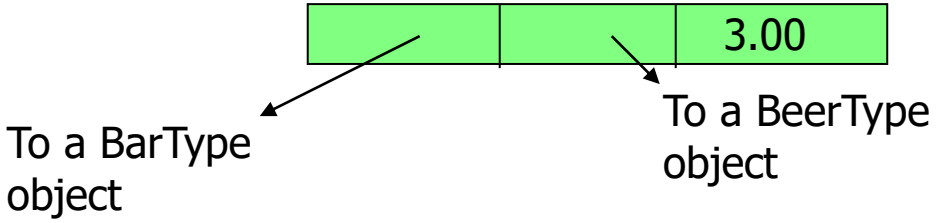
- MenuType2 objects look like:



Obtaining REFs

- To get the REF to a row object,
  - select the object from its object table applying the REF operator.
- Example

```
CREATE TABLE Sells OF MenuType2;
```




```
INSERT INTO sells VALUES(
    (SELECT REF(b) FROM bars b WHERE name='Jim''s'),
    (SELECT REF(e) FROM beers e WHERE name='Swan'),
    2.40);
```

Use aliases to retrieve objects

- To access an attribute of object type, you must use an alias for the relation.
  - Example:

```
SELECT s.Beer.name
    FROM Sells s;
```

- This will not work:

```
SELECT Beer.name
    FROM Sells;
```

- Neither will:

```
SELECT Sells.Beer.name
    FROM Sells;
```

Retrieving with REF Datatype

```
SELECT s.bar.name, s.beer.name, price
  FROM sells s;
  BAR.NAME     BEER.NAME       PRICE
  ---------------------------------------------------
  Jim's        Swan            2.40
  Jim's        Bud             3.00
  Sally's      Fosters         2.65
  Sally's      Miller          2.75
```

Dereferencing

- Accessing the object referred to by a REF is called dereferencing the REF.
  - Dereferencing is automatic, using the dot operator.
- Example

```
SELECT s.beer.name
   FROM Sells s
   WHERE s.bar.name = 'Joe''s Bar';
```

Another Example

```
CREATE TYPE person AS OBJECT (
   name VARCHAR2(30),
   manager REF person );
/
CREATE TABLE person_table OF person;
SELECT x.name, x.manager.name
   FROM person_table x;
```

- x.manager.name follows the pointer from the person x to x's manager (who is another person in the table), and retrieves the manager's name.

Scoped REFs

- Example:

```
CREATE type dept_t as object (dno integer, dname
    varchar(12))
/
CREATE TABLE dept_table OF dept_t;
CREATE TABLE dept_loc (
    dept REF dept_t references dept_table,
    loc VARCHAR(20));
```

  - Only objects of dept_table can be values of dept column.

Co-dependent Types

- Types can depend upon each other for their definitions.
  - Example: Object types EMPLOYEE and DEPARTMENT

$$\boxed{\text{EMPLOYEE}} \longleftrightarrow \boxed{\text{DEPARTMENT}}$$

  - one attribute of EMPLOYEE is the department the employee belongs to and
  - one attribute of DEPARTMENT is the employee who manages the department.

Incomplete Types

```
CREATE TYPE department;
/
CREATE TYPE employee AS
    OBJECT (
    name VARCHAR2(30),
    dept REF department,
    supv REF employee );
/
CREATE TYPE department AS
    OBJECT ( name
    VARCHAR2(30),
    mgr REF employee);
/
```

- CREATE TYPE department; is optional.
- DEPARTMENT is now an *incomplete object type*.
- A REF to an incomplete object type is accepted, so EMPLOYEE type is stored without error.
- Department type is then completed.

Constraints on Object Tables

- Define constraints on an object table just as on other tables
- Example:

```
CREATE TYPE person AS OBJECT (
    pid NUMBER,
    name VARCHAR(25),
    address VARCHAR(50));
/
CREATE TABLE person_table OF person
    ( pid PRIMARY KEY,
     name NOT NULL
   );
```

REF columns

- Oracle does not allow Unique or PRIMARY KEY constraints on REF columns.
- A REF column can be assigned a null value.
- NOT NULL constraint can be specified on such columns.

Null Objects and Attributes

- As in the relational model, a NULL represents an unknown value.
- The following can be NULL:
  - A column value in a table
  - Object
  - Object attribute value
  - A collection, or collection element
- A NULL can be replaced by an actual value later on.

Summary

- ORDB: introduction.
- Object type definitions.
- Creation of row and column objects.
- REF and DEREF operations.
- Constraints on object tables.

Database Systems

ORDB: Collections

Last Lecture

- Object types
  - Declaring
  - Row objects/ column objects
  - References
    - Dereferencing (implicit join)
- Constraints on object tables

- Any questions?

NAME	ADDRESS	INVESTMENTS			
		COMPANY	PURCHASE PRICE	DATE	QTY
John Smith	3 East Av Bentley WA 6102	BHP	12.00	02/10/01	1000
		BHP	10.50	08/06/02	2000
		IBM	58.00	12/02/00	500
		IBM	65.00	10/04/01	1200
		INFOSYS	64.00	11/08/01	1000
Jill Brody	42 Bent St Perth WA 6001	INTEL	35.00	30/01/00	300
		INTEL	54.00	30/01/01	400
		INTEL	60.00	02/10/01	200
		FORD	40.00	05/10/99	300
		GM	55.50	12/12/00	500

COMPANY	CURRENT PRICE	EXCHANGES TRADED	LAST DIVIDEND	EARNING PER SHARE
BHP	10.50	Sydney New York	1.50	3.20
IBM	70.00	New York London Tokyo	4.25	10.00
INTEL	76.50	New York London	5.00	12.40
FORD	40.00	New York	2.00	8.50
GM	60.00	New York	2.50	9.20
INFOSYS	45.00	New York	3.00	7.80

Collection Types

- Useful for modelling one-to-many relationships.
  - Example: An investor makes many share purchases.
- Collection datatypes in Oracle:
  - varrays
  - nested tables.

VARRAYs

- Arrays of variable size.
  - Specify a maximum size when you declare the array type.
  - Creating an array type does not allocate space.
    - Since it is only a type definition.
- Examples:

```
CREATE TYPE price_arr AS VARRAY(10) OF
    NUMBER(12,2);
```

  - The VARRAYs of type PRICES have no more than ten elements, each of data type NUMBER(12,2).

VARRAYs

- A varray type can be used as:
  - the data-type of a column of a relational table;
  - an attribute data-type in an object type definition;
- Example:
  - Create type excharray as varray(5) of varchar(12)
    /
    Create type share_t as object(
    cname varchar(12),
    cprice number(6,2),
    exchanges excharray,
    dividend number(4,2),
    earnings number(6,2))
    /

Creating a VARRAY

- To insert a collection use its type constructor method.
  - The type constructor method has same name as the type.
  - Its argument is a comma-separated list of collection elements.
- Example:
  - create table shares of share_t(
    cname primary key);
  - insert into shares values(‘BHP’, 10.50,
    excharray(‘Sydney’ ,’New York’), 1.50, 3.20);

VARRAY Example

```
CREATE TYPE price_arr AS
   VARRAY(10) OF NUMBER(12,2)
/
CREATE TABLE pricelist (
   pno integer,
   prices price_arr);

INSERT INTO pricelist
   VALUES(1, price_arr(2.50,3.75,4.25));
```

Retrieving from a VARRAY

- SELECT * FROM pricelist;

```
PNO    PRICES
  ---------------------------------
  1      PRICE_ARR(2.5, 3.75, 4.25)
```

- SELECT pno, s.COLUMN_VALUE price FROM pricelist p, TABLE(p.prices) s;

```
PNO     PRICE
---------------
 1       2.5
 1       3.75
 1       4.25
```

Nested Tables

- Allow values of tuple components to be whole relations.
- If *T* is a UDT, we can create a table type *S:*

  CREATE TYPE *S* AS TABLE OF *T* ;

  - Values of type S are relations with rowtype *T*.
  - S can be the type of an attribute in another UDT or in a relation.
  - See the example on next slide.

Example: Nested Table Type

```
CREATE TYPE BeerType AS OBJECT (
   name CHAR(20),
   kind   CHAR(10),
   colour CHAR(10))
/
CREATE TYPE BeerTableType AS
   TABLE OF BeerType
/
```

Example - Continued

- CREATE TABLE Manfs (
  name CHAR(30),
  addr CHAR(50),
  beers beerTableType)
  NESTED TABLE beers STORE AS beer_table;
- BeerTableType used in Manfs relation to store the set of beers by each manufacturer in one tuple.

Storing Nested Tables

- Oracle doesn't really store each nested table as a separate relation
  - it just makes it look that way.
  - tuples of all the nested tables for one attribute *A* are stored in one relation *R*.
- Declare a storage of nested tuples in CREATE TABLE by:

  NESTED TABLE *A* STORE AS *R*
- In previous example,

  NESTED TABLE beers STORE AS beer_table;

Querying a Nested Table

- We can retrieve the value of a nested table like any other value.
- But these values have two type constructors:
  - For the table.
  - For the type of tuples in the table.
- Find the beers by Anheuser-Busch:

```
SELECT beers FROM Manfs
WHERE name = 'Anheuser-Busch';
```

- Produces one value like:

```
BeerTableType(
   BeerType('Bud', 'lager', 'yellow'),
   Beertype('Lite', 'malt', 'pale'),
…)
```

Querying Within a Nested Table

- A nested table can be converted to an ordinary relation by applying TABLE(…).
- This relation can be used in FROM clauses like any other relation.
- Find the ales made by Anheuser-Busch:

```
SELECT b.name
FROM TABLE( SELECT beers
            FROM Manfs
            WHERE name = 'Anheuser-Busch') b
WHERE b.kind = 'ale';
```

Nested Table Example 2

- -- ‘/’ after each type definition omitted to save space

```
CREATE TYPE proj_t AS OBJECT (
    projno NUMBER,
    projname VARCHAR (15));
CREATE TYPE proj_list AS TABLE OF proj_t;
CREATE TYPE employee_t AS OBJECT (
    eno number,
    projects proj_list);
CREATE TABLE employees of employee_t (eno primary key)
NESTED TABLE projects STORE AS employees_proj_table;
```

Inserting and Retrieving

- Insert a row into employees table:

```
INSERT INTO employees VALUES(1000, proj_list(
    proj_t(101, 'Avionics'),
    proj_t(102, 'Cruise control')
));
```

- To retrieve the projects of eno 1000:

```
SELECT *
FROM TABLE(SELECT t.projects FROM employees t
WHERE t.eno = 1000);
```

```
PROJNO  PROJNAME
--------------------
 101    Avionics
 102    Cruise control
```

Collection Unnesting

- Unnest or flatten the collection attribute of a row
  - by joining each row of the nested table with the row that contains the nested table.
  - Example:

```
SELECT e.eno, p.*
FROM employees e, TABLE (e.projects) p;
 ENO      PROJNO   PROJNAME
---- ---------- ----------------
1000      101     Avionics
1000      102     Cruise control
2000      100     Autopilot
```

DML on Collections:

- Use a TABLE expression to identify the nested table values.

```
INSERT INTO TABLE(SELECT e.projects
                  FROM employees e
                  WHERE e.eno = 1000)
  VALUES (103, 'Project Neptune');
UPDATE TABLE(SELECT e.projects
             FROM …) p
  SET p.projname = 'Project Pluto'
  WHERE p.projno = 103;
DELETE TABLE(SELECT e.projects
             FROM …) p
  WHERE p.projno = 103;
```

DML on Nested Tuples

- To drop a particular nested table, set the nested table column in the parent row to NULL.

```
UPDATE employees e
    SET e.projects = NULL
    WHERE e.eno = 1000;
```

- To add back a nested table row:

```
UPDATE employees e
    SET e.projects = proj_list(proj_t(103, 'Project Pluto'))
    WHERE e.eno=1000;
```

DML on Nested Tuples

- There is a difference between a NULL value and an empty constructor. To add back a nested table row, we could have done it in two steps as follows:

```
UPDATE employees e
   SET e.projects = proj_list() // Creates a nested table w/o any rows
   WHERE e.eno=1000;

INSERT INTO TABLE
   (SELECT e.projects FROM employees e WHERE e,eno = 1000)
   VALUES (proj_t(102, 'Project Pluto'));
```

Multilevel Collection Types

- Collection types whose elements are themselves another collection type.

- Possible multilevel collection types are:
  - Nested table of nested table type
  - Nested table of varray type
  - Varray of nested table type
  - Varray of varray type
  - Nested table or varray of a user-defined type that has an attribute that is a nested table or varray type

Multilevel Collection Types Example

- Models a system of stars in which each star has a collection of the planets revolving around it, and each planet has a collection of its satellites.
  - CREATE TYPE sat_t AS OBJECT (name VARCHAR2(20), orbit NUMBER);

    /
  - CREATE TYPE sat_ntt AS TABLE OF sat_t

    /
  - CREATE TYPE planet_t AS OBJECT (name VARCHAR2(20), mass NUMBER, satellites sat_ntt)

    /

Multilevel Collection Types Example… (contd.)

- CREATE TYPE planet_ntt AS TABLE OF planet_t;
  /
- CREATE TYPE star_t AS OBJECT (name VARCHAR2(20), age NUMBER, planets planet_ntt)
  /
- CREATE TABLE stars_tab of start_t (name PRIMARY KEY)
  NESTED TABLE planets STORE AS planets_nttab
  (NESTED TABLE satellites STORE AS satellites_nttab);
- Separate nested table clauses are provided for the outer planets nested table and for the inner satellites one.

Multilevel Collection Types Example… (contd.)

- Inserting a new star called ‘Sun’…

- INSERT INTO stars VALUES('Sun',23,

```
nt_pl_t( planet_t( 'Neptune',10,
                nt_sat_t(satellite_t('Proteus',67),
                         satellite_t('Triton',82))),
            planet_t('Jupiter',189,
                nt_sat_t(satellite_t('Callisto',97),
                         satellite_t('Ganymede', 22)) ) ));
```

Multilevel Collection Types Example… (contd.)

- Inserting a planet called ‘Saturn’ to the star ‘Sun’…

- INSERT INTO TABLE(SELECT planets FROM stars WHERE name = 'Sun')

```
  VALUES ('Saturn', 56,
    nt_sat_t(
      satellite_t('Rhea', 83)
    )
  );
```

Multilevel Collection Types Example… (contd.)

- Inserting a satellite called ‘Miranda’ to planet ‘Uranus’ of the star ‘Sun’…

- INSERT INTO TABLE(

```
SELECT p.satellites
FROM TABLE( SELECT s.planets
                   FROM stars s
                   WHERE s.name = 'Sun') p
WHERE p.name = 'Uranus')
VALUES ('Miranda', 31);
```

Summary

- Collection Types
  - VARRAYs
  - Nested Tables
- DDL, DML and SELECTs on Collection Types
- Multilevel collection types

Database Systems

ORDB: Methods and Inheritance

Last Week

- Nested Collections
  - VARRAYs and Nested tables
    - Storing
    - Querying
    - Manipulating

- Any questions?

Encapsulation and UDTs: Methods

- Functions or procedures declared in an object type definition to implement behaviour of objects.
  - Declared in a CREATE TYPE statement and Defined in a CREATE TYPE BODY statement.
- Methods written in PL/SQL or Java are stored in the database.
  - preferable for data-intensive procedures and short procedures that are called frequently.
- Procedures in other languages, such as C, are stored externally.
  - preferable for computationally intensive procedures that are called less frequently.

Member Method

- Define a member method in the object type for each operation an object of that type should perform.
- Example: Add a method priceInYen to MenuType.
- CREATE TYPE MenuType AS OBJECT (

```
  bar REF BarType,
  beer REF BeerType,
  price FLOAT,
  MEMBER FUNCTION priceInYen(rate IN FLOAT)
    RETURN FLOAT
)
/
```

Example: Type Body

```
CREATE TYPE BODY MenuType AS
MEMBER FUNCTION
priceInYen(rate FLOAT)
RETURN FLOAT IS
   BEGIN
     RETURN rate * SELF.price;
   END;
END;
/
CREATE TABLE Sells OF MenuType;
```

Some Points to Remember

- SELF is a built-in parameter that denotes the object instance on which the method is currently being invoked.
- Member methods can reference the attributes and methods of SELF without using a qualifier.
  - The SELF bit in SELF.price is optional.
- Many methods will take no arguments.
  - In that case, do not use parentheses after the function name.
- The body can have any number of function definitions, separated by semicolons.
  - The body must include all the functions;

Adding a new method

- Use ALTER TYPE to add a method:

```
ALTER TYPE MenuType
    ADD MEMBER
    FUNCTION
    priceInUSD(rate FLOAT)
    RETURN FLOAT
    CASCADE;
```

```
CREATE OR REPLACE TYPE BODY
    MenuType AS
    MEMBER FUNCTION
    priceInYen(rate FLOAT)
    RETURN FLOAT IS
        BEGIN
            RETURN rate * SELF.price;
        END priceInYen;
    MEMBER FUNCTION
    priceInUSD(rate FLOAT)
    RETURN FLOAT IS
        BEGIN
            RETURN rate * SELF.price;
        END priceInUSD;
END;
/
```

Example of Method Use

- Use an alias for the object followed by a dot, the name of the method, and argument(s) if any.
- EXAMPLE:

```
SELECT s.beer.name, s.priceInYen(106.0)
FROM Sells s
WHERE s.bar.name = 'Joe''s Bar';
```

- Use parentheses, even if a method has no arguments.
  - E.g., select e.ename, e.age()
    from oremp e;
  - Assume age() is computed from attribute birthdate of the object type.

Object Comparison

- The values of scalar data types such as CHAR or REAL have a predefined order.
- But, instances of an object type have no predefined order.
- To compare two items of a user-defined type, define an order relationship using a *map* or an *order* method.
  - At most one map method (or one order method) for an object type.

Map Methods

- Compare objects by mapping object instances to a scalar type.
  - DATE, NUMBER, VARCHAR, etc.
- Example: For an object type called RECTANGLE, the map method AREA can return its (HEIGHT * WIDTH) .
  - Then two rectangles can be compared by their areas.

Map Method

- A parameter-less member function that uses the MAP keyword.
- If an object type defines one, the method is called automatically to evaluate
  - comparisons such as obj_1 > obj_2 and
  - comparisons implied by the DISTINCT, GROUP BY, and ORDER BY clauses.

Example

```
CREATE TYPE Rectangle_type AS OBJECT
( length NUMBER,
  width NUMBER,
 MAP MEMBER FUNCTION area RETURN NUMBER
);
CREATE TYPE BODY Rectangle_type AS MAP MEMBER
   FUNCTION area RETURN NUMBER IS
   BEGIN
     RETURN length * width;
  END area;
END;
```

Example

```
CREATE TABLE rectangles OF Rectangle_type;
INSERT INTO rectangles VALUES (1,2);
INSERT INTO rectangles VALUES (2,1);
INSERT INTO rectangles VALUES (2,2);
SELECT DISTINCT VALUE(r) FROM rectangles r;
   VALUE(R)(LEN, WID)
---------------------
  RECTANGLE_TYP(1, 2)
  RECTANGLE_TYP(2, 2)
```

Order Methods

- Order methods make direct object-to-object comparisons.
- A function with one declared parameter for another object of the same type.
- Definition of this method must return
  - < 0 if "self " is less than the argument object.
  - 0 if "self " is equal to the argument object.
  - > 0 if "self " is greater than the argument object.

Order Methods

- Called automatically whenever two objects need to be compared.

- Useful where comparison semantics may be too complex to use a map method.
  - E.g., to compare images, create an order method to compare by their brightness or number of pixels.

Example

- An order method that compares customers by customer ID:

- CREATE TYPE Customer_typ AS OBJECT
  (id NUMBER,
    name VARCHAR2(20),
    addr VARCHAR2(30),
    ORDER MEMBER FUNCTION match (c Customer_typ) RETURN INTEGER);
  /

Example

```
CREATE TYPE BODY Customer_typ AS
ORDER MEMBER FUNCTION match (c Customer_typ)
RETURN INTEGER IS
BEGIN
    IF id < c.id THEN RETURN -1; -- any num <0
    ELSIF id > c.id THEN RETURN 1; -- any num >0
    ELSE RETURN 0;
    END IF;
END;
END;
 /
```

On Comparison Methods

- In defining an object type, you can specify either a map method or an order method for it, but not both.

- If an object type has no comparison method, Oracle can compare two objects of that type only for equality or inequality.
  - Two objects of the same type count as equal only if the values of their corresponding attributes are equal.

On Comparison Methods

- When sorting or merging a large number of objects, use a map method.
  - One call maps all the objects into scalars, then sorts the scalars.
  - An order method is less efficient because it must be called repeatedly (it can compare only two objects at a time).

Methods on Nested Tables

```
CREATE TYPE proj_t AS OBJECT (projno number,
    Projname varchar(15));
CREATE TYPE proj_list AS TABLE OF proj_t;
CREATE TYPE emp_t AS OBJECT
   ( eno number,
       projects proj_list,
     MEMBER FUNCTION projcnt RETURN INTEGER
   );
```

Methods on Nested Tables

```
CREATE OR REPLACE TYPE BODY emp_t AS MEMBER
  FUNCTION projcnt RETURN INTEGER IS
    pcount INTEGER;
    BEGIN
      SELECT count(p.projno) INTO pcount
      FROM TABLE(self.projects) p;
      RETURN pcount;
    END;
  END;
  /
```

Methods on Nested Tables

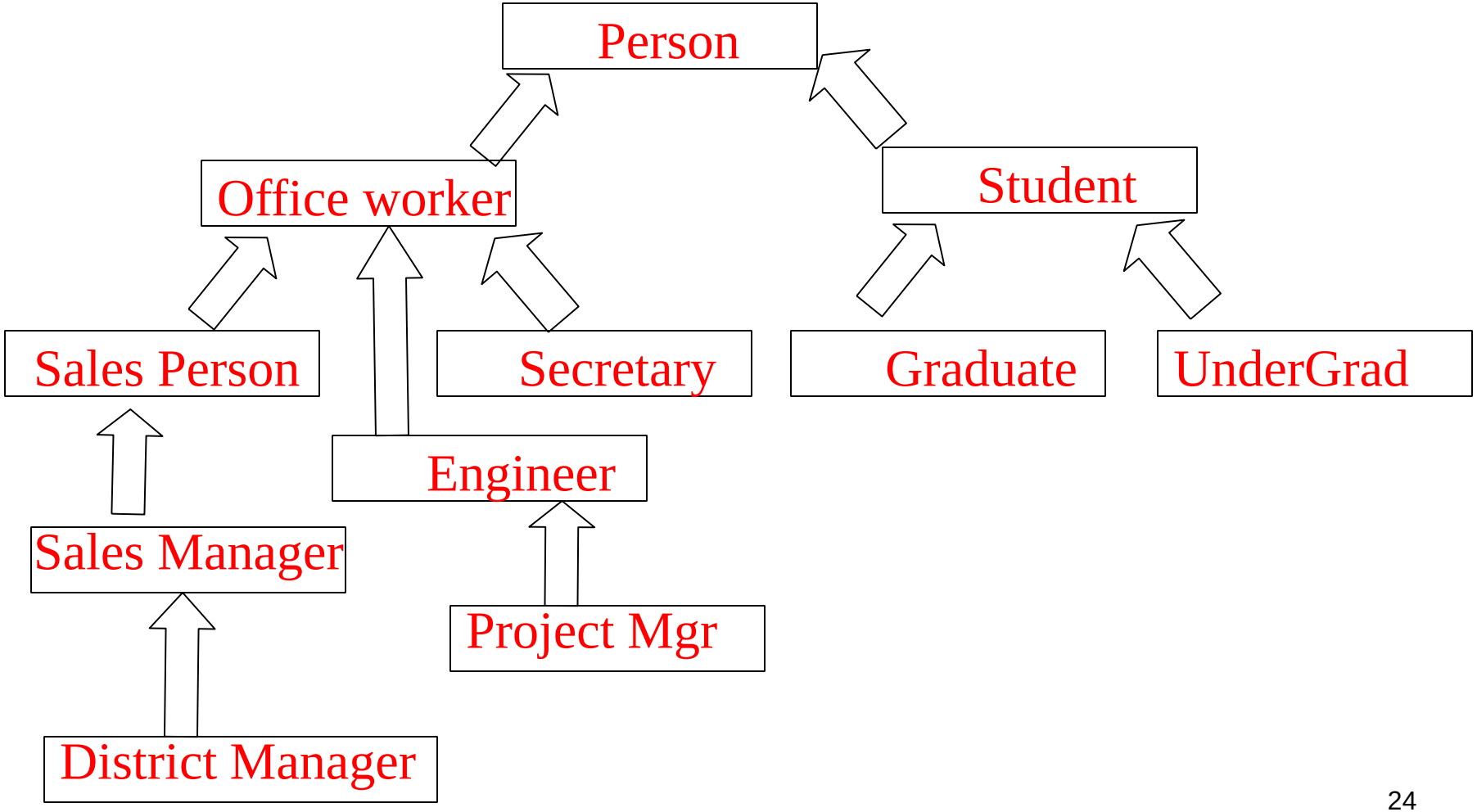
```
CREATE TABLE emptab OF emp_t
  (Eno PRIMARY KEY)
   NESTED TABLE projects STORE AS emp_proj_tab;

SELECT e.eno, e.projcnt() projcount
FROM emptab e;
```

Inheritance

- A natural model for organising information.
  - e.g. captures the fact that sales managers are also salespeople.
- Methods and representation can be shared.
  - Reduces redundancy.
- New types and objects can be defined in existing hierarchies rather than from scratch.
  - Increases flexibility and extensibility.

Example: Person hierarchy



Inheritance in Oracle

- It consists of a parent base type, or supertype, and one or more levels of child object types, or subtypes.
- Subtypes in a hierarchy are connected to their supertypes by inheritance.
  - subtypes automatically acquire the attributes and methods of their parent type.
  - any attribute or method updated in a supertype is automatically updated in subtypes also.

Specializing Subtypes

- Add new attributes the supertype does not have.
- A subtype cannot drop or change the type of an attribute it inherits from its parent.
- Add new methods that the parent does not have.
- Override the implementation of a parent method.

Specializing Subtypes

- Change the implementation of some methods a subtype inherits.
  - E.g., a shape object type might define a method calculate_area().
  - Two subtypes, rectangular and circular, might implement this method in a different way.

FINAL and NOT FINAL Types

- To permit subtypes, the object type must be defined as not final.
  - By including the keyword NOT FINAL in the type declaration.
  - By default, an object type is final.

- Example
  - CREATE TYPE Person_type AS OBJECT

```
( pid NUMBER,
  name VARCHAR2(30),
  address VARCHAR2(100) )   NOT FINAL;
```

  - Subtypes of Person_type can be defined.

Altering object type

- You can change a final type to a not final type and vice versa with an ALTER TYPE statement.
  - If a NOT FINAL type has no current subtypes.

- For example,
  - ALTER TYPE Person_type FINAL;

Creating Subtypes

- Use a CREATE TYPE statement with an UNDER parameter to specify the parent type:

```
CREATE TYPE Student_type UNDER Person_type
   ( deptid NUMBER,
      major VARCHAR2(30)) NOT FINAL;
/
```

  - Student_type inherits all the attributes and methods declared in or inherited by Person_type.
- New attributes in a subtype must have different names from the attributes or methods in all its supertypes in the type hierarchy.

Multiple child types

- A type can have multiple child subtypes, and these can also have subtypes.
- Example:

```
CREATE TYPE Employee_type UNDER Person_type
   ( empid NUMBER,
       mgr VARCHAR2(30)
    );
/
```

  - In addition to student_typ under person_type given earlier

Subtype under another subtype

- The new subtype inherits all the attributes and methods of its parent type, both declared and inherited.
- Example:

```
CREATE TYPE PartTimeStudent_type UNDER
    Student_type
 (numhours NUMBER);
```

Table of supertype

- Creating a supertype table

```
Create table person_tab of person_type
   (pid primary key);
```

- Inserting a subtype object/row

```
Insert into person_tab values
   ( student_type(4, 'Edward Learn',
         '65 Marina Blvd, Ocean Surf, WA, 6725',
         40, 'CS')
  );
```

Selecting all instances

- Using VALUE() function to select all instances of a super type:
  - Select all persons such as employees, students, etc. in the table:
- SELECT VALUE(p) FROM person_tab p;

```
VALUE(P)(PID, NAME, ADDRESS)
-----------------------------------------------
Person_type(21937, 'Fred', '4 Ambrose Street')
Student_type(27362, `Peter', … , 21, 'Oragami')
PartTimeStudent_type(2134, 'Jack',…, 13, 'Physics',
  5)
Person_type(21362, 'Mary', …)
Student_type(18437, `Susan', … , 13, 'Maths')
PartTimeStudent_type(4318, 'Jill',…, 21, 'Pottery',
  2)
Person_type(39374, 'George', …)
```

Selecting instances

- From student type and its subtypes

```
SELECT VALUE(s)
   FROM person_tab s
   WHERE VALUE(s) IS OF (Student_type);
```

```
VALUE(P)(PID, NAME, ADDRESS)
-----------------------------------------------
Student_typ(27362, `Peter', … , 21, 'Oragami')
PartTimeStudent_type(2134,'Jack',…,13,'Physics',
   5)
Student_typ(18437, `Susan', … , 13, 'Maths')
PartTimeStudent_type(4318,'Jill',…,21,'Pottery',
   2)
```

Selecting instances

- From student type but not from subtypes

```
SELECT VALUE(s)
FROM person_tab s
WHERE VALUE(s) IS OF (ONLY student_type);
```

```
VALUE(P)(PID, NAME, ADDRESS)
-----------------------------------------------
Student_typ(27362, `Peter', … , 21, 'Oragami')
Student_typ(18437, `Susan', … , 13, 'Maths')
```

Selecting a Subtype Attribute

- TREAT() function to make the system treat each person as a part-time student to access the subtype attribute numhours:

```
SELECT Name, TREAT(VALUE(p) AS
PartTimeStudent_type).numhours hours
FROM person_tab p
WHERE VALUE(p) IS OF (ONLY PartTimeStudent_type);
```

```
NAME        hours
-----------------------------------------------
Jack        5
Jill        2
```

NOT INSTANTIABLE Types

- Use this option with types intended solely as supertypes of specialized subtypes.
  - CREATE TYPE Address_typ AS OBJECT(...) NOT INSTANTIABLE NOT FINAL;*
  - CREATE TYPE AusAddress_typ UNDER Address_typ(...);
  - CREATE TYPE IntlAddress_typ UNDER Address_typ(...);

* You cannot create instances of the Address_typ only (similar to "*abstract classes*" in OO)

NOT INSTANTIABLE Methods

- Use this option to declare a method in a type without implementing it there.
  - A type that contains a non-instantiable method must itself be declared not instantiable.
  - CREATE TYPE T AS OBJECT (

    x NUMBER,

    NOT INSTANTIABLE MEMBER FUNCTION func1()*
    RETURN NUMBER) NOT INSTANTIABLE NOT FINAL;

* The type body for T does not contain a definition for func1

NOT INSTANTIABLE Methods

- Define a method as non-instantiable if every subtype is to override the method in a different way.

- If a subtype does not implement every inherited non-instantiable method, the subtype must be declared not instantiable.
  - A non-instantiable subtype can be defined under an instantiable supertype.

FINAL and NOT FINAL Methods

- If a method is declared to be final, subtypes cannot override it by providing their own implementation.
  - Unlike types, methods are not final by default.
  - They must be explicitly declared to be final.
- An overriding method is specified in a CREATE TYPE BODY statement.

Example

```
CREATE TYPE MyType AS OBJECT
   (  ...,
      MEMBER PROCEDURE Print,
      FINAL MEMBER FUNCTION foo(x NUMBER) …, ...
   )   NOT FINAL;
/
CREATE TYPE MySubType UNDER MyType
   ( ...,
     OVERRIDING MEMBER PROCEDURE Print,
      ...);
/
```

Overloading Methods

- A subtype can add new methods that have the same names as methods it inherits.
  - Methods that have the same name but different signatures in a type are called overloads.
  - The compiler uses the methods' signatures to tell them apart.

Example: Overloading Methods

```
CREATE TYPE MyType AS OBJECT
  ( ...,
    MEMBER FUNCTION fun(x NUMBER)…,
    ...) NOT FINAL;
/
CREATE TYPE MySubType UNDER MyType
  ( ...,
     MEMBER FUNCTION fun(x DATE) …,
    ...);
/
```

- Same function name, different signature

Summary

- Nested Collections
  - Varrays and nested tables
    - Storing
    - Querying
    - Manipulating
- Inheritance in Oracle
  - FINAL/NOT FINAL
  - Subtypes (UNDER)
  - Getting at particular subtypes
  - INSTANTIABLE/NOT INSTANTIABLE (Types and methods)
  - Overriding & Overloading

Database Systems

ORDB: Triggers and ER-ORDB Mapping

Last Week

- Methods
  - Member methods
  - Comparison methods (MAP & ORDER)
  - Methods on nested tables
- Inheritance
  - Type inheritance (UNDER, FINAL, NOT FINAL)
  - INSTANTIABLE/NOT INSTANTIABLE Types and Methods
  - Methods (Overriding, Overloading)

- Any questions?

Triggers

- Procedures stored in the database and implicitly run, or *fired*, when something happens.
  - INSERT, UPDATE, or DELETE on a table or view (DML Triggers).
  - System and other data events on DATABASE and SCHEMA (System Triggers).

Triggers on Object Tables

- Triggers can be defined on an object table just as on other tables.

- Example:
  - CREATE TYPE location as OBJECT (campus VARCHAR2(20), building NUMBER, room NUMBER);
  - CREATE TYPE staff as OBJECT(sid NUMBER, name VARCHAR2(100), address VARCHAR2(100), office location);
  - CREATE TABLE staff_tab OF staff (sid PRIMARY KEY);

Example

- CREATE TABLE movement (sid NUMBER, old_office location, new_office location);

- CREATE TRIGGER trig1
  BEFORE UPDATE OF office ON staff_tab
  FOR EACH ROW
  WHEN (new.office.campus = 'Bentley')
  BEGIN
    IF :new.office.building = 314 THEN
     INSERT INTO movement VALUES (:old.sid,:old.office, :new.office);
   END IF;
  END;

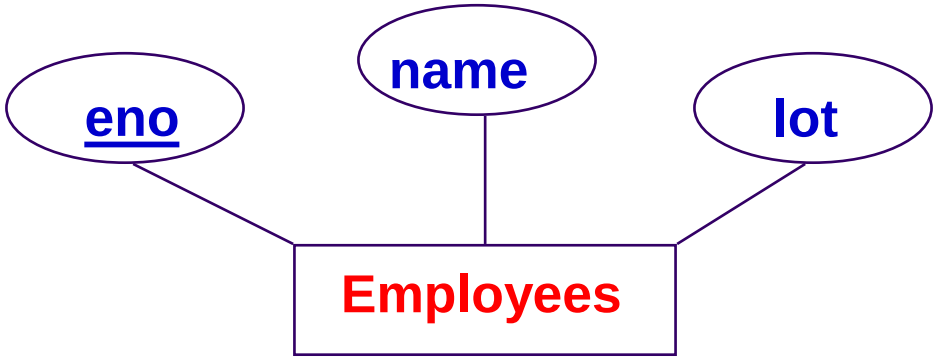
Database design for ORDB

- ER model may be extended with more constructs such as:
  - Multi-valued attributes (e.g., locations of a store)
  - Structured-type attributes (e.g., address made up of house no, street, suburb, etc.)
  - Collection-type attributes (e.g., list of dates)

Mapping ERD to ORDB

- Map each ER entity type to ORDB type
  - Include all attributes of ER type in ORDB type
  - Declare a table for each ORDB type and specify key attributes as (primary) keys
  - Sub-types in ISA hierarchy inherit attributes of super type

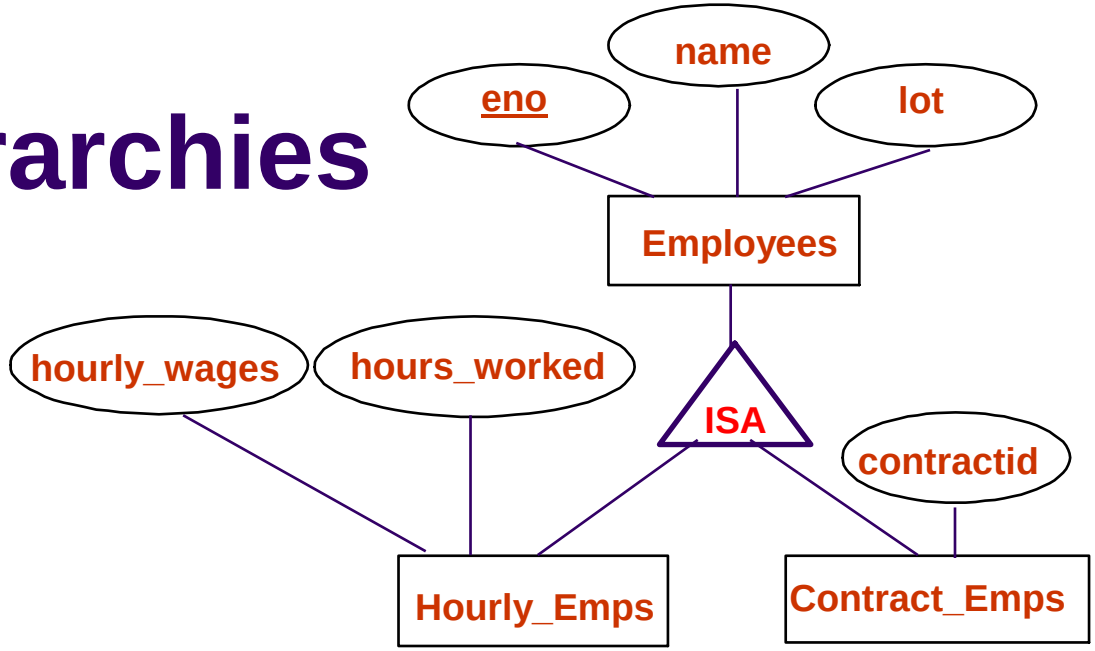
Entity Types



- Entity type can be mapped to a type easily.

```
CREATE TYPE Emp_t
        (eno CHAR(11),
        name CHAR(20),
        lot  INTEGER);
Create table employees of Emp_t(
    eno primary key);
```

ISA Hierarchies

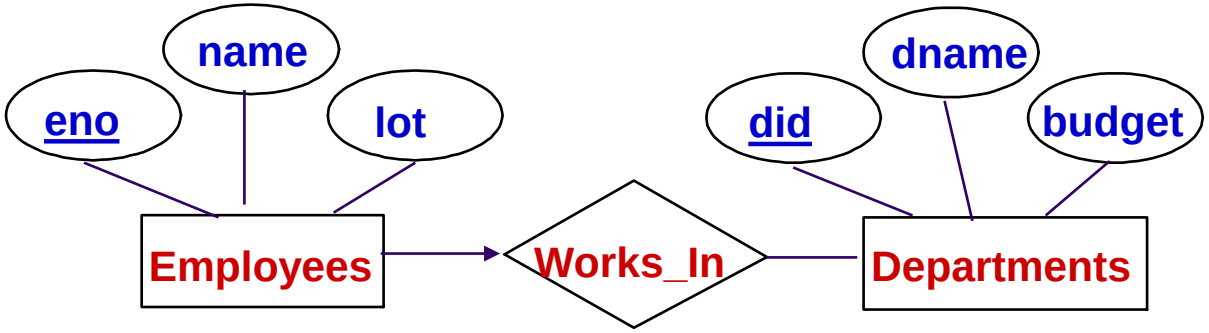



```
CREATE TYPE emp_t AS OBJECT (…) NOT FINAL;
CREATE TYPE hourly_emps_t UNDER emp_t
    (hourly_wages number(6,2),
     hours_wkd number(4,2));
CREATE TYPE contract_emps_t UNDER emp_t
    (contractid char(10));
```

Mapping ERD to ORDB

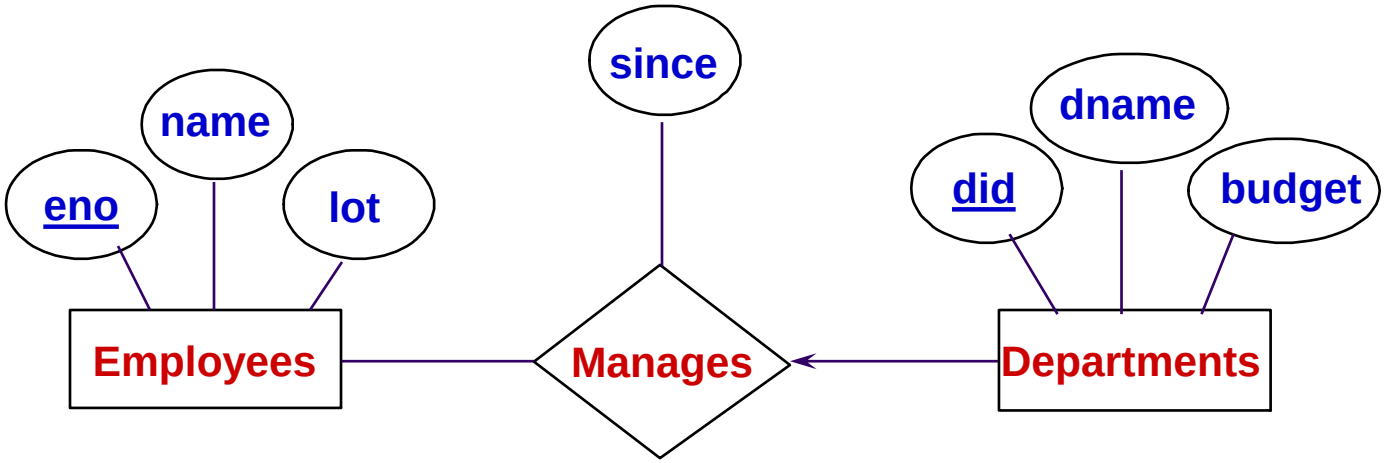
- Binary relationship types
  - Cardinality 1:1, N:1, or M:N
- Weak entity types
  - Similar to regular entity types
  - Alternative mapping possible if the weak entity does not participate in any other relationship
- N-ary relationships with n>2
  - Mapped as a separate type
  - Appropriate references to each participating class
- Methods for types (not in ER)
  - UML?

Relationship sets



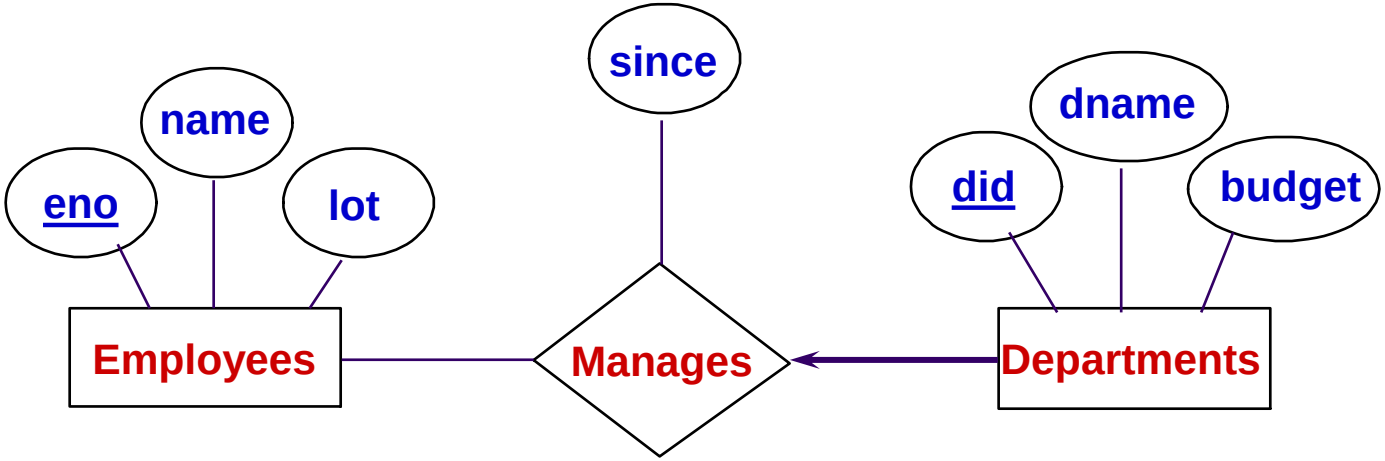
- Add a reference attribute for a binary relationship with key constraint into the corresponding object type.
  - CREATE TYPE Emp_t
    (eno CHAR(11),
    name CHAR(20),
    lot INTEGER,
    workdept ref dept_t);

Translating Key Constraints



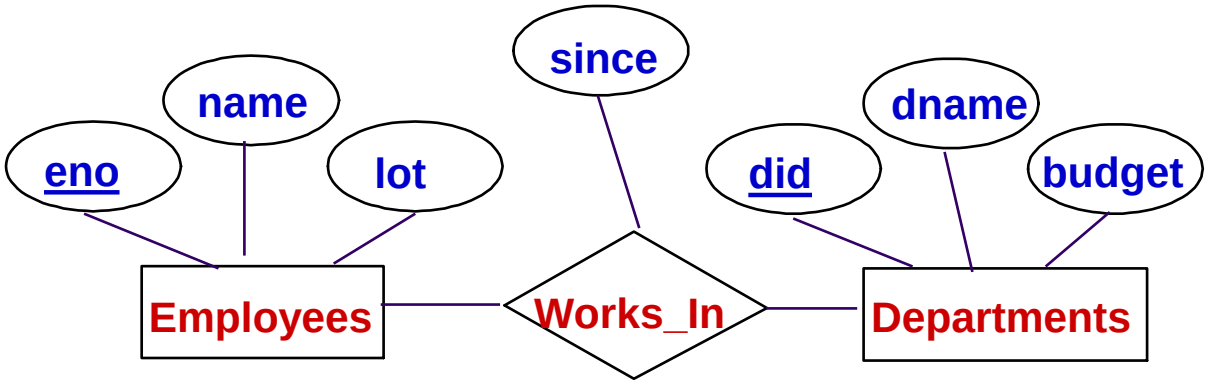

```
CREATE TYPE  Dept_t as object
   ( did  INTEGER,
     dname  CHAR(20),
     budget  REAL,
     mgr  ref emp_t
     since  DATE);
```

Participation Constraints



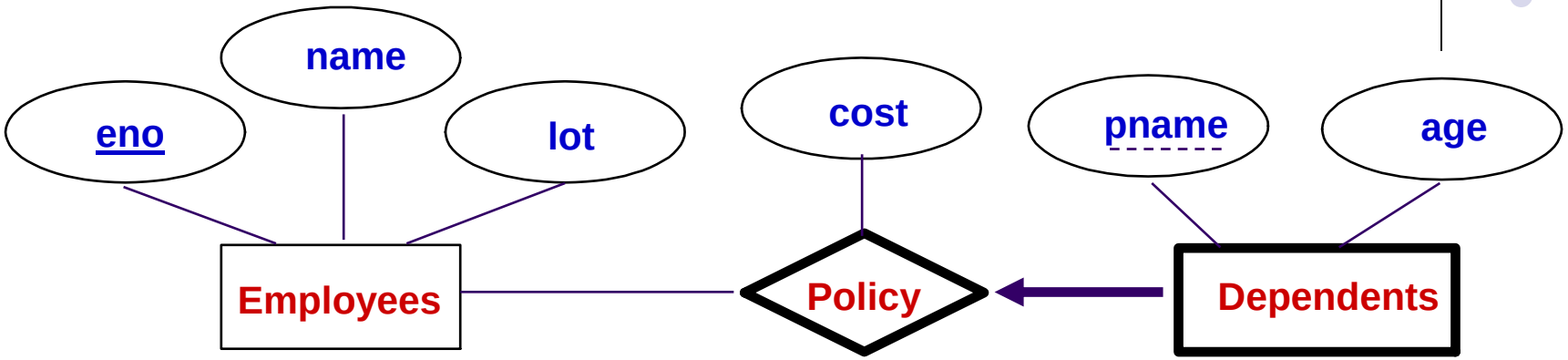

```
CREATE TABLE  Department of Dept_t
   ( did  primary key,
     mgr NOT NULL REFERENCES Employees
   );
```

M:N Relationship set




```
CREATE TYPE WorksIn_t AS OBJECT(
    emp  REF emp_t,
    dept  REF dept_t,
    since  DATE);
CREATE TABLE worksIn OF WorksIn_t(
    emp REFERENCES Employees,
    dept REFERENCES Departments);
```

Weak Entities

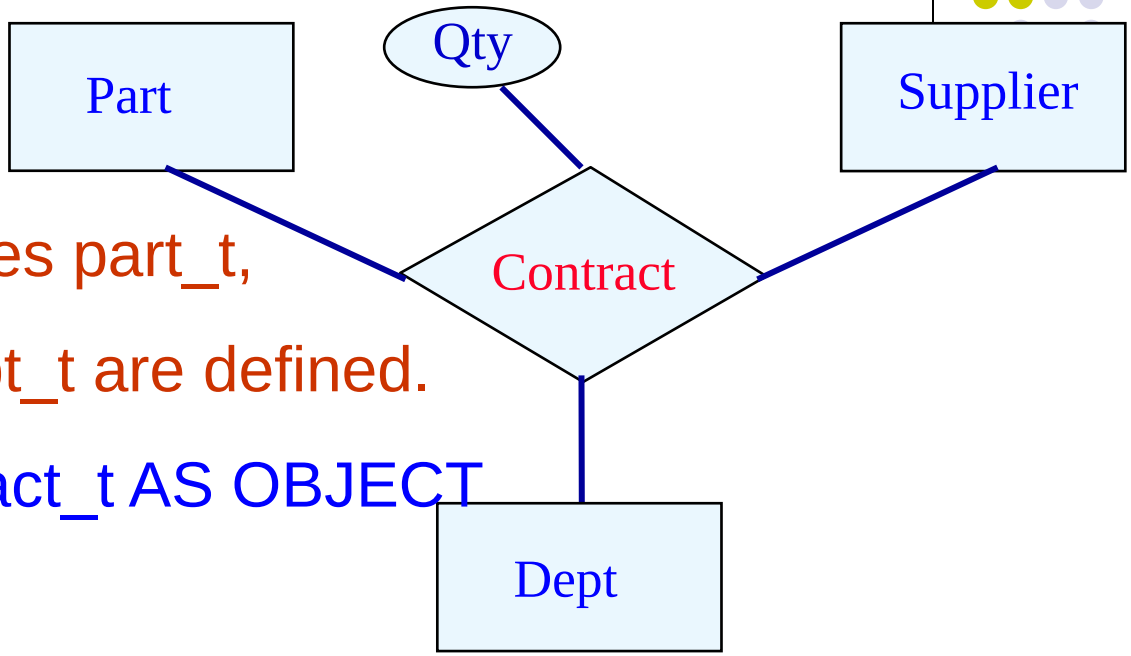


- Option 1: Map like a regular entity
- Option 2: Weak entity types that do not participate in any relationships except their identifying relationship with owning entity.
  - Use nested collection type for the weak entity information.
- Option 3: Weak entity type objects are not to be referenced by attributes of any other object.
  - Use nested collection type for the weak entity information.

Weak Entity Sets: Option 2

```
Create type depend_t as object(
   Pname varchar(12),
   Age integer,
   policyCost number (6,2));
Create type policy_t as table of depend_t;
Create type emp_t as object( …,
   dependents policy_t)
```

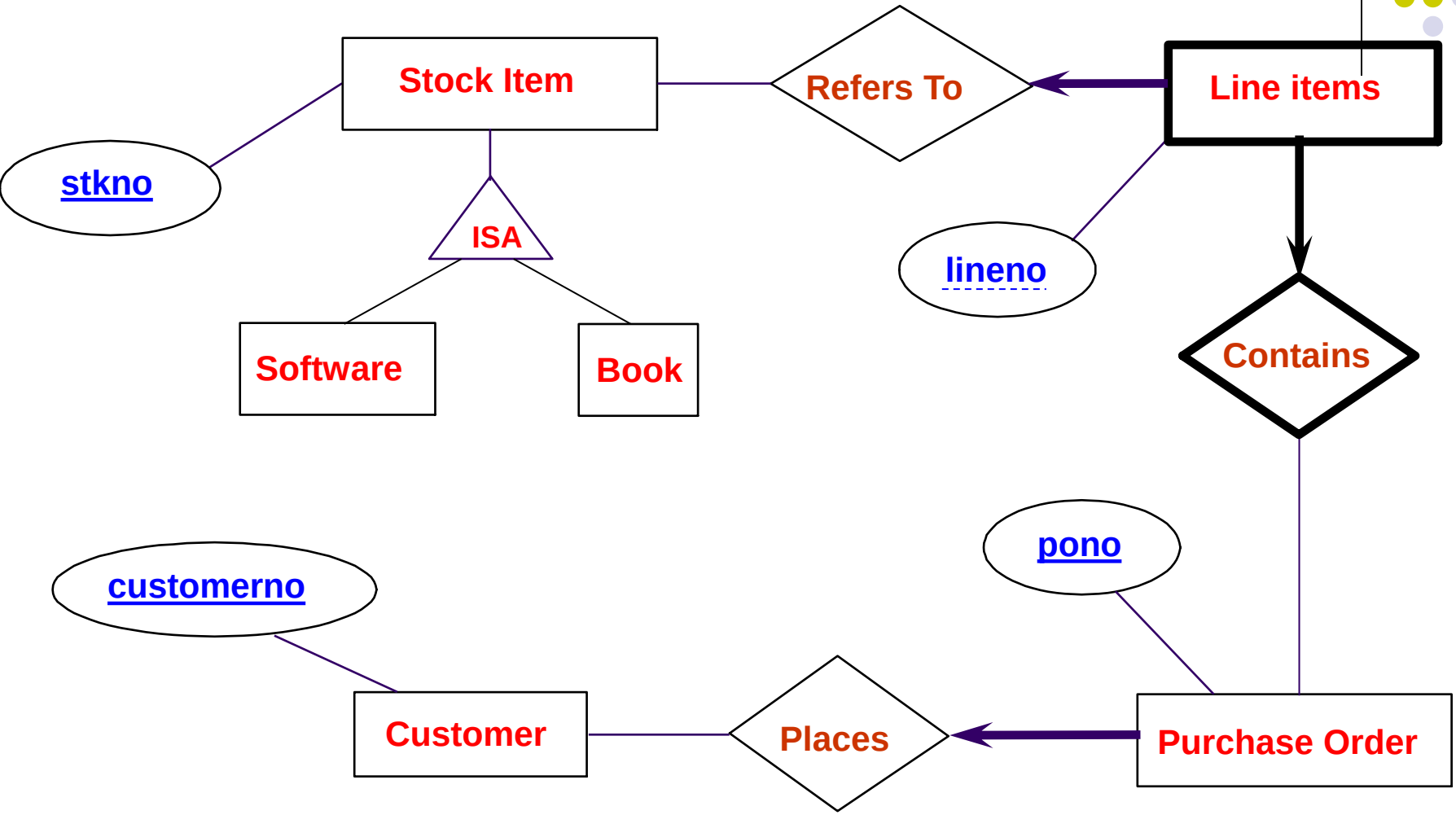
N-ary relationship



- Assuming object types part_t, supplier_t, and dept_t are defined.

```
CREATE TYPE Contract_t AS OBJECT
  (part REF part_t,
   supplier REF supplier_t,
   dept REF dept_t,
   qty NUMBER(6,2)
  );
```

Cyber Shop Example



Mapping to OR Schema

- Regular entities to types
  - Customer_typ (custNo, …)
  - StockItem_typ (StockNo, …)
  - PurchaseOrder_typ (PONo, …)
- Inheritance
  - StockItem_typ (StockNo, …) NOT FINAL
  - Book_typ(…) under StockItem_typ
  - Software_typ(…) under StockItem_typ

Mapping to OR Schema

- Weak entity type
  - LineItem_typ (LineItemNo, …)
  - LineItemList_ntabtyp AS TABLE OF LineItem_typ
  - PurchaseOrder_typ (PONo, …,
    LineItemList_ntab LineItemList_ntabtyp)
- Binary relationship with key constraint
  - PurchaseOrder_typ (PONo, …,
    LineItemList_ntab LineItemList_ntabtyp,
    Cust_ref REF Customer_typ)
  - LineItem_typ (LineItemNo, …,
    Stock_ref REF StockItem_typ)

Final types

- Customer_typ (custNo, …)
- StockItem_typ (StockNo, …)
- Book_typ(…) under StockItem_typ
- Software_typ(…) StockItem_typ
- LineItem_typ (LineItemNo, …,
  Stock_ref REF StockItem_typ)
- LineItemList_ntabtyp AS TABLE OF LineItem_typ
- PurchaseOrder_typ (PONo, …,
  LineItemList_ntab LineItemList_ntabtyp,
  Cust_ref REF Customer_typ)

Tables and constraints

- Customers of Customer_typ (custNo primary key)
- Stocks of StockItem_typ (StockNo primary key)
- Orders of PurchaseOrder_typ (PONo primary key, Cust_ref references Customers)
  nested table LineItemlist

Summary

- Trigger Example
- Mapping ER to ORDB
  - Mapping various ER features to ORDB
  - Example

Database Systems

XML Databases

Last Lecture

- Query Optimization
- Any questions?

Outline

- Introduction to XML
  - XML Document, XML Schema
- XML data storage
  - Native XML Databases
    - XML data type
  - XML enable Databases
- XML data retrieve
  - XPath
  - XQuery
    - FLWOR

XML

- XML = eXtensible Markup Language.
- While HTML uses tags for *formatting* (e.g., “italic”), XML uses tags for *semantics* (e.g., “this is an address”).
- Key idea: create tag sets for a domain, and translate all data into properly tagged XML documents.

Example: XML Document

```
…
<catalog>
  <product dept="WMN">
    <number>557</number>
    <name language="en">Fleece Pullover</name>
    <colorChoices>navy black</colorChoices>
  </product>
  <product dept="ACC">
…
```

Well-Formed XML

- Start the document with a *declaration*, surrounded by <?xml … ?> .

- Normal declaration is:

```
<?xml version = “1.0”?>
```

- Balance of document is a *root tag* surrounding nested tags.

Tags

- Tags, as in HTML, are normally matched pairs, as <FOO> … </FOO> .
- Tags may be nested arbitrarily.
- XML tags are case sensitive.

Example: Well-Formed XML

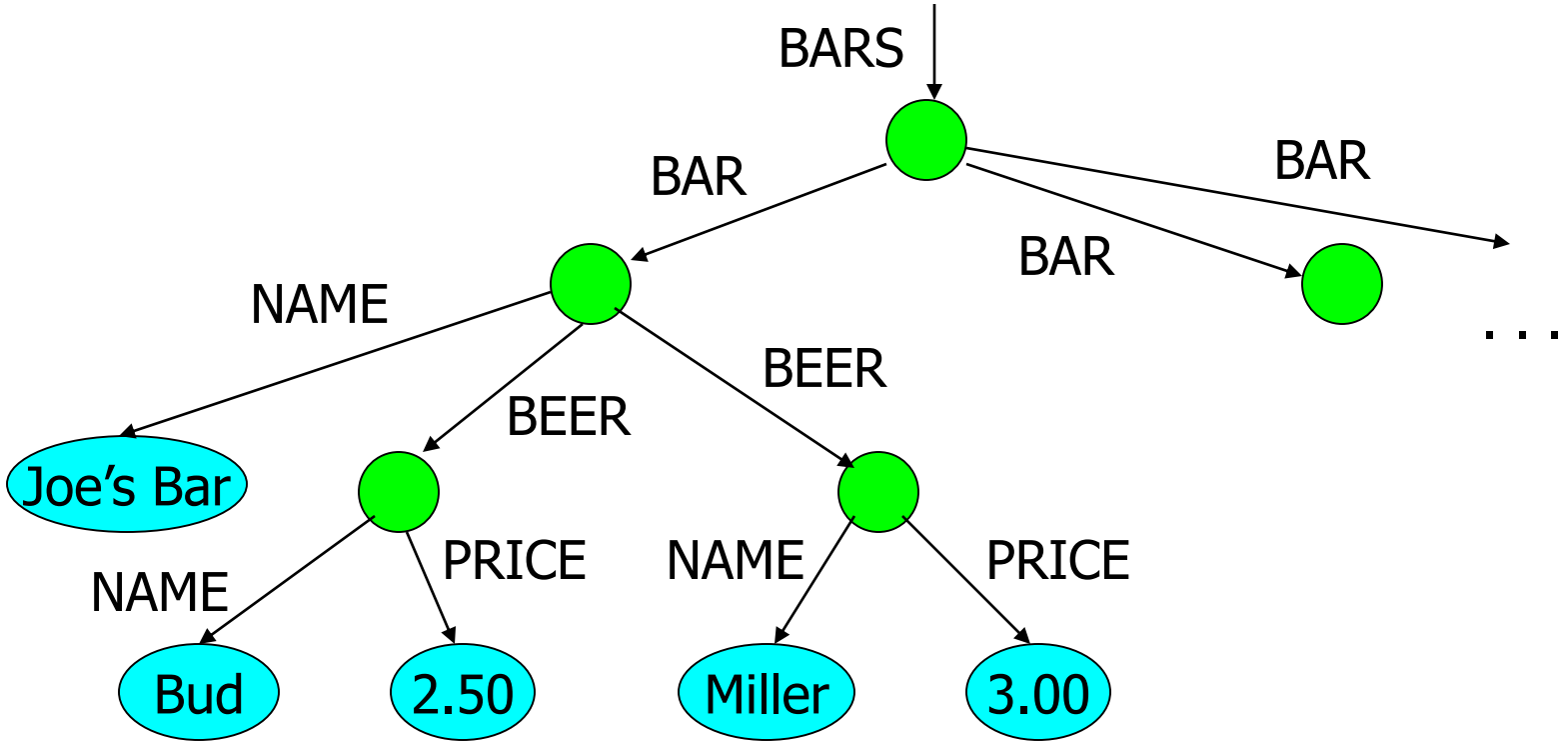
```
<?xml version = “1.0”?>
<BARS>
    <BAR><NAME>Joe’s Bar</NAME>
        <BEER><NAME>Bud</NAME>
                <PRICE>2.50</PRICE></BEER>
        <BEER><NAME>Miller</NAME>
                <PRICE>3.00</PRICE></BEER>
    </BAR>
    <BAR> …
</BARS>
```

A NAME subobject

A BEER subobject

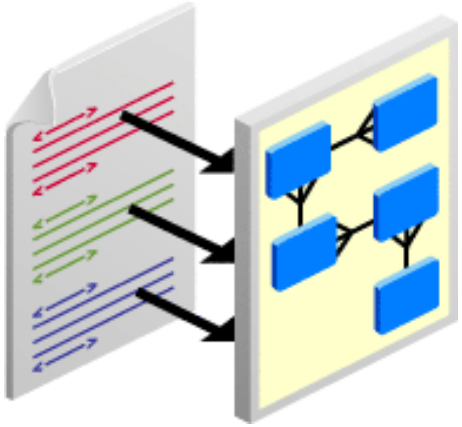
Example

- The <BARS> XML document is:



XML schema language

- Describe the structure and data content of an XML document
  - The description of an element consists of its name (tag), and a parenthesized description of any nested tags.
    - Includes order of sub tags and their multiplicity.
    - etc…
- Can be used to validate XML documents
- Type of schema languages
  - DTD (Document type Definition)
  - XML Schema
  - *RELAX NG*



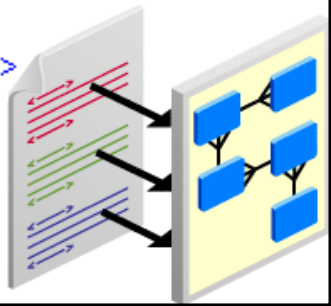
Example:
XML Schema and XML Document

```
<?xml version="1.0"?>
<People    xmlns="http://www.sliit.lk"
           xmlns:xsi="http://www.w3.org/2001/XMLSch
           xsi:schemaLocation="http://www.sliit.lk peo
  <Student>
    <Pid>p123</Pid>
    <Name>Kamal</Name>
    <Sex>Male</Sex>
    <StudentNo>MSC/CS/2004/001</StudentNo>
  </Student>
  <Student>
    <Pid>p124</Pid>
    <Name>Aamal</Name>
    <Sex>Male</Sex>
    <StudentNo>MSC/CS/2004/002</StudentNo>
  </Student>
  <Employee>
    <Pid>p123</Pid>
```

Xml doc

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" xm
http://www.sliit.lk" elementFormDefault="qualified">
  <xs:complexType name="PersonType" abstract="true">
    <xs:sequence>
      <xs:element name="Pid" type="xs:string"/>
      <xs:element name="Name" type="xs:string"/>
      <xs:element name="Sex" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="StudentType">
    <xs:complexContent>
      <xs:extension base="PersonType">
        <xs:sequence>
          <xs:element name="StudentNo" type="xs:string"/>
        </xs:sequence>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
  <xs:complexType name="EmployeeType">
    <xs:complexContent>
      <xs:extension base="PersonType">
        <xs:sequence>
```

XML Schema



XML Data Storage

- XML is de-facto standard for exchanging data
- Storing and accessing large XML data stores is gaining in importance

The main approaches…

- Native XML databases (Eg: Timber, Tamino, Xindice)
  - stores and retrieves XML data in its native form
  - contain new techniques for storage and retrieval
- XML-enabled databases (Eg: Oracle, MS SQL Server,DB2)
  - Use existing database technology to store XML data
  - Database technology has matured over the last three decades
  - **Advantage:** Use more powerful existing database technologies
- Both approaches have merit!
  - Schema is static (structured) → XML enabled DB approach is advantageous
  - Schema is not static (unstructured/semi-structured) → Native XML DB approach is advantageous

XML enabled approach

XML storage options

- Mapping between XML and existing data models supported by DBMSs
- Large object storage (CLOB, BLOB)

Native storage as XML data type

- Native storage as XML data type
  - XMLType data type - Oracle

    R. Murthy and S. Banerjee, "*XML Schemas in Oracle XML DB*", *VLDB,* 2003.
  - Xml Type - SQL Server 2005

    S. Pal, I. Cseri, O. Seeliger, G. Schaller, L. Giakoumakis and V. Zolotov, "*Indexing XML Data Stored in a Relational Database*", *VLDB*, 2004.
- Internal representation of stored data is preserve the XML content of the data, such as containment hierarchy, document order, element and attribute values, and so on
- Difference between Native XML databases and XML type are blurred

Untyped and typed XML data type

- XML values can be stored natively in an XML data type column, which can be typed according to a collection of XML schemas, or can be left untyped
- Use untyped XML data type under the following conditions:
  - You do not have a schema for your XML data
  - You have schemas but you do not want the server to validate the data
- Use typed XML data type under the following conditions:
  - You have schemas for your XML data and you want the server to validate your XML data according on the XML schemas
  - You want to take advantage of storage and query optimizations based on type information

Untyped and typed XML data type

For MS SQL Server,

Untyped XML Column in Table:

```
CREATE TABLE AdminDocs (
    id int  primary key,
    xDoc XML not null
    )
```

Schema collection

Typed XML Column in Table:

```
CREATE TABLE AdminDocs (
    id int primary key,
    xDoc XML (CONTENT myCollection)
    )
```

Untyped and typed XML data type

Inserting Data: Example

```
INSERT INTO AdminDocs VALUES (1,
'<book >
   <title>Writing Secure Code</title>
   <author>
      <first-name>Michael</first-name>
      <last-name>Howard</last-name>
   </author>
   <author>
      <first-name>David</first-name>
      <last-name>LeBlanc</last-name>
   </author>
   <price>39.99</price>
</book>')
```

How to Query XML Data?

- Main two ways to extract data
- Path expressions
  - great if you just want to copy certain elements and attributes as is
- XQuery expressions
  - XQuery extends XPath to a query language that has power similar to SQL.

Path Expressions

- *XPath* is a language for describing paths in XML documents.
  - i.e. Xpath 1.0 and Xpath 2.0

- Really think of the semistructured data graph and *its* paths.

Path Descriptors

- Simple path descriptors are sequences of tags separated by slashes (/).
- If the descriptor begins with /, then the path starts at the root and has those tags, in order.
- If the descriptor begins with //, then the path can start anywhere.

Value of a Path Descriptor

- Each path descriptor, applied to a document, has a value that is a sequence of elements.
- An *element* is an atomic value or a node.
- A *node* is matching tags and everything in between.
  - i.e., a node of the semistructured graph.

Example: /BARS/BAR/PRICE

```
<BARS>
   <BAR name = “JoesBar”>
      <PRICE theBeer = “Bud”>2.50</PRICE>
      <PRICE theBeer = “Miller”>3.00</PRICE>
   </BAR> …
   <BEER name = “Bud” soldBy = “JoesBar
      SuesBar …”/> …
</BARS>
```

/BARS/BAR/PRICE describes the set with these two PRICE elements as well as the PRICE elements for any other bars.

Example: //PRICE

```
<BARS>
   <BAR name = “JoesBar”>
      <PRICE theBeer = “Bud”>2.50</PRICE>
      <PRICE theBeer = “Miller”>3.00</PRICE>
   </BAR> …
   <BEER name = “Bud” soldBy = “JoesBar
      SuesBar …”/>…
</BARS>
```

//PRICE describes the PRICE Elements to appear within the document.

Wild-Card *

- A star (*) in place of a tag represents any one tag.
- Example: /*/*/PRICE represents all price objects at the third level of nesting.

Example: /BARS/*

```
<BARS>
  <BAR name = “JoesBar”>
      <PRICE theBeer = “Bud”>2.50</PRICE>
      <PRICE theBeer = “Miller”>3.00</PRICE>
  </BAR> …
  <BEER name = “Bud” soldBy = “JoesBar
      SuesBar …”/> …
</BARS>
```

/BARS/* captures all BAR and BEER elements, such as these.

Attributes

- In XPath, we refer to attributes by prepending @ to their name.
- Attributes of a tag may appear in paths as if they were nested within that tag.

Example: /BARS/*/@name

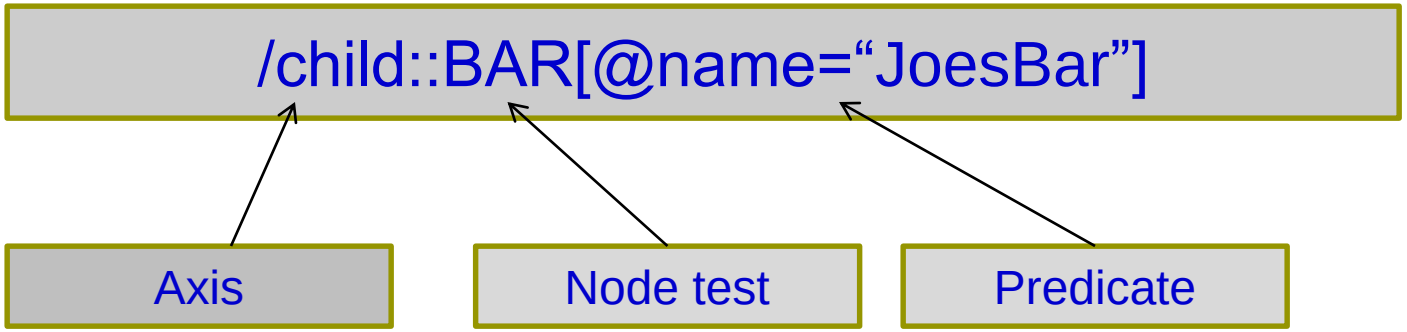
```
<BARS>
   <BAR name = “JoesBar”>
      <PRICE theBeer = “Bud”>2.50</PRICE>
      <PRICE theBeer = “Miller”>3.00</PRICE>
   </BAR> …
   <BEER name = “Bud” soldBy = “JoesBar
      SuesBar …”/> …
</BARS>
```

/BARS/*/@name selects all name attributes of immediate subelements of the BARS element.

Axes

- In general, path expressions allow us to start at the root and execute steps to find a sequence of nodes at each step.
- At each step, we may follow any one of several *axes*.

Axes



1. The axis (Optional)
   - Direction to navigate
2. The node test
   - The node of interest by name
3. Predicate
   - The criteria used to filter nodes

Example: Axes

- /BARS/BEER is really shorter way for /BARS/child::BEER .
- @ is really shorthand for the attribute:: axis.
  - Thus, /BARS/BEER[@name = “Bud”] is shorthand for

    /BARS/BEER[attribute::name = “Bud”]

More Axes

- Some other useful axes are:
  - parent:: = parent(s) of the current node(s).
  - descendant-or-self:: = the current node(s) and all descendants.
    - Note: // is really shorthand for this axis.
  - ancestor::, ancestor-or-self, etc.
  - the default axis is child:: --- go to all the children of the current set of nodes.

Predicates

- A condition inside […] may follow a tag.
- If so, then only paths that have that tag and also satisfy the condition are included in the result of a path expression.

Example: Selection Condition

- /BARS/BAR[PRICE < 2.75]/PRICE

```
<BARS>
  <BAR name = “JoesBar”>
      <PRICE theBeer = “Bud”>2.50</PRICE>
      <PRICE theBeer = “Miller”>3.00</PRICE>
  </BAR> …
```

The condition that the PRICE be < $2.75 makes this price but not the Miller price satisfy the path descriptor.

Example: Attribute in Selection

- /BARS/BAR/PRICE[@theBeer = “Miller”]

```
<BARS>
  <BAR name = “JoesBar”>
      <PRICE theBeer = “Bud”>2.50</PRICE>
      <PRICE theBeer = “Miller”>3.00</PRICE>
  </BAR> …
```

Now, this PRICE element is selected, along with any other prices for Miller.

```
<catalog>
  <product dept="WMN">
    <number>557</number>
    <name language="en">Linen Shirt</name>
    <colorChoices>beige sage</colorChoices>
  </product>
  <product dept="ACC">
    <number>563</number>
    <name language="en">Ten-Gallon Hat</name>
  </product>
  <product dept="ACC">
    <number>443</number>
    <name language="en">Golf Umbrella</name>
  </product>
  <product dept="MEN">
    <number>784</number>
    <name language="en">Rugby Shirt</name>
    <colorChoices>blue/white blue/red</colorChoices>
    <desc>Our <i>best-selling</i> shirt!</desc>
  </product>
</catalog>
```

Example: Predicates and Returned Elements

- Using number in a predicate does not mean that number elements are returned:

All product elements whose number child is less than 500:

```
product[number < 500]
```

All number elements whose value is less than 500:

```
product/number[. lt 500]
```

A period (".") is used to indicate the context item itself

Using Position in Predicates

- Can use a number in the predicate to indicate the position of the child

The 4th `product` child of `catalog`:

```
catalog/product[4]
```

The 4th child of `catalog` (regardless of its name):

```
catalog/*[4]
```

Using Position in Predicates

- More than one predicate can be used to specify multiple constraints in a step
  - evaluated left to right

Products whose number is less than 500 *and* whose department is ACC:

```
product[number < 500][@dept = "ACC"]
```

Of the products whose department is 'ACC', select the 2nd one:

```
product[@dept = "ACC"][2]
```

Take the 2nd product, if it's in the ACC department, select it:

```
product[2][@dept = "ACC"]
```

XQuery

- XQuery extends XPath to a query language that has power similar to SQL.
- Basic building blocks,

FLWOR Expression

Path Expression

```
for $d in distinct-values(doc("ord.xml")//item/@dept)
let $items := doc("ord.xml")//item[@dept = $d]
order by $d
return <department name="{$d}"
            totalQuantity="{sum($items/@quantity)}"/>
```

Element Constructor

Function Call

Variable Reference

Comparison Expression

```
<order num="00299432" date="2004-09-15" cust="0221A">
  <item dept="WMN" num="557" quantity="1" color="beige"/>
  <item dept="ACC" num="563" quantity="1"/>
  <item dept="ACC" num="443" quantity="2"/>
  <item dept="MEN" num="784" quantity="1" color="blue/white"/>
  <item dept="MEN" num="784" quantity="1" color="blue/red"/>
  <item dept="WMN" num="557" quantity="1" color="sage"/>
</order>
```

XQuery

- **Variables** are identified by a name preceded by a $

```
for $prod in
   (doc("cat.xml")//product)
return $prod/number
```

- Literal values can be expressed as:
  - strings (in single or double quotes)

```
doc("cat.xml")//product/@dept = "WMN"
```

  - numbers

```
doc("ord.xml")//item/@quantity > 1
```

Comments

- XQuery comments
  - Delimited by `(:` and `:)`
  - Anywhere insignificant whitespace is allowed
  - Do not appear in the results

```
(: This query... :)
```

- XML comments
  - May appear in the results
  - XML-like syntax

```
<!-- This element... -->
```

Clauses of a FLWOR Expression

- `for` clause
  - iteratively binds the `$prod` variable to each item returned by a path expression.
- `let` clause
  - binds the `$prodDept` variable to the value of the `dept` attribute
- `where` clause
  - selects nodes whose `dept` attribute is equal to "`WMN`" or "`ACC`"
- `return` clause
  - returns the `name` children of the selected nodes

```
for $prod in doc("cat.xml")//product
let $prodDept := $prod/@dept
where $prodDept = "ACC" or $prodDept = "WMN"
return $prod/name
```

for Clauses

- Iteratively binds the variable to each item returned by the `in` expression
- The rest of the expression is evaluated once for each item returned
- Multiple `for` clauses are allowed in the same FLWOR

expression after `in` can evaluate to any sequence

```
for $prod in doc("cat.xml")//product
```

Range expressions

- Create sequences of consecutive integers
- Use `to` keyword
  - `(1 to 5)` evaluates to a sequence of 1, 2, 3, 4 and 5
- Useful in `for` clauses to iterate a specific number of times

```
for $i in (1 to 5)
...
```

```
for $i in (1 to $prodCount)
...
```

Multiple Variables in a *for* clause

- Use commas to separate multiple `in` expressions
  - or multiple `for` clauses
- Return clause evaluated for every combination of variable values

```
for $i in (1, 2), $j in (11, 12)
return <eval>i is {$i} and j is {$j}</eval>
```

```
<eval>i is 1 and j is 11</eval>
<eval>i is 1 and j is 12</eval>
<eval>i is 2 and j is 11</eval>
<eval>i is 2 and j is 12</eval>
```

Positional variables in *for* clause

- Positional variable keeps track of the iteration number
- Use `at` keyword

```
for $prod at $i in doc("cat.xml")//
  product[@dept = "ACC" or @dept = "WMN"]
return <eval>{$i}. {data($prod/name)}</eval>
```

```
<eval>1. Linen Shirt</eval>
<eval>2. Ten-Gallon Hat</eval>
<eval>3. Golf Umbrella</eval>
```

let clauses

- Convenient way to bind a variable
  - avoids repeating the same expression many times
- Does not result in iteration

```
for $i in (1 to 3)
return <eval>{$i}</eval>
```

↓

```
<eval>1</eval>
<eval>2</eval>
<eval>3</eval>
```

```
let $i := (1 to 3)
return <eval>{$i}</eval>
```

↓

```
<eval>1 2 3</eval>
```

Multiple *for* and *let* clauses

- `for` and `let` can be repeated and combined

```
let $catDoc := doc("cat.xml")
for $prod in $catDoc//product
let $prodDept := $prod/@dept
where $prodDept = "ACC" or $prodDept = "WMN"
return $prod/name
```

where clause

- Used to filter results
- Can contain many subexpressions
- Evaluates to a boolean value
  - effective boolean value is used
- If `true`, return clause is evaluated

```
where $prod/number > 100
  and starts-with($prod/name, "L")
  and exists($prod/colorChoices)
  and ($prodDept="ACC" or $prodDept="WMN")
```

return clause

- The value that is to be returned

```
for $prod in doc("cat.xml")//product
return $prod/name
```

- Single expression only
  - can combine multiple expressions into a sequence

```
return <a>{$i}</a>
       <b>{$j}</b>
```

```
return (<a>{$i}</a>,
        <b>{$i}</b>)
```

order by Clause

- Only way to sort results in XQuery
- Use `order by` before `return` clause
- Order by
  - atomic values, or
  - nodes that contain individual atomic values
- Can specify multiple values to sort on

```
for $item in doc("ord.xml")//item
order by $item/@dept, $item/@num
return $item
```

Variable binding and referencing

- Variables are *bound* in the let/for clauses
- Once bound, variables can be *referenced* anywhere in the FLWOR

binding variables

```
for $prod in doc("cat.xml")//product
let $prodDept := $prod/@dept
where $prodDept = "ACC"
return $prod/name
```

referencing variables

- Values cannot be changed once bound
  - e.g. no `let $count := $count + 1`

Comparisons

- Two kinds:
  - *value* comparisons
    - `eq, ne, lt, le, gt, ge`
    - used to compare individual values
    - each operand must be a single atomic value or a node containing a single atomic value
  - *general* comparisons
    - `=, !=, <, <=, >, >=`
    - can be used with sequences of multiple items

Comparisons

General Comparisons

Example	Value
`doc("catalog.xml")/catalog/product[2]/name = 'Floppy Sun Hat'`	true
`doc("catalog.xml")/catalog/product[4]/number < 500`	false
`1 > 2`	false
`( ) = (1, 2)`	false
`(2, 5) > (1, 3)`	true

Value vs. General Comparisons

```
doc("ord.xml")//item/@quantity > 1
```

- returns true if any `quantity` attributes have values greater than 1

```
doc("ord.xml")//item/@quantity gt 1
```

- returns true if there is only *one* `quantity` attribute returned by the expression, and its value is greater than 1
- if more than one `quantity` is returned, an error is raised

Conditional expressions

- if-then-else syntax

```
for $prod in (doc("cat.xml")/catalog/product)
return if ($prod/@dept = 'ACC')
       then <acc>{data($prod/number)}</acc>
       else <other>{data($prod/number)}</other>
```

- parentheses around `if` expression are required
- else is always required
  - but it can be just `else ()`

Effective boolean value

- "`if`" expression must be boolean
  - if it is not, its *effective boolean value* is found
- effective boolean value is false for:
  - the `xs:boolean` value `false`
  - the number `0` or `NaN`
  - a zero-length string
  - the empty sequence
- it's an error for a sequence of many atomic values

Nesting conditional expressions

- Conditional expressions can be nested
  - provides "else if" functionality

```
if ($prod/@dept = 'ACC')
then <accessory>{data($prod/number)}</accessory>
else if ($prod/@dept = 'WMN')
     then <womens>{data($prod/number)}</womens>
     else if ($prod/@dept = 'MEN')
          then <mens>{data($prod/number)}</mens>
          else <other>{data($prod/number)}</other>
```

Functions

- Built-in functions
  - over 100 functions built into XQuery
  - names do not need to be prefixed when called
- User-defined functions
  - defined in the query or in a separate module
  - names must be prefixed

Built in Functions: A Sample

- String-related
  - `substring, contains, matches, concat, normalize-space, tokenize`
- Date-related
  - `current-date, month-from-date, adjust-time-to-timezone`
- Number-related
  - `round, avg, sum, ceiling`
- Sequence-related
  - `index-of, insert-before, reverse, subsequence, distinct-values`

More built in functions

- Node-related
  - `data, empty, exists, id, idref`
- Name-related
  - `local-name, in-scope-prefixes, QName, resolve-QName`
- Error handling and trapping
  - `error, trace, exactly-one`
- Document- and URI-related
  - `collection, doc, root, base-uri`

MS SQL Server: Methods on XML Data Type

- **query() -** fragments of an XML document can be extracted using the query() method of XML data type. The query() method accepts an XQuery expression as an argument and returns an untyped XML instance.
- Eg.

```
SELECT xDoc.query('/doc[@id = 123]//section')
FROM   AdminDocs
```

MS SQL Server: Methods on XML Data Type

- **value()** - Scalar values can be extracted from an XML instance using the value() method by specifying an XQuery expression and the desired SQL type to be returned.

Eg.

```
SELECT xDoc.value(
  'data((/doc//section[@num = 3]/title)[1])', 'nvarchar(max)')
FROM   AdminDocs
```

MS SQL Server : Methods on XML Data Type

- **exist()** - method is useful for existential checks on an XML instance. It returns 1 if the XQuery expression evaluates to non-null node list; otherwise it returns 0.

```
SELECT xDoc.query('/doc[@id = 123]//section')
FROM   AdminDocs
WHERE  xDoc.exist ('/doc[@id = 123]') = 1
```

MS SQL Server: Methods on XML Data Type

- **modify()** - Data manipulation operations can be performed on an XML instance using the modify() method. Support for XML DML is provided through insert, delete, and update keywords added to XQuery. One or more nodes can be inserted, deleted, and updated using the insert, delete, and update keywords, respectively.
- Eg.

```
UPDATE AdminDocs SET xDoc.modify('
  insert
   <section num="2">
       <title>Background</title>
   </section>
  after (/doc//section[@num=1])[1]')
```

Summary

-

Database Systems

File Organization and Indexes

This Lecture…

- File Organization
- Indexes

Files of Records

- Page or block is OK when doing I/O, but higher levels of DBMS operate on *records*, and *files of records*.
- FILE: A collection of pages, each containing a collection of records. Must support:
  - insert/delete/modify record
  - read a particular record (specified using *record id*)
  - scan all records (possibly with some conditions on the records to be retrieved)

File Organization

- Three types
  - Heap File Organization
  - Sequential File Organization
  - Hashing File Organization

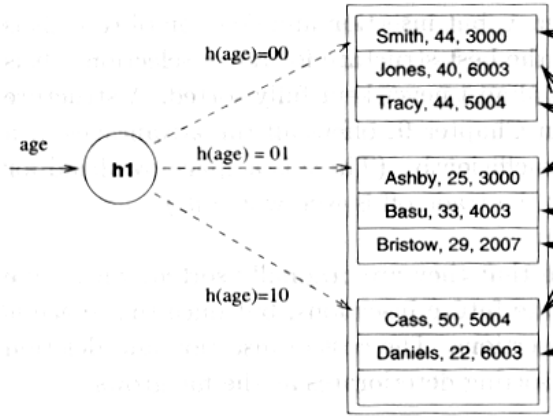
Alternative File Organizations

Many alternatives exist, *each ideal for some situation , and not so good in others:*

- Heap files: Suitable when typical access is a file scan retrieving all records.
  - Search (Equality/Range) needs to scan the file
  - Insert: At the end of file
  - Delete: Search for record and delete record
- Sorted Files: Best if records must be retrieved in some order, or only a `range' of records is needed.
  - Search (Equality/Range): Efficient
  - Insert: Finding the position, inserting & move records
  - Delete Search for record, delete & move records

Alternative File Organizations… (contd.)

- Hashed Files: Good for equality selections.
  - File is a collection of *buckets.* Bucket = *primary* page plus zero or more *overflow* pages.
  - *Hashing function* **h**: **h**(*r*) = bucket in which record *r* belongs. **h** looks at only some of the fields of *r*, called the *search fields*.



File hashed on age

Alternative File Organizations… (contd.)

- Hashed Files:
  - Search (Equality): good for equality (if based on search key). Otherwise scan table
  - Search (Range): needs to scan the file
  - Insert: search for primary bucket (hash) and insert
  - Delete: search for primary bucket (hash) if available, else scan file & delete record

Indexes

- An *index* on a file speeds up selections on the *search key fields* for the index.
  - Any subset of the fields of a relation can be the search key for an index on the relation.
  - *Search key* is not the same as *key* (minimal set of fields that uniquely identify a record in a relation).

Characteristics

- Indexes provide fast access
- Indexes takes space
  - Need to be careful in creating only useful indexes
- May slow-down certain inserts/updates/ deletes (maintain indexes)

Explain on board

Alternatives for Data Entry **k*** in Index

- An index contains a collection of *data entries*, and supports efficient retrieval of all data entries **k*** with a given key value **k**.

- Three alternatives:
  1. Data record with key value **k** (Alt. 1)
  2. <**k**, rid of data record with search key value **k**> (Alt. 2)
  3. <**k**, list of rids of data records with search key **k**> (Alt. 3)

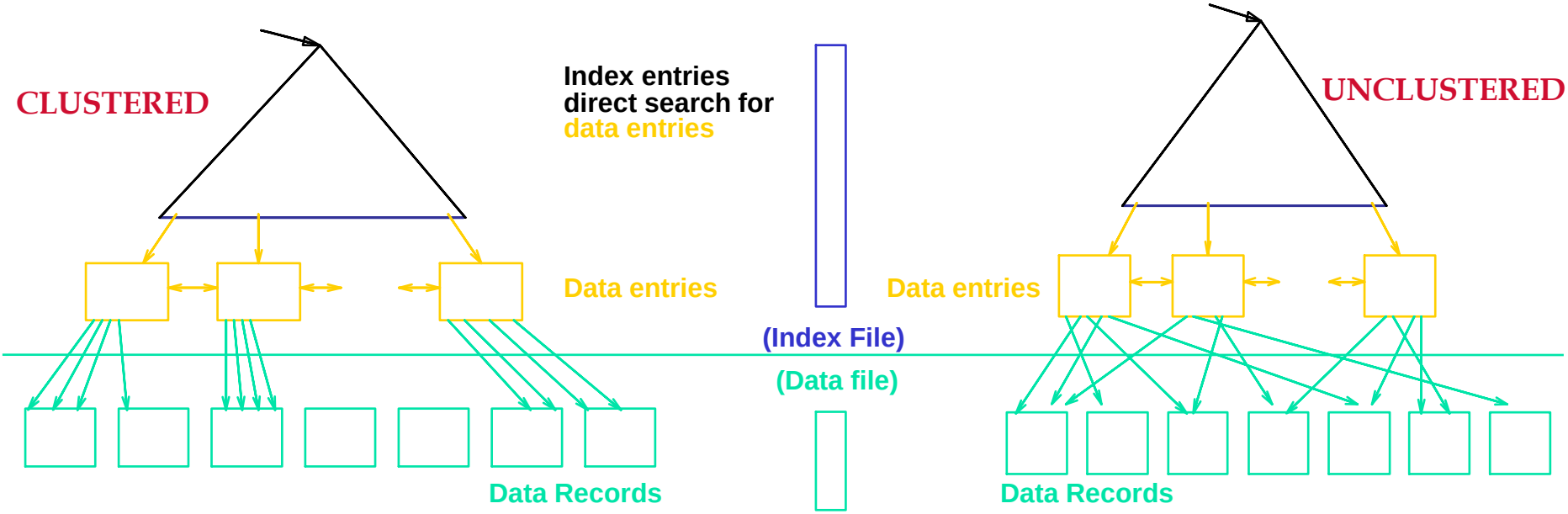
Terminology

- File of records containing index entries = **index file**

- There are several organization techniques for building index files = **access methods**

Properties of Indexes…

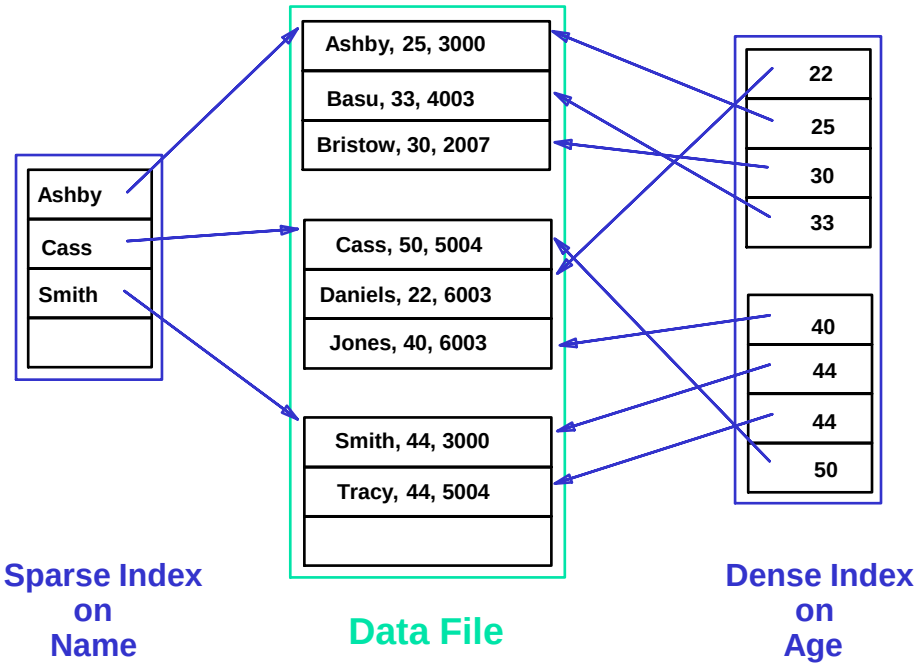
- Clustered vs. Unclustered Index



- Can have at most one clustered index per table
- Cost of retrieving data records through index varies *greatly* based on whether index is clustered or not!

Properties… (contd.)

- *Dense* vs. *Sparse*: If there is at least one data entry per search key value (in some data record), then dense.
  - Alt. 1 always leads to dense index.
  - Every sparse index is clustered!
  - Sparse indexes are smaller; however, some useful optimizations are based on dense indexes.



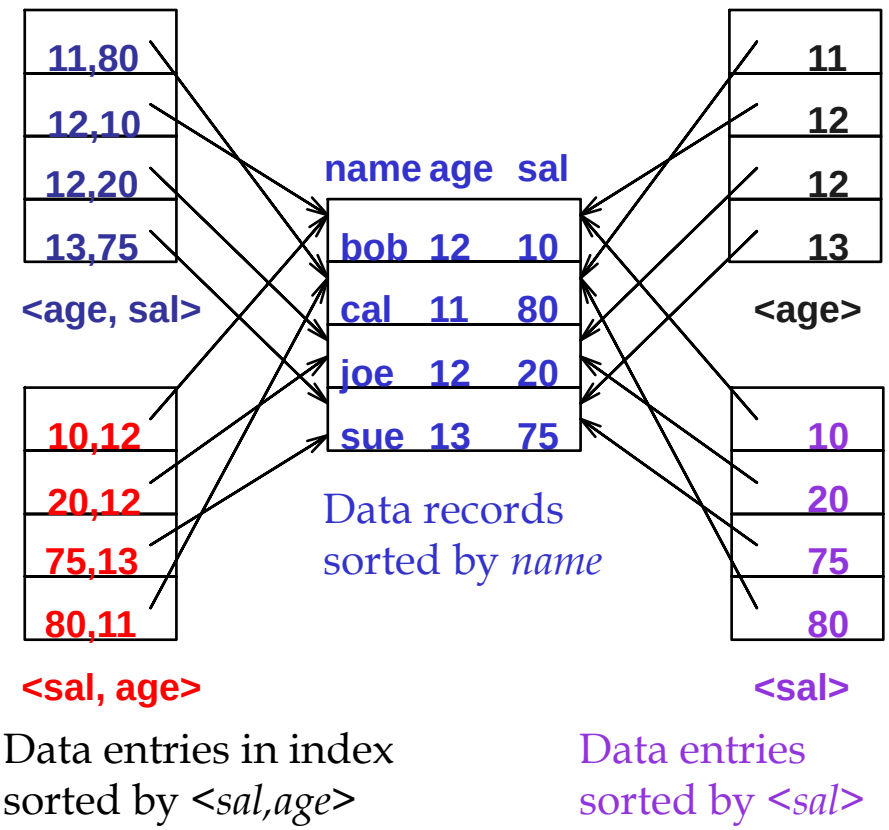
Properties… (contd.)

- *Primary* vs. *secondary*: If search key contains primary key, then called primary index.
  - *Unique* index: Search key contains a candidate key.

Properties… (contd.)

- *Composite Search Keys*: Search on a combination of fields.
  - Equality query: Every field value is equal to a constant value. E.g. wrt <sal,age> index:
    - age=20 and sal =75
  - Range query: Some field value is not a constant. E.g.:
    - age =20; or age=20 and sal > 10
- Data entries in index sorted by search key to support range queries.

Examples of composite key indexes using lexicographic order.



Data entries in index sorted by <*sal,age*>

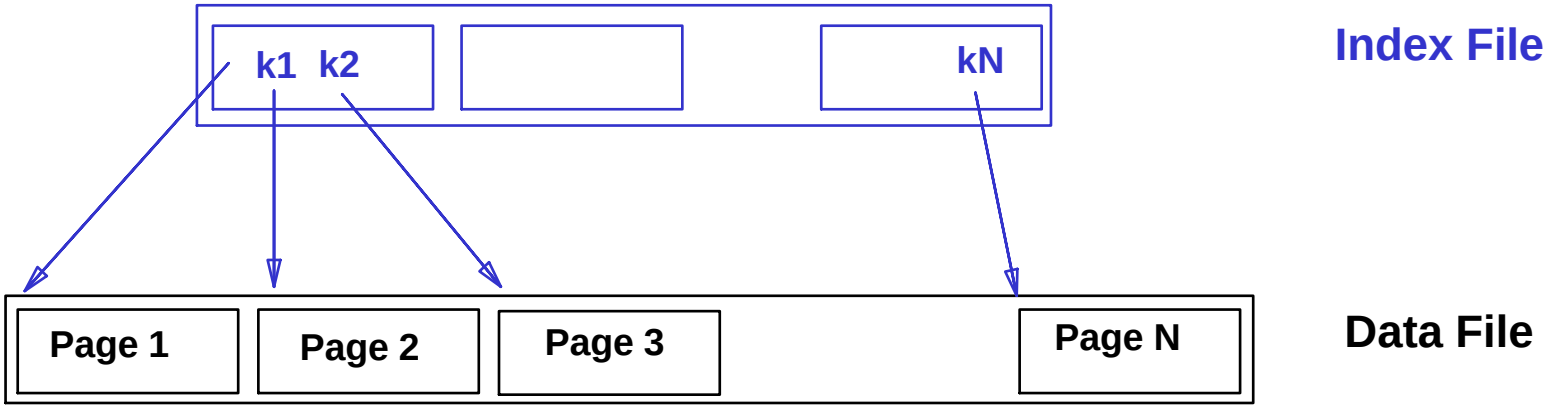
Data entries sorted by <*sal*>

Indexes in SQL…

- Index is not a part of SQL-92
- However, all major DBMSs provide facilities for index creation
  - CREATE INDEX…
  - DROP INDEX…
- SQL Server support indexes (clustered and non-clustered indexes)

Range Searches

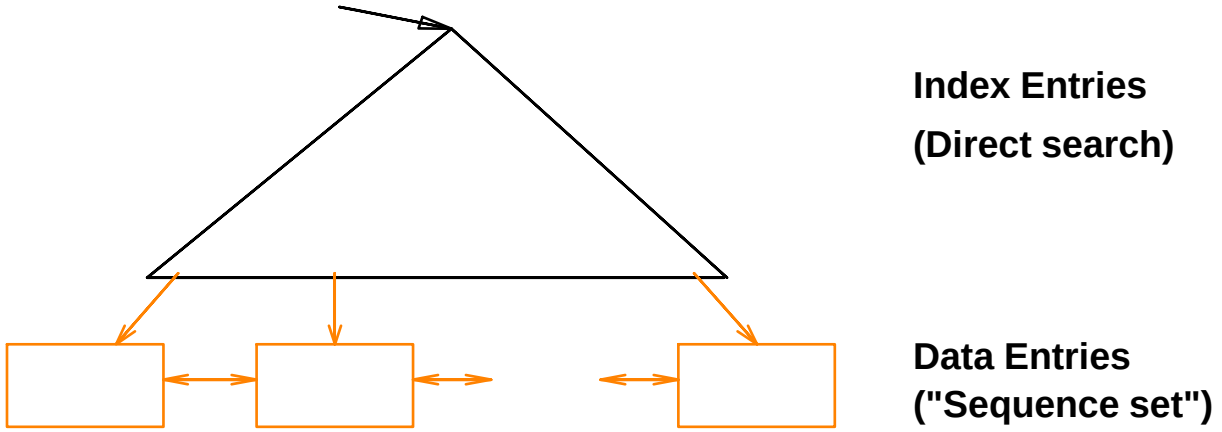
- ``*Find all students with gpa > 3.0*''
  - If data is in sorted file, do binary search to find first such student, then scan to find others.
  - Cost of binary search can be quite high.
- Simple idea: Create an `index' file.



- *Can do binary search on (smaller) index file!*

B+ Tree: The Most Widely Used Index

- Insert/delete at $\log_F N$ cost; keep tree *height-balanced*. (F = fanout, N = # leaf pages)
- Minimum 50% occupancy (except for root). Each node (*except root*) contains **d** <= *m* <= 2**d** entries. The parameter **d** is called the *order* of the tree.
- Supports equality and range-searches efficiently.

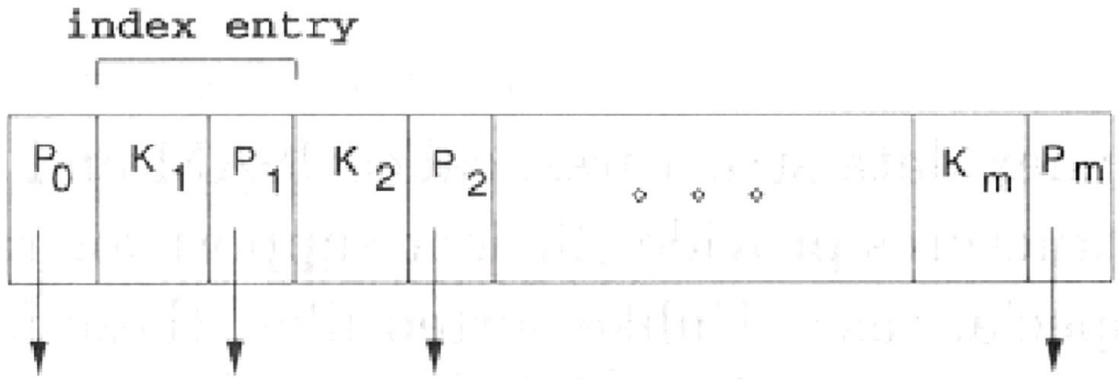


B+ Trees in Practice

- Typical order: 100. Typical fill-factor: 67%.
  - average fanout = 133
- Typical capacities:
  - Height 4: 133^4 = 312,900,700 records
  - Height 3: 133^3 = 2,352,637 records
- Can often hold top levels in buffer pool:
  - Level 1 = 1 page = 8 Kbytes
  - Level 2 = 133 pages = 1 Mbyte
  - Level 3 = 17,689 pages = 133 MBytes

B+ Tree…

- Search begins at root, and key comparisons direct it to a leaf
- Each Node has search keys (K_i) and pointers (P_i).
- P_i points to a sub-tree in which all key values K are such that $K_i \leq K < K_{i+1}$



Search

func *tree_search* (nodepointer, search key value *K)* **returns** nodepointer

| | Searches tree for entry

if *nodepointer is a leaf, return nodepointer;

else,

if *K* < *K*1 then return tree_search(Po, *K);*

else,

if *K* ≥ *Km* then return tree_search(Pm, *K)* // m = # entries

else,

find *i* such that *Ki* ≤ *K* < *Ki+1;*

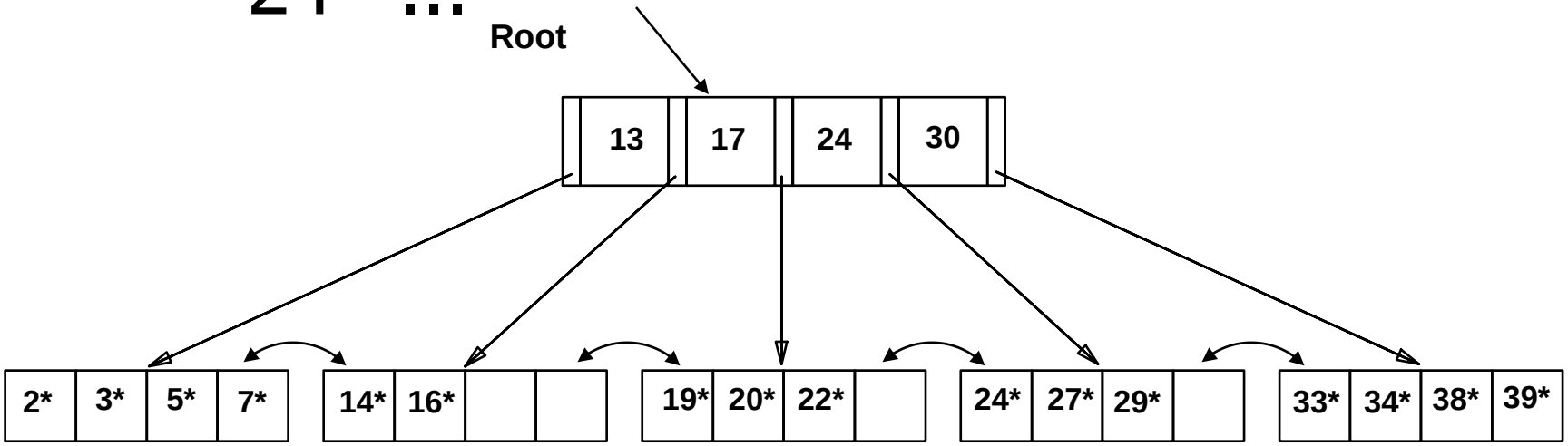
return tree_search(Pi, *K)*

end if

end if

Example B+ Tree…

- Search for 5*, 15*, all data entries >= 24* …



 Based on the search for 15, we <u>know</u> it is not in the tree!*

Inserting a Data Entry into a B+ Tree

Find correct leaf *L.*

Put data entry onto *L*.

- If *L* has enough space, *done*!
- Else, must *split* *L (into L and a new node L2)*
  - Redistribute entries evenly, **copy up** middle key.
  - Insert index entry pointing to *L2* into parent of *L*.

This can happen recursively

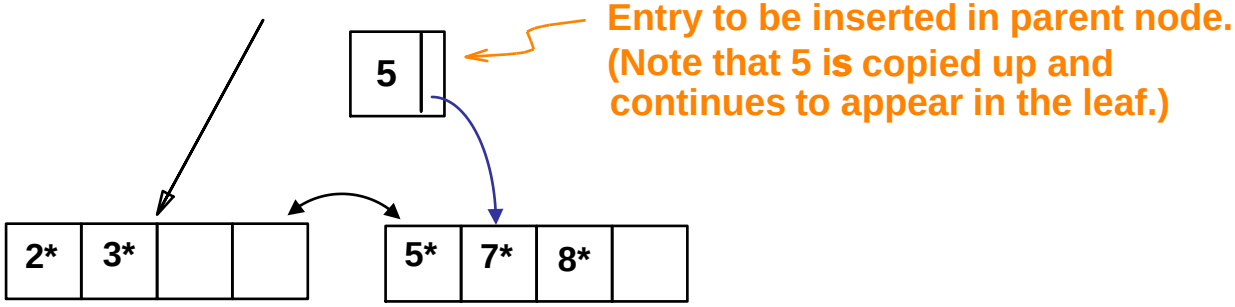
- To split index node, redistribute entries evenly, but **push up** middle key. (Contrast with leaf splits.)

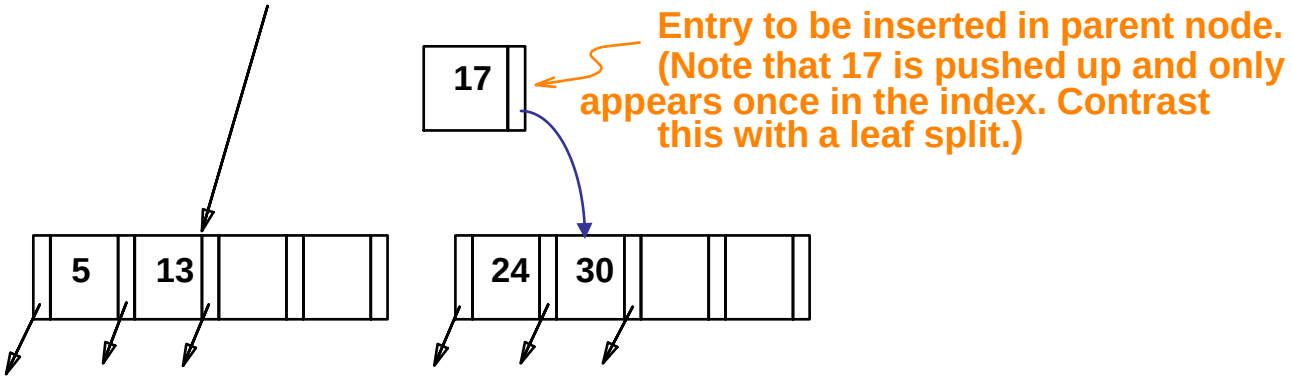
Splits "grow" tree; root split increases height.

- Tree growth: gets *wider* or *one level taller at top.*

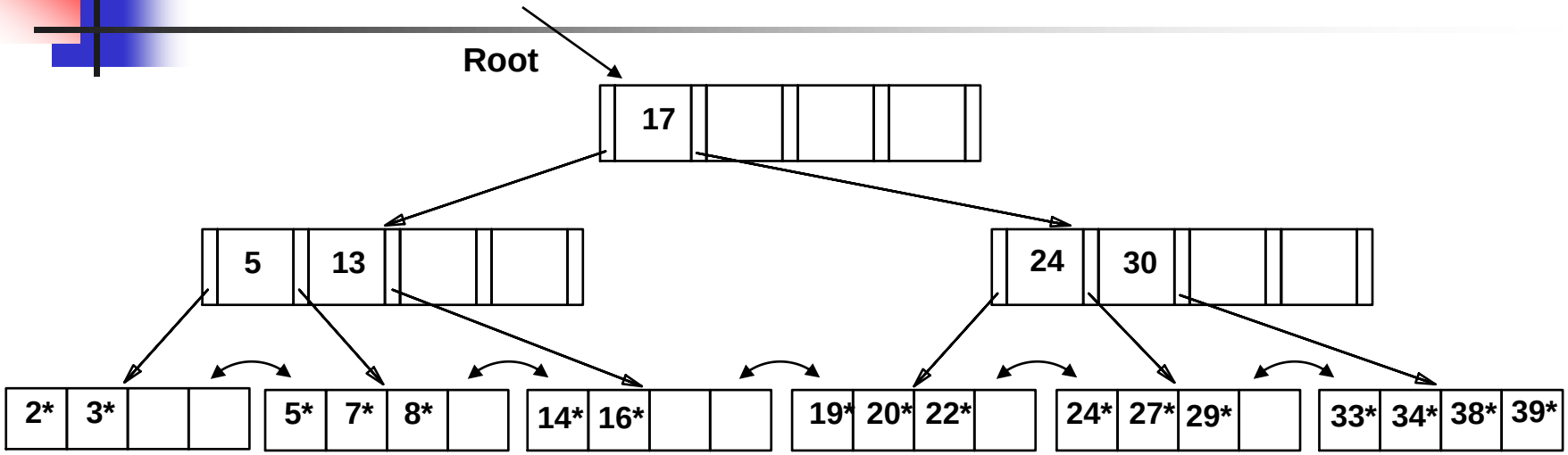
Inserting 8* into Example B+ Tree

- Observe how minimum occupancy is guaranteed in both leaf and index pg splits.
- Note difference between *copy-up* and *push-up*; be sure you understand the reasons for this.





Example B+ Tree After Inserting 8*

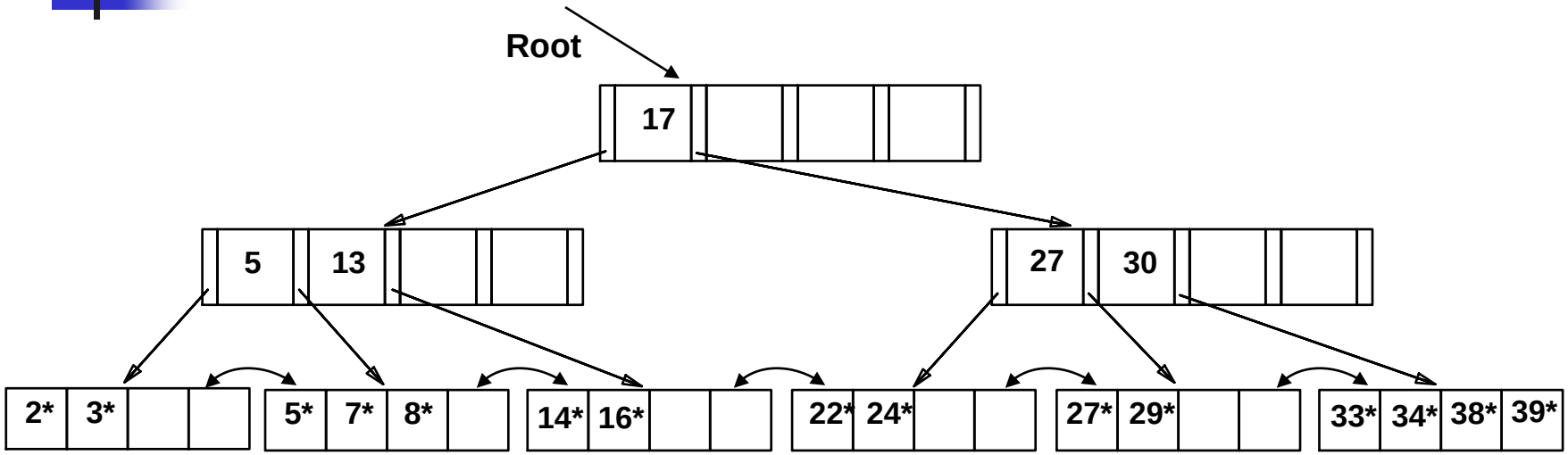


- Notice that root was split, leading to increase in height.
- In this example, we can avoid split by re-distributing entries; however, this is usually not done in practice.

Deleting a Data Entry from a B+ Tree

- Start at root, find leaf *L* where entry belongs.
- Remove the entry.
  - If L is at least half-full, *done!*
  - If L has only **d-1** entries,
    - Try to re-distribute, borrowing from *sibling (adjacent node with same parent as L)*.
    - If re-distribution fails, *merge* *L* and sibling.
- If merge occurred, must delete entry (pointing to *L* or sibling) from parent of *L*.
- Merge could propagate to root, decreasing height.

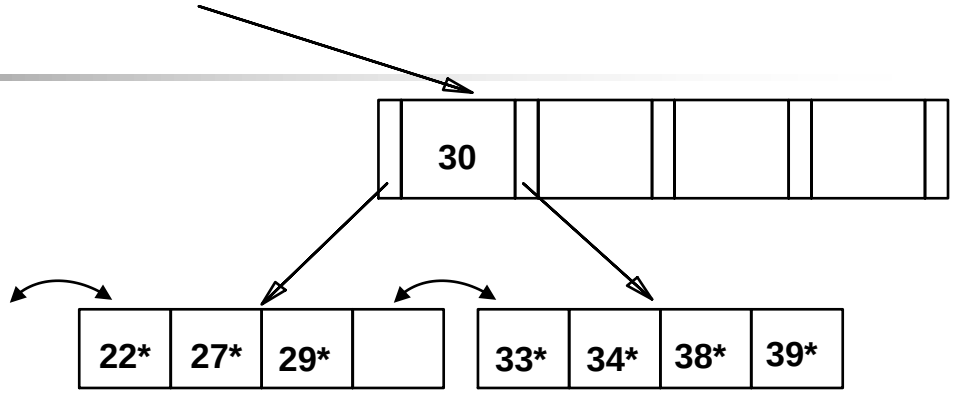
Example Tree After (Inserting 8*, Then) Deleting 19* and 20* ...

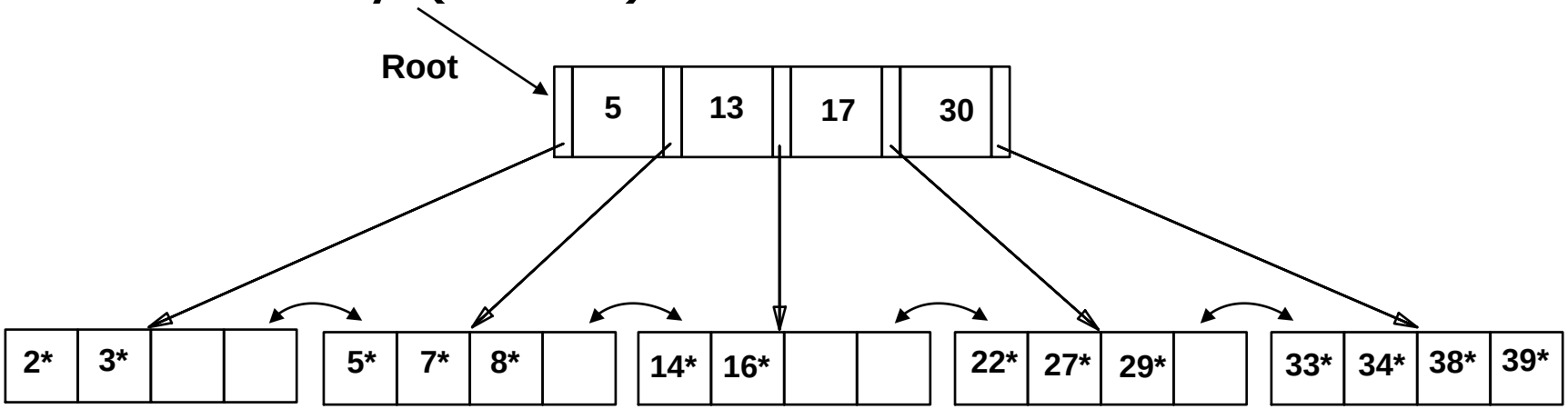


- Deleting 19* is easy.
- Deleting 20* is done with re-distribution. Notice how middle key is *copied up*.

... And Then Deleting 24*

- Must merge.
- Observe `*toss*' of index entry (on right), and `*pull down*' of index entry (below).





Duplicates in B+ Trees…

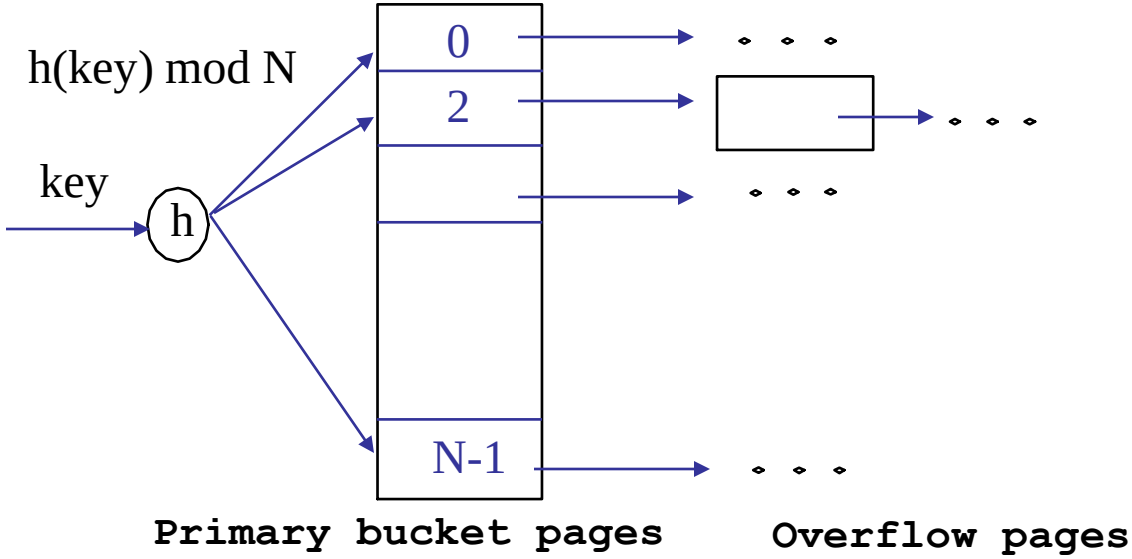
- We have ignored duplicates so far…
- Alternatives…
  - Overflow leaf pages
  - Duplicate values in the leaf pages
  - Make unique key values (by adding rowid's)
    - Preferred approach by many DBMSs

Hashing

- *Hash-based* indexes are best for *equality selections*.
- ***Cannot*** support range searches.
- Static and dynamic hashing techniques exists

Static Hashing

- # primary pages fixed, allocated sequentially, never de-allocated; overflow pages if needed.
- **h**(*k*) mod N = bucket to which data entry with key *k* belongs. (N = # of buckets)



Static Hashing… (contd.)

- Buckets contain *data entries*.
- Hash fn works on *search key* field of record *r*. Must distribute values over range 0…N-1.
  - **h**(*key*) = (a * *key* + b) usually works well.
  - a and b are constants; lots known about how to tune **h**.

Static Hashing… (contd.)

Problems…

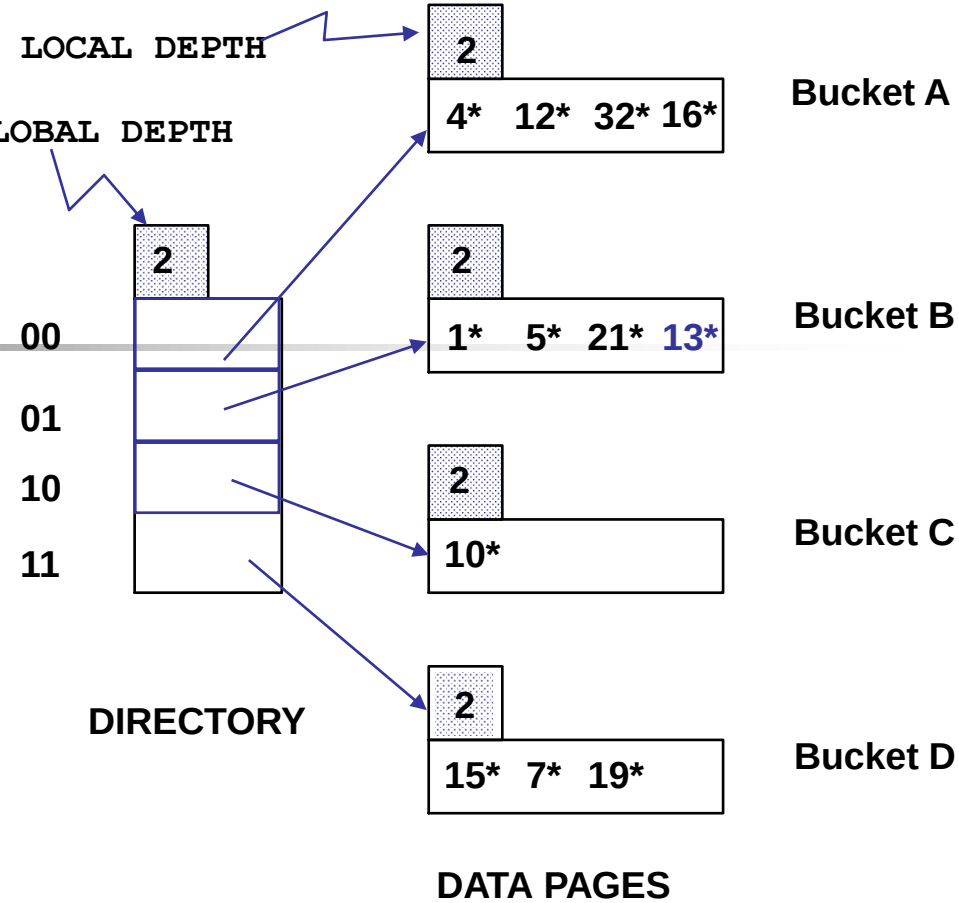
- Insertion can create long overflow chains can develop and degrade performance.
- Deletion may waste space
- *Extendible* and *Linear Hashing*: Dynamic techniques to fix this problem.

Extendible Hashing

- Situation: Bucket (primary page) becomes full. Why not re-organize file by *doubling* # of buckets?
  - Reading and writing all pages is expensive!
  - *Idea*: Use *directory of pointers to buckets*, double # of buckets by *doubling the directory,* splitting just the bucket that overflowed!
  - Directory much smaller than file, so doubling it is much cheaper. Only one page of data entries is split. *No overflow page*!
  - Trick lies in how hash function is adjusted!

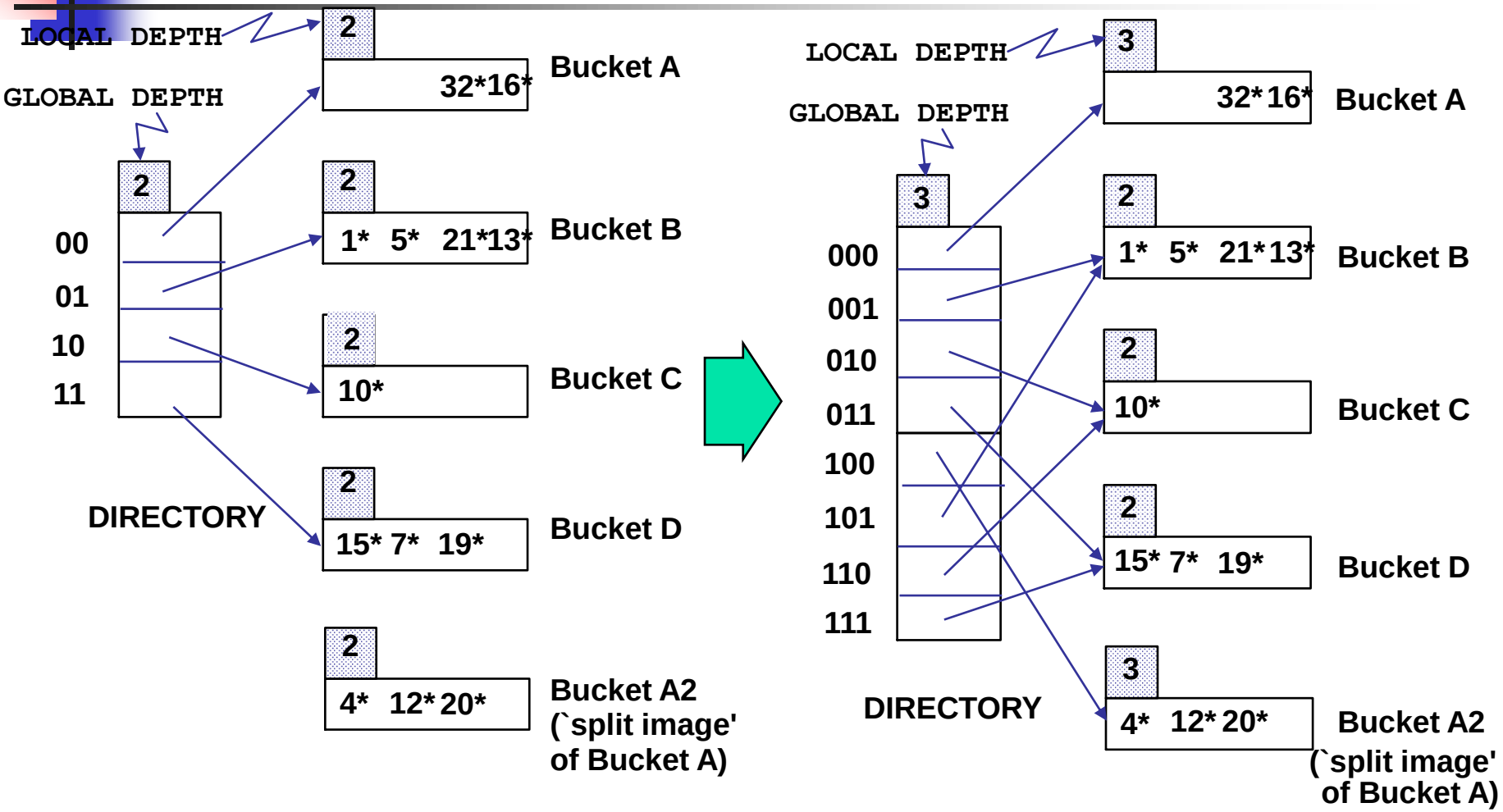
Example

- Directory is array of size 4.
- To find bucket for *r*, take last `*global depth*' # bits of **h**(*r*); we denote *r* by **h**(*r*).
  - If **h**(*r*) = 5 = binary 101, it is in bucket pointed to by 01.



- **Insert**: If bucket is full, *split* it (*allocate new page, re-distribute*).
- *If necessary*, double the directory. (As we will see, splitting a bucket does not always require doubling; we can tell by comparing *global depth* with *local depth* for the split bucket.)

Insert **h**(r)=20 (Causes Doubling)



Points to Note

- 20 = binary 10100. Last **2** bits (00) tell us *r* belongs in A or A2. Last **<u>3</u>** bits needed to tell which.
  - *Global depth of directory*: Max # of bits needed to tell which bucket an entry belongs to.
  - *Local depth of a bucket*: # of bits used to determine if an entry belongs to this bucket.

- Not all splits double the directory size
  - Example: Insert 9*

Points to Note (contd.)

- When does bucket split cause directory doubling?
  - Before insert, *local depth* of bucket = *global depth*. Insert causes *local depth* to become > *global depth*; directory is doubled by *copying it over* and `fixing' pointer to split image page. (Use of least significant bits enables efficient doubling via copying of directory!)

Comments on Extendible Hashing

- If directory fits in memory, equality search answered with one disk access; else two.
  - 100MB file, 100 bytes/rec, 4K pages contains 1,000,000 records (as data entries) and 25,000 directory elements; chances are high that directory will fit in memory.
  - Directory grows in spurts, and, if the distribution *of hash values* is skewed, directory can grow large.
  - Multiple entries with same hash value cause problems!

Comments on Extendible Hashing (contd.)

- **Delete**: If removal of data entry makes bucket empty, can be merged with `split image'. If each directory element points to same bucket as its split image, can halve directory.

Summary

- File Organizations
- Indexes
  - B+ Tree
  - Hashing (Extendible)

Database Systems

Query Processing

Last Lecture…

- Indexes
- Any questions?

This Lecture…

- Query Processing: Overview
- What?
- Why?
- How? (Basics)

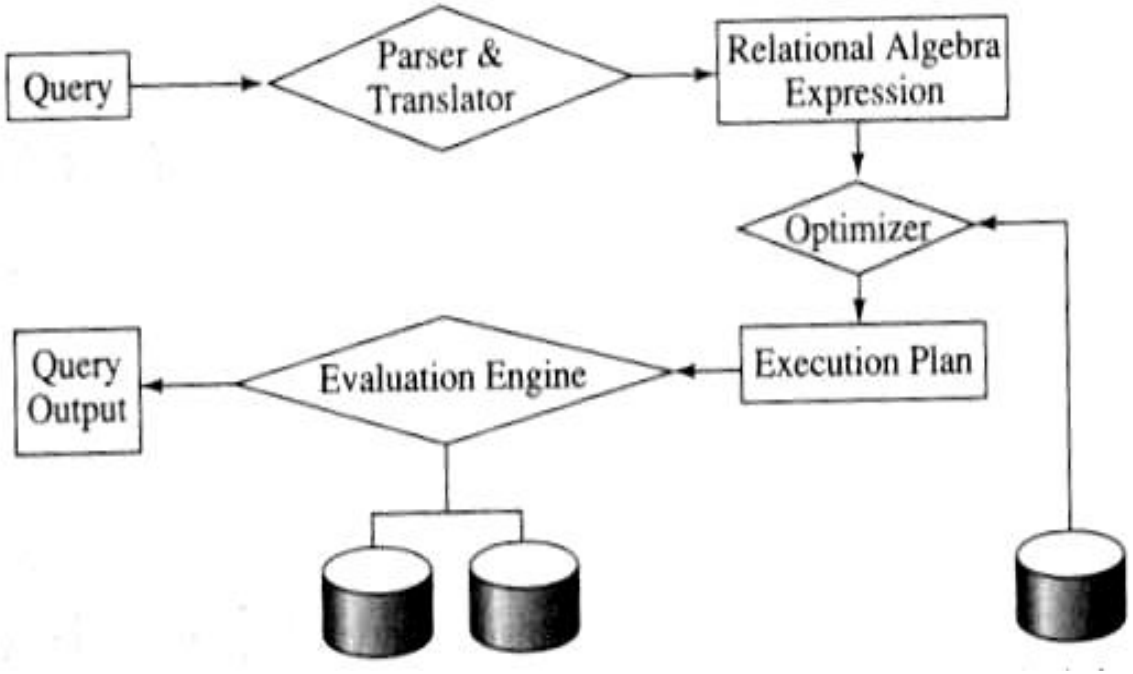
Query Processing

- What happens to a query inside the DBMS?

- Query processing considers this issue.

Steps in QP…

- The steps involved in query processing include…
  - Parsing and translation
  - Optimization
  - Evaluation



Steps in QP… (contd.)

- Parsing & translation step verifies the correctness of the query and converts the query into an internal form (usually an extended relational algebra expression)

- This internal form is either a tree (i.e. query tree) or a graph (i.e. query graph)

Steps in QP… (contd.)

- Next, an efficient execution strategy for retrieving the results of the query from the database files is generated. This step is called query optimization.
- This is the heart of query processing in a relational database system
- The evaluation engine executes the query according to the chosen plan

Parsing & Translating…

- The first step in query processing is to convert the query into an form that can be executed
- Consider the following schema

  S(sno, sname, status, city)

  P(pno, pname, colour, weight)

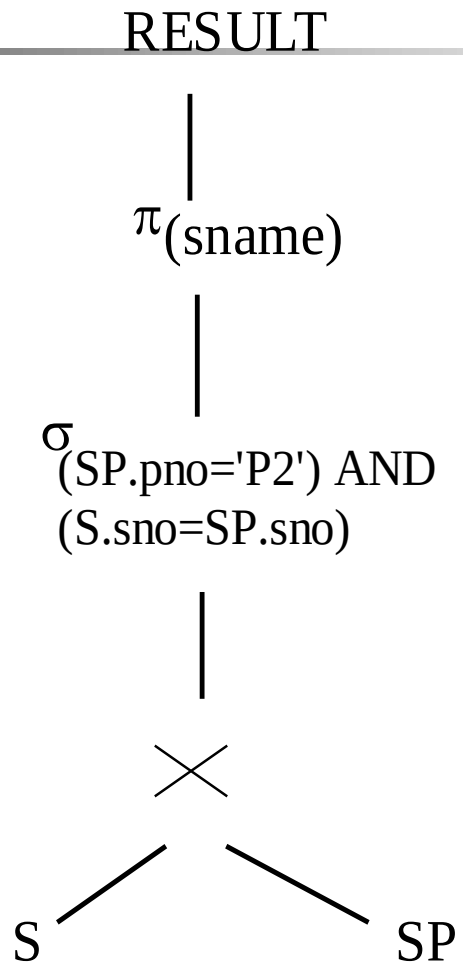
  SP(sno, pno, qty)

Parsing & Translating… (contd.)

```
SELECT   s.sname
FROM     S, SP
WHERE    S.sno = SP.sno AND
         SP.pno = 'P2'
```

We can express this query as a relational algebra as follows…

Parsing & Translating… (contd.)



Parsing & Translating… (contd.)

- Usually, a SELECT-FROM-WHERE-GROUP BY called a ***query block*** is converted an extended relational algebra expression

- There can be many query blocks (i.e. with nested queries) in a complex query

Why do we need Query Optimization?

- Consider the following number of tuples for S and SP

  S – 100 pages

  SP - 10000 pages
- Cost for computing Cartesian Product:

  - read 10,000 * 100 pages

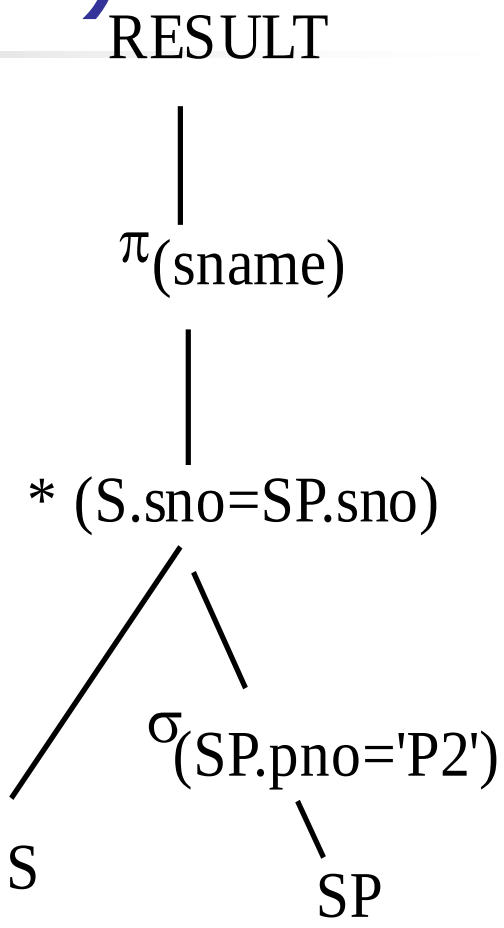
  - write 1,000,000 pages (intermediate result)
- Selection

  - read 1,000,000 pages

  - keep 50 tuples (assuming 50 tuple SP.pno = 'P2')
- Total number of disk I/O ~ 3,000,000 disk I/Os

Optimization? (contd.)

- Consider the following relational algebra, which produces the same result



- Selection operator
  - Read 10,000 pages
  - Keep the result, 50 tuples in memory
- Join with S
  - Read 100 pages

Total cost 10,100 disk I/Os

Heuristic Optimization

- A query can have many equivalent query trees
- Optimizer must find an efficient query plan to execute
- Heuristic rules are used for algebraic optimization

Equivalence Rules…

- To transform a relational algebra expression from to an equivalent efficient query expression, certain equivalence rules are used.

Equivalence Rules… (contd.)

- Conjunctive selection operations can be deconstructed into a sequence of individual selections. This transformation is referred to as a cascade of σ.

$$\sigma_{C1 \wedge C2}(E) = \sigma_{C1} (\sigma_{C2} (E))$$

- Selection operations are commutative.

$$\sigma_{C1} (\sigma_{C2} (E)) = \sigma_{C2} (\sigma_{C1} (E))$$

- Cascade of Π.

$$\Pi_{L1} (\Pi_{L2} (\ldots (\Pi_{Ln} (E)\ldots)) = \Pi_{L1} (E)$$

Equivalence Rules… (contd.)

- Selections can be combined with Cartesian products and theta joins.

  a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E2$

- This expression is just the definition of the theta join.

  b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta 2} E2) = E1 \bowtie_{\theta 1 \wedge \theta 2} E2$

Etc.

Equivalence Rules (contd.)

- Equivalence rules states that two expressions are equal
- It does not state, which one is better

- A large number of plans are possible! Estimating the cost for each is prohibitively expensive

- The optimizer uses heuristic rules to prune the plan space (reduce the number of plans to be considered)
  - E.g. Bring selects down, avoid cartesian products, etc.

Indexes and cost of query plans…

- Using an index does not necessarily mean efficient query plan.
- Can you think of an instance where using an index is inefficient?
- Query optimizer estimates costs to compare different execution plans

Cost estimation

- Execution plan has
  - A set of relational algebra operators (query plan) to obtain the result of the query
  - Algorithm used to evaluate each relational algebra operators
- Some relational algebra operators have many possible ways (algorithms).
- Costs may differ significantly based on the chosen algorithm
- We will study algorithms some algorithms used for simple selections and joins

Schema for Examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)
Reserves (*sid*: integer, *bid*: integer, *day*: dates, *rname*: string)

Reserves:

- Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.

- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages.

Simple Selections

```
SELECT  *
FROM    Reserves R
WHERE   R.rname < 'C%'
```

- Of the form $\sigma_{R.attr\ op\ value}(R)$
- Size of result approximated as *size of R * reduction factor*; we will consider how to estimate reduction factors later.
- With no index, unsorted: Must essentially scan the whole relation; cost is M (#pages in R).
- With an index on selection attribute: Use index to find qualifying data entries, then retrieve corresponding data records. (Hash index useful only for equality selections.)

Using an Index for Selections

- Cost depends on #qualifying tuples, and clustering.
  - Cost of finding qualifying data entries (typically small) plus cost of retrieving records (could be large w/o clustering).
  - In example, assuming uniform distribution of names, about 10% of tuples qualify (100 pages, 10000 tuples) With a clustered index, cost is little more than 100 I/Os; if unclustered, upto 10000 I/Os!

Equality Joins With One Join Column

```
SELECT  *
FROM    Reserves R1, Sailors S1
WHERE   R1.sid=S1.sid
```

- In algebra: $R \bowtie S$. Common! Must be carefully optimized. $R \times S$ is large; so, $R \times S$ followed by a selection is inefficient.

- Assume: M tuples in R, p_R tuples per page, N tuples in S, p_S tuples per page.
  - In our examples, R is Reserves and S is Sailors.

- *Cost metric*: # of I/Os. We will ignore output costs.

Simple Nested Loops Join

```
foreach tuple r in R do
    foreach tuple s in S do
        if r_i == s_j then add <r, s> to result
```

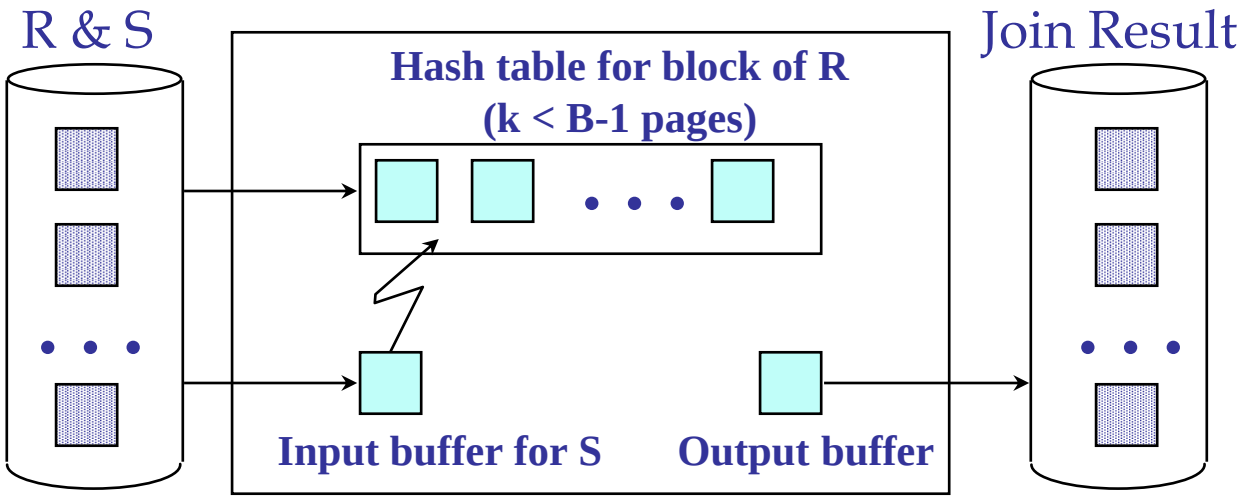
- For each tuple in the *outer* relation R, we scan the entire *inner* relation S.
  - Cost: $M + p_R * M * N = 1000 + 100*1000*500$ I/Os.

Page-oriented Nested Loop Join

- Page-oriented Nested Loops join: For each *page* of R, get each *page* of S, and write out matching pairs of tuples <r, s>, where r is in R-page and S is in S-page.
  - Cost: M + M*N = 1000 + 1000*500
  - If smaller relation (S) is outer, cost = 500 + 500*1000

Block Nested Loops Join

- Use one page as an input buffer for scanning the inner S, one page as the output buffer, and use all remaining pages to hold ``block" of outer R.
  - For each matching tuple r in R-block, s in S-page, add <r, s> to result. Then read next R-block, scan S, etc.



Examples of Block Nested Loops

- Cost: Scan of outer + #outer blocks * scan of inner
  - #outer blocks = $\lceil \# \ of \ \ pages \ of \ \ outer \ / \ blocksize \rceil$
  - Block size = available buffers - 2
- With Reserves (R) as outer, and 100 pages of R:
  - Cost of scanning R is 1000 I/Os; a total of 10 *blocks*.
  - Per block of R, we scan Sailors (S); 10*500 I/Os.
  - If space for just 90 pages of R, we would scan S 12 times.
- With 100-page block of Sailors as outer:
  - Cost of scanning S is 500 I/Os; a total of 5 blocks.
  - Per block of S, we scan Reserves; 5*1000 I/Os.

Index Nested Loops Join

foreach tuple r in R do
foreach tuple s in S where $r_i == s_j$ do
add <r, s> to result

- If there is an index on the join column of one relation (say S), can make it the inner and exploit the index.
  - Cost: $M + ((M*p_R) *$ cost of finding matching S tuples)
- For each R tuple, cost of probing S index is about 1.2 for hash index, 2-4 for B+ tree. Cost of then finding S tuples (assuming Alt. (2) or (3) for data entries) depends on clustering.
  - Clustered index: 1 I/O (typical), unclustered: upto 1 I/O per matching S tuple.

Sort-Merge Join $(R \bowtie_{i=j} S)$

- Sort R and S on the join column, then scan them to do a "merge" (on join col.), and output result tuples.
  - Advance scan of R until current R-tuple >= current S tuple, then advance scan of S until current S-tuple >= current R tuple; do this until current R tuple = current S tuple.
  - At this point, all R tuples with same value in Ri (*current R group*) and all S tuples with same value in Sj (*current S group*) *<u>match</u>*; output <r, s> for all pairs of such tuples.
  - Then resume scanning R and S.

Example of Sort-Merge Join

sid	sname	rating	age
22	dustin	7	45.0
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	bid	day	rname
28	103	12/4/96	guppy
28	103	11/3/96	yuppy
31	101	10/10/96	dustin
31	102	10/12/96	lubber
31	101	10/11/96	lubber
58	103	11/12/96	dustin

- R is scanned once; each S group is scanned once per matching R tuple. (Multiple scans of an S group are likely to find needed pages in buffer.)
- Cost: O(M log M) + O(N log N) + (M+N)
  - The cost of scanning, M+N, could be M*N (very unlikely!)

Cost-based Optimization

- The fact that there are many algorithms for relational operators, choosing a good query plan using heuristics is not enough
- We can calculate the cost of each candidate query tree with the possible algorithms for each operator (& the difference can be significant)
- To compare such query plans, cost-based optimization techniques are used

Cost Estimation

- For each plan considered, must estimate cost:
  - Must estimate *cost* of each operation in plan tree.
    - Depends on input cardinalities.
    - We've already discussed how to estimate the cost of operations (sequential scan, index scan, joins, etc.)
  - Must estimate *size of result* for each operation in tree!
    - Use information about the input relations.
    - For selections and joins, assume independence of predicates.

Statistics and Catalogs

- Need information about the relations and indexes involved. ***Catalogs*** typically contain at least:
  - # tuples (NTuples) and # pages (NPages) for each relation.
  - # distinct key values (NKeys) and NPages for each index.
  - Index height, low/high key values (Low/High) for each tree index.

Statistics and Catalogs

- Catalogs updated periodically.
  - Updating whenever data changes is too expensive; lots of approximation anyway, so slight inconsistency ok.
- More detailed information (e.g., histograms of the values in some field) are sometimes stored.

Size Estimation and Reduction Factors

```
SELECT  attribute list
FROM  relation list
WHERE  term1 AND ... AND termk
```

- Consider a query block:
- Maximum # tuples in result is the product of the cardinalities of relations in the FROM clause.
- *Reduction factor (RF)* associated with each *term* reflects the impact of the *term* in reducing result size. *Result cardinality* = Max # tuples * product of all RF's.
  - Implicit assumption that *terms* are independent!

System R:

- Term *col=value* has RF *1/NKeys(I),* given index I on *col*
- Term *col1=col2* has RF *1/MAX(NKeys(I1), NKeys(I2))*
- Term *col>value* has RF *(High(I)-value)/(High(I)-Low(I))*

Summary

- Query Processing consists of several steps
- Query optimization is an important task in a relational DBMS.
- Must understand optimization in order to understand the performance impact of a given database design (relations, indexes) on a workload (set of queries).
- Two parts to optimizing a query:
  - Consider a set of alternative plans (*heuristics are used to reduce the number of possible plans to consider*).
  - Must estimate cost of each plan that is considered.
    - Must estimate size of result and cost for each plan node.
    - *Key issues*: Statistics, indexes, operator implementations.

Database Systems

Query Optimisation

Last lecture

- Steps in query processing
  - Cast to relational algebra expression
  - Convert to a efficient relational algebra expression
    - Equivalence rules and heuristics
  - Choose low level procedures
    - Selections
    - Projection

  - Any questions?

Nested Loop Join

- Assume joining relations R and S,

  $R \bowtie_{R.a=S.b} S$

  For each tuple r in R do

  for each tuple s in S do

  if $r_i == s_j$ then add <r, s> to result

- Three algorithms
  - Simple Nested Loop Join
  - Page Oriented Nested Loop Join
  - Block Oriented Nested Loop Join

Simple Nested Loops join

- For each tuple of R, get all tuple of S, and write out matching pairs of tuples <r, s>, where r is in R-page and S is in S-page.
- Assume M pages in R, N pages in S and P_R tuples per page in R.

$$\boxed{Cost = M + M * P_R * N}$$

- Minimum buffer needed is 3 pages.
- Selecting the smaller relation as outer may gives a low cost.
- $SP \bowtie_{SP.sno=S.sno} S$, SP as outer

$$Cost = 1000 + 1000 * 100 * 100 = 1000 + 10{,}000{,}000 = 10{,}001{,}000 \text{ IOs}$$

- $S \bowtie_{SP.sno=S.sno} SP$, S as outer

$$Cost = 100 + 100 * 10 * 1000 = 100 + 1{,}000{,}000 = 1{,}000{,}100 \text{ IOs}$$

Page Oriented Nested Loops join

- For each page read from R, get all pages of S, and write out matching pairs of tuples <r, s>, where r is in R-page and S is in S-page.
- Assume M pages in R and N pages in S.

$$\boxed{Cost = M + M * N}$$

- Minimum buffer needed is 3 pages.
- Selecting the smaller relation as outer gives the lowest cost.
- SP $\bowtie_{SP.sno=S.sno}$ S, SP as outer

$$Cost = 1000 + 1000 * 100 = 1000 + 100{,}000 = 101{,}000 \text{ IOs}$$

- S $\bowtie_{SP.sno=S.sno}$ SP, S as outer

$$Cost = 100 + 100 * 1000 = 100 + 100{,}000 = 100{,}100 \text{ IOs}$$

Block Oriented Nested Loops join

- For each block of R, get all pages of S, and write out matching pairs of tuples <r, s>, where r is in R-page and S is in S-page.
- Assume M pages in R, N pages in S and buffer size is B.

$$\boxed{Cost = M + \left\lceil \frac{M}{B-2} \right\rceil * N}$$

- Minimum buffer needed is 3 pages. If buffer size is 3 the algorithm is same as page oriented nested loop join.
- Selecting the smaller relation as outer gives the lowest cost.
- SP $\bowtie_{SP.sno=S.sno}$ S, SP as outer. (Assume B=12)

$$Cost = 1000 + \left\lceil \frac{1000}{12-2} \right\rceil * 100 = 1000 + 10{,}000 = 11{,}000 \text{ IOs}$$

- S $\bowtie_{SP.sno=S.sno}$ SP, S as outer. (Assume B=12)

$$Cost = 100 + \left\lceil \frac{100}{12-2} \right\rceil * 1000 = 100 + 10{,}000 = 10{,}100 \text{ IOs}$$

Index Nested Loops Join

For each tuple r in R do
    for each tuple s in S where $r_i == s_j$ do
        add <r, s> to result

- If there is an index on the join column of one relation (say S), can make it the inner and use the index.
- For each tuple in R find matching tuples of S using the index.
- Assume M pages in R, N pages in S and P_R tuples per page in R.

$$\text{Cost} = M + M * P_R * (\text{Cost of getting matching S tuples})$$

Index Nested Loop Join …

- For each R tuple, cost of probing S index:
  - About 1.2 for hash index,
  - 2-4 for B+ tree.
- Cost of finding S tuples (assuming <key, rid> or <key, rid list> for index entries) depends on
  - Type of index (B+ / Hash)
  - Clustering.
  - Number of matching S tuples for each R tuple

Example of Index Nested Loops

- $SP \bowtie_{SP.sno=S.sno} S$, SP as outer.
- sno is the primary key of the inner relation. One tuple in outer exactly matches one tuple in inner.
- Since one tuple in inner matches the join condition clustering has no effect on the cost.

$$\text{Cost} = 1000 + 1000 * 100 * (\text{Cost of g.m.t.})$$

- Clustered or un-clustered Hash index on S<sno> (S as inner):

$$\text{Cost of g.m.t.} = 1.2 + 1 \ \text{IO}$$

- Clustered or un-clustered B+ index on S<sno> (S as inner):

$$\text{Cost of g.m.t.} = h + 1 \ \text{IO}$$

Example of Index Nested Loops

- $S \bowtie_{SP.sno=S.sno} SP$, S as outer.
- sno is NOT a candidate key in the inner relation. One tuple in outer matches many tuple in inner.
  - 100 SP tuples (1 page) matches one S tuple

$$\text{Cost} = 100 + 100 * 10 * (\text{Cost of g.m.t.})$$

- Clustering has an effect on the cost.
- Clustered Hash index on SP<sno> (SP as inner):

$$\text{Cost of g.m.t.} = 1.2 + RF * Data\ \text{Pages} = 1.2 + \frac{0.1}{100} * 1000 = 1.2 + 1 = 2.2 \quad \text{IO}$$

Example of Index Nested Loops

- Un-clustered Hash index on SP<sno> (SP as inner):

$$\text{Cost of g.m.t.} = 1.2 + RF * Data\,\text{Record} = 1.2 + \frac{0.1}{100} * 100000 = 1.2 + 100 = 101.2 \quad \text{IO}$$

- Clustered B+ index on SP<sno> (SP as inner):

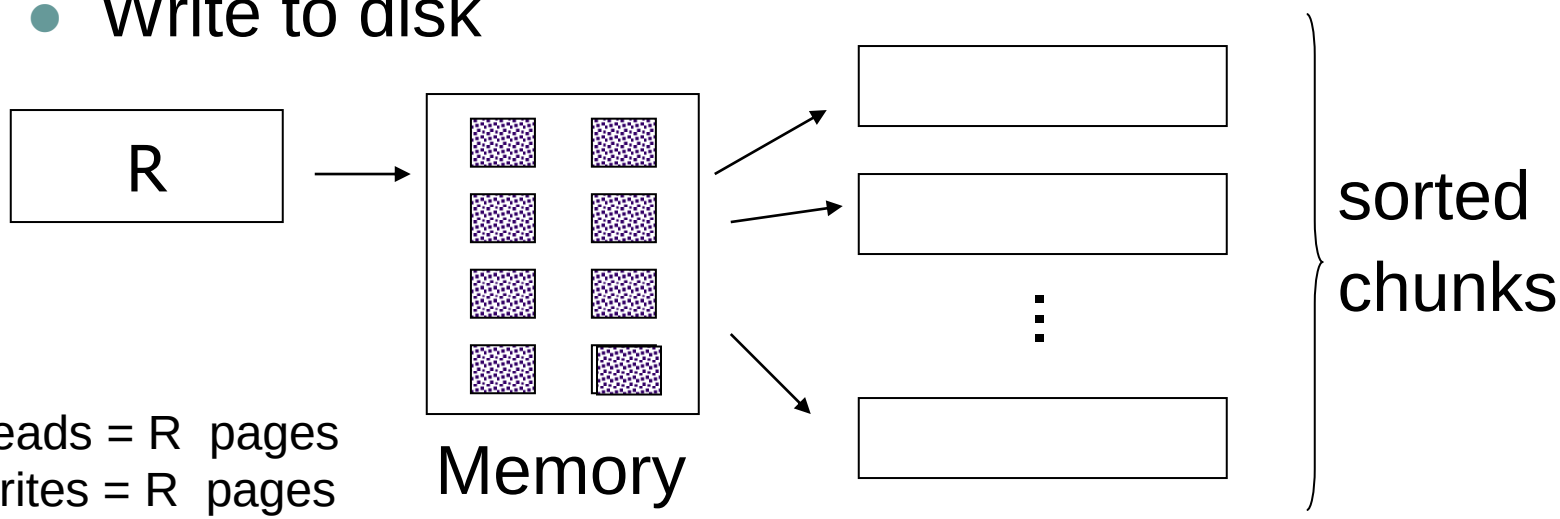
$$\text{Cost of g.m.t.} = \text{h} + RF * (Leaf\ \text{Pages} + Data\,\text{Pages}) = 3 + \frac{0.1}{100} * (334 + 1000) = 3 + 1 + 1 = 5 \quad \text{IOs}$$

- Un-clustered B+ index on SP<sno> (SP as inner):

$$\text{Cost of g.m.t.} = \text{h} + RF * (Leaf\ \text{Pages} + Data\,\text{Record}) = 3 + \frac{0.1}{100} * (334 + 100000) = 3 + 1 + 100 = 104 \quad \text{IO}$$

External Merge Sort (with two passes)

- Sorting take place in two passes
- Pass – 0
  - Read the relation as chunks of several pages
  - Sort in memory
  - Write to disk

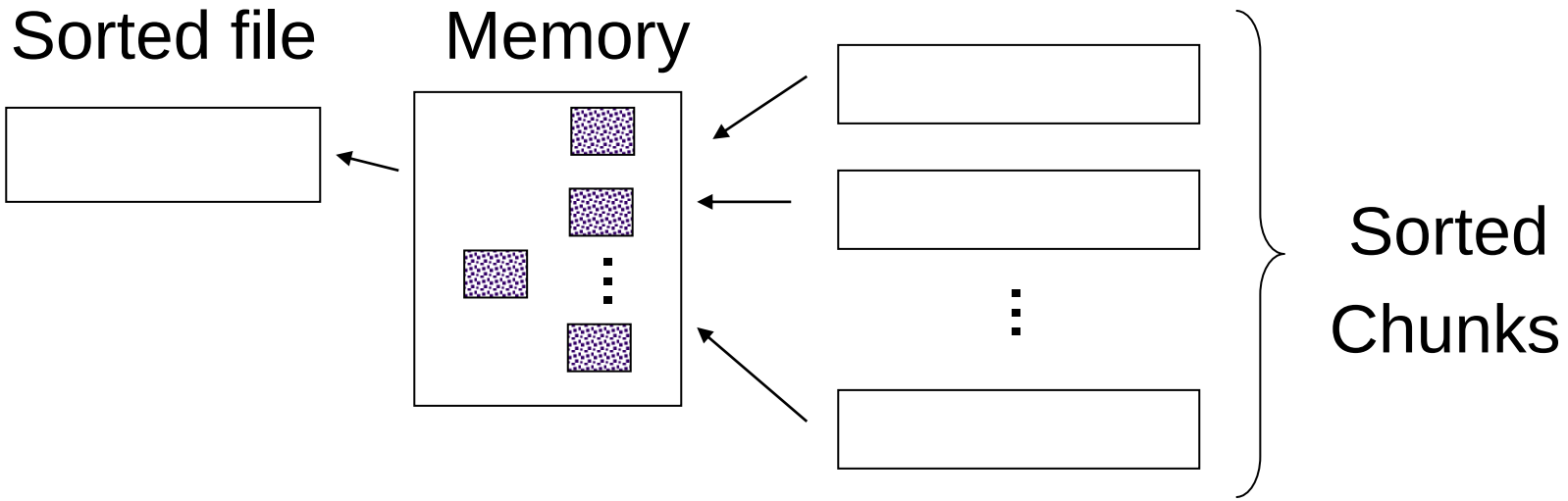


Reads = R pages
Writes = R pages
Cost = 2*R IOs

$$\text{No. of chunks} = \left\lceil \frac{R}{B} \right\rceil$$

External Merge Sort (with two passes)….

- Pass – 1
  - Read all chunks + merge + write out



Reads = R pages
Writes = R pages
Cost = 2*R IOs

$$\text{Buffer size} \geq (\text{No. of chunks}) + 1$$

$$\text{Min Buffer Size} = \lceil \sqrt{R} \rceil + 1$$

Sort-Merge Join ($R \bowtie_{R.a=S.b} S$)

- Sort R and S on the join column, then scan them to do a "merge" (on join col.), and output result tuples.
  - R is scanned once;
  - Each S group (matching S tuples of an R tuple) is scanned once per matching R tuple.
  - Multiple scans of an S group are likely to find needed pages in buffer.

$$\text{Cost} = \text{Sort R} + \text{Sort S} + \text{Merge R \& S}$$

- Best Case : For each R page it is enough to read one S page for merging.

$$\boxed{\text{Cost} = 4*M + 4*N + (M+N)}$$

Sort-Merge Join ……

- Worst Case (Very unlikely)
  - Every tuple in R matches all tuplees in S.
  - All N blocks of S read for each block of R during merge (e.g., join attr has just one value).

$$\text{Merge Cost} = M + M * N$$

- Cost of $S \bowtie_{SP.sno=S.sno} SP$.

$$\text{Cost} = 4*100 + 4*1000 + (100+1000) = 400 + 4000 + 1100 = 5500 \text{ IOs}$$

$$Min \text{ Buffer} = \left| \sqrt{1000} \right| + 1 = 32 + 1 = 33 \text{ Pages}$$

Aggregate Operations

- Without grouping:
  - In general, requires scanning the relation.
    - Scan relation to accumulate values (e.g. select sum(salary) from emp)
  - Given index whose search key includes all attributes in the SELECT or WHERE clauses, can do index-only scan.
    - Using index on <dept, salary> to process the query:

      select avg(salary) from emp where dept = ‘D1’

Aggregate Operations

- With grouping:
  - Sort on group-by attributes, then scan relation and compute aggregate for each group.
    - Select dept, avg(salary) from emp group by dept;
    - Sort emp table by dept column and then read the sorted tuples in the sequence of dept to compute avg(salary) for each dept.
  - Can improve upon this by combining sorting and aggregate computation.
    - In the last stage of sort, while writing out the sorted tuples, do the computation of avg(salary) for each dept (saves a read of the whole sorted table)

Aggregate Operations

- With grouping:
  - Similar approach based on hashing on group-by attributes.
    - Hash the tuples of emp on dept values.
    - Then compute avg(salary) for each dept by going through the buckets of the hash table.
  - Given tree index whose search key includes all attributes in SELECT, WHERE and GROUP BY clauses, can do index-only scan.
    - Select avg(salary) from emp where job='LEC' group by dept;
    - If there is an index on <dept, job, salary>, do index only scan.

Aggregate Operations

- With grouping:
  - If group-by attributes form prefix of search key, can retrieve index entries/tuples in group-by order.
  - select job, count(*) from emp group by job;
    - If the index is on <job, dept, salary>, do an index only scan.
  - select pno, avg(qty) from sp group by pno;
    - If the index is on <pno, sno>, then retrieve the sp tuples in pno order using the index.

Choice of low level procedures

- Optimiser chooses one or more candidate procedures for each low level operation
  - using information from catalog about indexes, sizes of relations, etc.
  - this process is sometimes called access path selection (this term is used by some to cover both Stage 3 and Stage 4)
    - The procedure is tied up with the access path.
    - For example, separate selection algorithms for file scan, for using primary index, etc.

Step 4: Generate query plans and choose the cheapest

- Construct a set of candidate query plans and choose the best of those plans
- Query plans are built by combining low-level operations in different ways
  - could be combinatorially expensive to do!
  - Many possible procedures for each operation in the query tree.
  - For example, 4 ways each for two selections, 5 ways to join and 3 ways to project gives 8*5*3=120 ways to execute a simple query.
- Use heuristics to reduce the search space of alternate plans
  - e.g. ignore expensive low-level procedures
- Choose the cheapest plan

Choosing the cheapest plan

- Requires assigning cost to a plan
- Cost of a plan is the sum of the costs of the individual procedures that make up the plan
  - Use the estimation methods discussed earlier for selection, join, and grouping.
- Cost formulae estimate
  - number of disk I/O's
  - CPU utilisation (we ignore this in our calculations)
- Intermediate results also cause disk I/O's
  - difficult to estimate as they are dependent on actual data values

Enumeration of Alternative Plans

- There are two main cases:
  - Single-relation plans
  - Multiple-relation plans (we consider only 2-relation queries)
- Queries over a single relation consist of a combination of selects, projects, and aggregate ops:
  - Each available access path (file scan / index) is considered, and the one with the least estimated cost is chosen.
  - The different operations are essentially carried out together.
  - e.g., if an index is used for a selection, projection is done for each retrieved tuple, and the resulting tuples are *pipelined* into the aggregate computation.
  - Unless sorting is required for aggregate computation, the query can be executed by reading the tuples at most once.

Queries Over Multiple Relations

- We consider only two relation queries.
- Enumerated using 2 passes:
  - Pass 1: Find best 1-relation plan for each relation.
  - Pass 2: Find best way to join result of each 1-relation plan (as outer) to the other relation. (All 2-relation plans.)
- Retain only:
  - Cheapest plan overall, plus
  - Cheapest plan for each interesting order of the tuples.
- ORDER BY, GROUP BY, aggregates etc. handled as a final step, using either an `interestingly ordered' plan or an additional sorting operator.

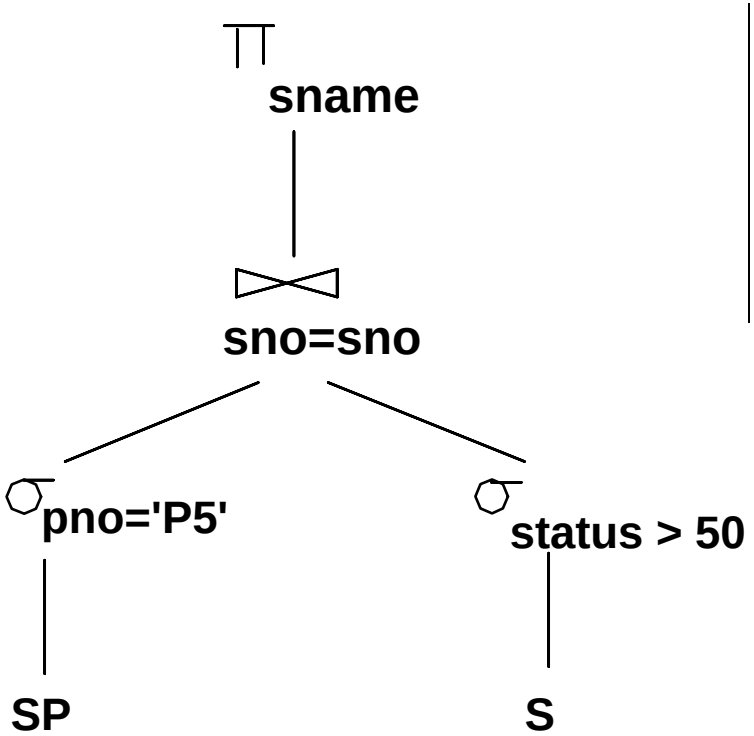
Cost of 2-relation Plans

```
SELECT  attribute list
FROM  relation1, relations2
 WHERE  term1 AND ... AND termk
```

- Consider the above query block.
- Maximum # tuples in result is the product of the cardinalities of relations in the FROM clause.
- *Reduction factor (RF)* associated with each *term* reflects the impact of the *term* in reducing result size.
- *Result cardinality* = Max # tuples * product of all RF's.
- Cost of join method, plus estimation of join cardinality gives us both cost estimate and result size estimate

Example

S:
B+ tree on *status*
Hash on *sno*
SP:
B+ tree on *pno*
All indexes are unclustered.



- Pass1:
  - S: Consider 2 access methods (Full file scan, Index scan with B+ tree)
    - Assume min status =0, max status = 100 and uniform distribution
      status > 50, RF = 50 %
    - Full file Scan
      Cost = 100 IOs
    - Index Scan with B+ on S<status>

$$Cost = h + RF * (LeafPages + Data\,Record) = 2 + \frac{50}{100} * (25 + 1000) = 2 + 13 + 500 = 515\ \text{IOs}$$

Example

- Pass1:
  - SP: Consider 2 access methods (Full file scan, Index scan with B+ tree)
    - pno='P5', RF = 0.5%
    - Full file scan

      Cost = 1000 IOs
    - Index Scan with B+ on SP<pno>

$$Cost = h + RF * (\text{Data Pages} + \text{Data Records})$$

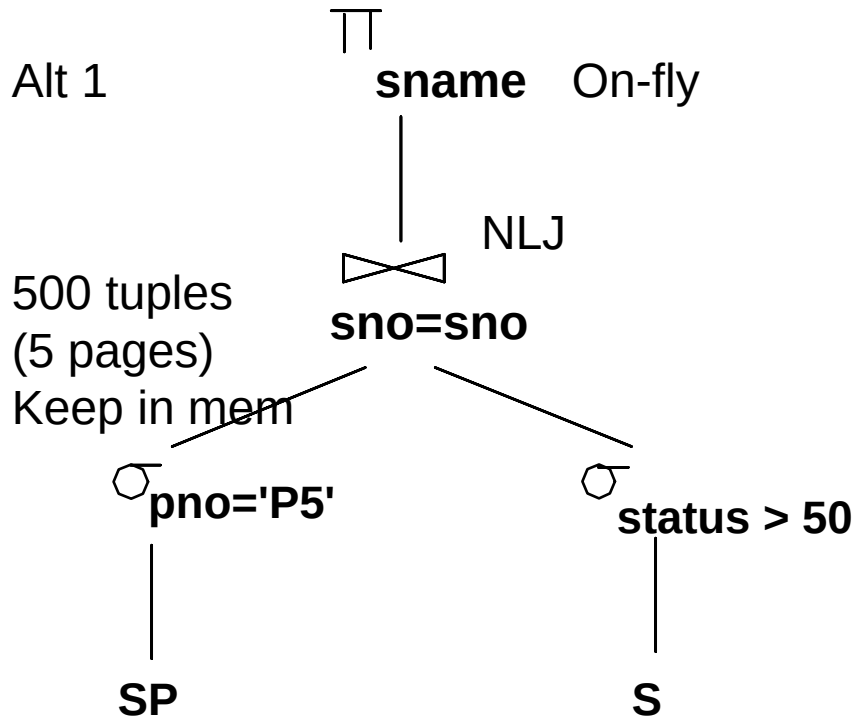
$$= 3 + \frac{0.5}{100} * (334 + 100000)$$

$$= 3 + 2 + 500 = 505 \text{ IOs}$$

Example

Pass 2:

- Index scan on SP using B+
- 500 SP tuples satisfy select condition
- Result can be kept in mem.
- Result joined with S using NLJ



- S will be accessed using full file scan
- Join result pipelined to projection to project on fly.

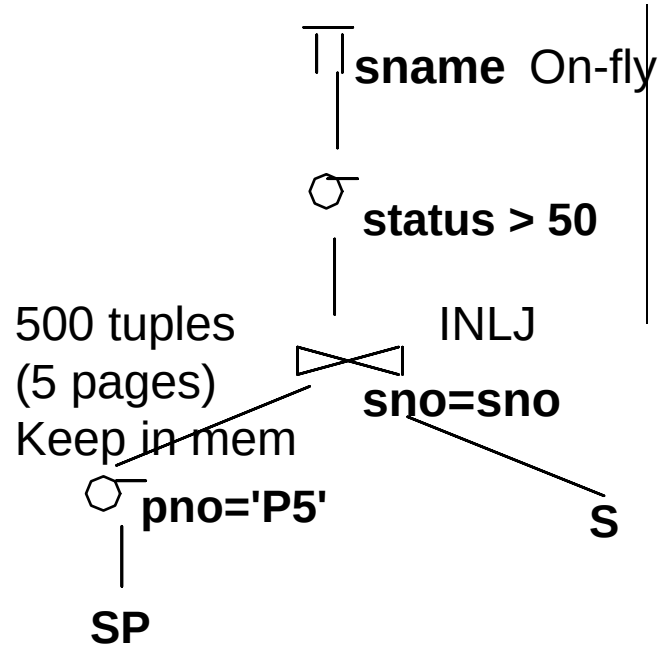
$$\text{Cost} = \text{Cost of } \sigma + \text{cost of join} + \text{cost of } \pi$$

$$= 505 + (M + M * N) + 0$$

$$= 505 + (0 + 1*100) = 605 \text{ IOs}$$

Example

Alt 2



Pass 2:

- Index scan on SP using B+
- 500 SP tuples satisfy select condition
- Result can be kept in mem.
- Result joined with S using INLJ

- Hash index on S<sno> will be used for INLJ
- Join result pipelined to selection
- Result of selection pipelined to projection to project on fly.

$$\text{Cost} = \text{Cost of } \sigma \text{ on SP} + \text{cost of join} + \text{cost of } \sigma \text{ on status} + \text{cost of } \pi$$

$$= 505 + (M + M * P_R * \text{Cost of g.m.t.}) + 0 + 0$$

$$= 505 + (0 + 500 * (1.2 + 1)) = 505 + 1100 = 1605 \text{ IOs}$$

Summary

- A query is evaluated by converting it to a tree of algebraic operators.
  - Apply transformation rules to ‘improve’ the query tree.
- Two parts to optimizing a query tree:
  - Consider a set of alternative plans using alternative access paths and algorithms for each operator.
  - Estimate cost of each plan that is considered.
    - Need to estimate size of result and cost for each opertaion.
    - Key issues: Statistics, indexes, operator implementations.

Database Systems

Transactions and Concurrency Control – Part 1

Last Week…

- Database Tuning Overview
- Any questions?

This Lecture…

- Transactions, and
- Concurrency Control

Transactions…

- A user's program may carry out many operations on the data retrieved from the database, but the DBMS is only concerned about what data is read/written from/to the database.

- A *transaction* is the DBMS's abstract view of a user program: a sequence of reads and writes.
  - Example: Transferring a balance from Account A to Account B

Transactions… (contd.)

- Properties of a transaction…
  - **A**tomicity
  - **C**onsistency
  - **I**solation
  - **D**urability

Atomicity of Transactions

- This property says that the transaction is executed as a single unit of work. That is, for every transaction, either all actions within the transaction are carried out or none are.
- A transaction might *commit* after completing all its actions, or it could *abort* (or be aborted by the DBMS) after executing some actions.
- In a committed transaction all actions are executed within a DBMS and an aborted transaction none of the actions are executed
  - DBMS *logs* all actions so that it can *undo* the actions of aborted transactions.

Consistency of Transactions

- **Database consistency** is the property that every transaction sees a consistent database instance.
- Users are mainly responsible for ensuring transaction consistency.
- ICs are verified by DBMSs. But program logic must guarantee the consistency of a transactions
  - E.g. transferring the balance from one account to another. The total amount in both accounts before and after the transaction must be equal.

Isolation of Transactions

- The isolation property ensures that even though actions of several transactions are interleaved, the net effect is identical to executing all transactions in some serial order.

- Example

Durability of Transactions

- This property ensures that transactions survive system crashes and failures.
- That is, a committed transaction is always reflected in the database & an uncommitted transaction is always rolled backed after a system crash
- DBMSs maintains a log & recovery manager is responsible to ensuring atomicity and durability properties

Transactions & Schedules…

- A transaction is seen by a DBMS as a series of ordered **actions**
- **Actions** include **reads** & **writes** of database objects
- Notation:
  - Transaction T reading object O: $R_T(O)$
  - Transaction T writing object O: $W_T(O)$

Transactions & Schedules… (contd.)

- In addition, the final action of a transaction is *commit* or *abort*.

- Notation:
  - Commiting transaction T: $Commit_T$
  - Aborting transaction T: $Abort_T$

Note: We omit the subscript T where unambiguous

Transactions & Schedules… (contd.)

- **Schedule**: A list of actions (*reads*, *writes*, *commits*, *aborts*) from a set of transactions & the order of actions in the schedule is the same as the order of actions in transactions

- Intuitively, a schedule represents an actual or potential execution sequence.

Transactions & Schedules… (contd.)

- Example

$T1$	$T2$
$R(A)$	
$W(A)$	
	$R(B)$
	$W(B)$
$R(C)$	
$W(C)$	

Concurrent Execution of Transactions

- DBMS interleaves actions of different transactions to improve performance
- Motivation for concurrent transactions
  - CPU can process one transaction while another is waiting for a page to be read from disk
  - Interleaving short transactions with longer transactions allows to complete quicker

Concurrent Execution of Transactions… (contd.)

- Example
  - T1: A transaction calculates the interest for all accounts in the bank (2 hours)
  - T2: A customer wants to withdraw funds from the account

Scheduling Transactions

- *Serial schedule:* Schedule that does not interleave the actions of different transactions.
- *Equivalent schedules*: For any database state, the effect (on the set of objects in the database) of executing the first schedule is identical to the effect of executing the second schedule.
- *Serializable schedule*: A schedule that is equivalent to some serial execution of the transactions.

(Note: If each transaction preserves consistency, every serializable schedule preserves consistency.)

Serializability

- Multiple transactions executed concurrently must be equivalent to some serial execution of the transactions in order to preserve consistency

* Order of transactions within the schedule doesn’t matter

Anomalies with Interleaved Execution

- Reading Uncommitted Data (WR Conflicts, "dirty reads"):

T1 transfers fund

$T1$	$T2$
$R(A)$	
$W(A)$	
	$R(A)$
	$W(A)$
	$R(B)$
	$W(B)$
	Commit
$R(B)$	
$W(B)$	
Commit	

T2 calculates interest

Anomalies with Interleaved Execution… (contd.)

- Unrepeatable Reads (RW Conflicts):

T1:	R(A),		R(A), W(A), C
T2:		R(A), W(A), C	

Example T1 is incrementing A by 1 and T2 is decrementing A by 1

Anomalies with Interleaved Execution… (contd.)

- Overwriting Uncommitted Data (WW Conflicts):

T1:	W(A),			W(B), C
T2:		W(A), W(B), C		

Values of A and B are equal.
T1 makes A & B to be 1000.
T2 makes A & B to be 2000.

Unserializability

- The root cause for unserializable schedules (or violation of *Isolation* property) are conflicts.
  - WR conflict
  - RW conflict
  - WW conflict

- Idea: Avoid conflicts!!!

Lock-Based Concurrency Control

- *Strict Two-phase Locking (Strict 2PL) Protocol*:
  - Each Xact must obtain a S (*shared*) lock on object before reading, and an X (*exclusive*) lock on object before writing.
  - All locks held by a transaction are released when the transaction completes
  - If an Xact holds an X lock on an object, no other Xact can get a lock (S or X) on that object.

Strict 2PL

- Strict 2PL does not allow
  - WR conflict
  - RW conflict, or
  - WW conflict
- How?
- Therefore, Strict 2PL allows only serializable schedules.

Aborting a Transaction

- If a transaction *Ti* is aborted, all its actions have to be undone. Not only that, if *Tj* reads an object last written by *Ti*, *Tj* must be aborted as well!

- Some considerations with aborts:
  - Cascading aborts
  - Unrecoverable Schedules

Cascading Abort

- Abort of one transaction causes the abort of another transaction

- Example

T_1	T_2
W(A)	
	R(A)
abort	

  - T_1's abort causes abort of T_2 as well.

Cascading abort (contd.)

- Root cause for cascading abort:
  - WR conflict +
  - Transaction which wrote the data aborts
- Good News: Strict 2PL does not allow cascading aborts!!!
  - How?
- This is also known as ***avoiding cascading aborts***

Unrecoverable Schedules

- An unrecoverable schedule:

T_1	T_2
W(A)	
	R(A)
	W(B)
	Commit
abort	

Unrecoverable Schedules (contd.)

- Root cause for unrecoverable schedule:
  - WR conflict +
  - Transaction which read the data commits before transaction which wrote the data
- Good News: Strict 2PL does not allow unrecoverable schedules!!!
  - How?

The Log

- In order to *undo* the actions of an aborted transaction, the DBMS maintains a *log* in which every write is recorded. This mechanism is also used to recover from system crashes: all active Xacts at the time of the crash are aborted when the system comes back up

The Log (contd.)

- The following actions are recorded in the log:
  - *Ti writes an object*: the old value and the new value.
    - Log record must go to disk *before* the changed page!
  - *Ti commits/aborts*: a log record indicating this action.

- Log records are chained together by Xact id, so it's easy to undo a specific Xact.
- Log is often *duplexed* and *archived* on stable storage.
- All log related activities (and in fact, all CC related activities such as lock/unlock, dealing with deadlocks etc.) are handled transparently by the DBMS.

Summary

- Transaction
- Properties
- Interleaved Execution
- Conflicts
- Avoiding conflicts with Strict 2PL Protocol
- Aborting Transaction and Log

Database Systems

Transactions and Concurrency Control – Part 2

Overheads of Lock-based CC Protocols

- Locking based CC Protocols comes with a numbers of overheads:
  - Handling deadlocks
  - Handling phantoms
  - Maintaining and handling locks

Deadlocks

- Deadlock: Cycle of transactions waiting for locks to be released by each other.

- Two ways of dealing with deadlocks:
  - Deadlock prevention
  - Deadlock detection

Deadlock Prevention

- Assign priorities based on timestamps. Assume Ti wants a lock that Tj holds. Two policies are possible:
  - Wait-Die: It Ti has higher priority, Ti waits for Tj; otherwise Ti aborts
  - Wound-wait: If Ti has higher priority, Tj aborts; otherwise Ti waits
- If a transaction re-starts, make sure it has its original timestamp.
  - Why?

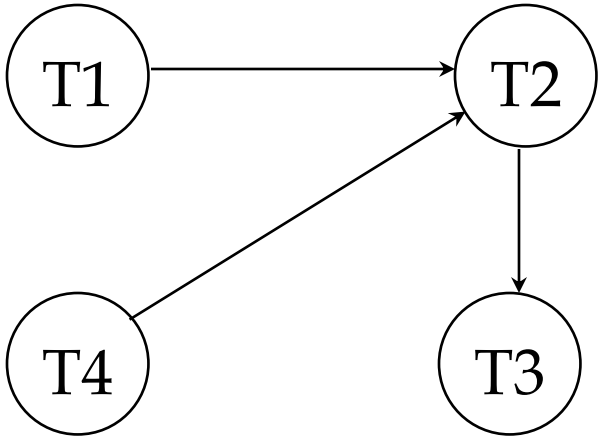
Deadlock Detection

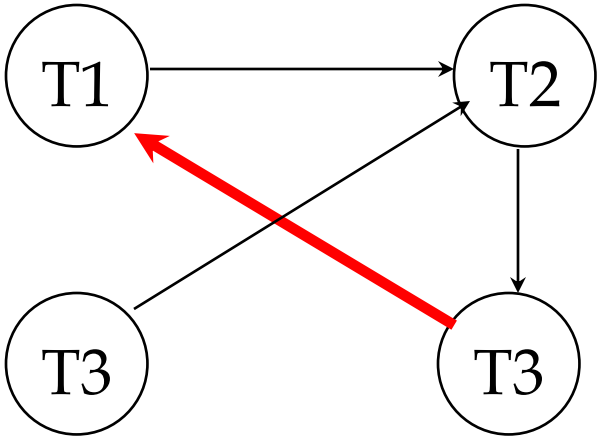
- Create a waits-for graph:
  - Nodes are transactions
  - There is an edge from Ti to Tj if Ti is waiting for Tj to release a lock
- Periodically check for cycles in the waits-for graph
- If deadlock occurs, pick a *victim* transaction and abort.

Deadlock Detection (contd.)

Example:

T1:	S(A), R(A),		S(B)			
T2:		X(B),W(B)		X(C)		
T3:			S(C), R(C)			X(A)
T4:				X(B)		





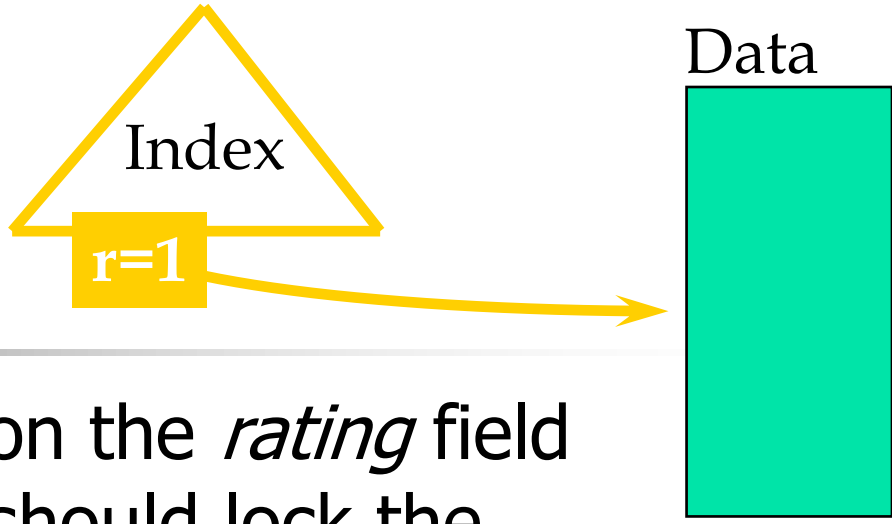
Dynamic Databases & Phantoms

- If we relax the assumption that the DB is a fixed collection of objects, even Strict 2PL will not assure serializability:
  - T1 locks all pages containing sailor records with *rating* = 1, and finds oldest sailor (say, *age* = 71).
  - Next, T2 inserts a new sailor; *rating* = 1, *age* = 96.
  - T2 also deletes oldest sailor with rating = 2 (and, say, *age* = 80), and commits.
  - T1 now locks all pages containing sailor records with *rating* = 2, and finds oldest (say, *age* = 63).
- No consistent DB state where T1 is "correct"!

The Problem

- T1 implicitly assumes that it has locked the set of all sailor records with *rating* = 1.
  - Assumption only holds if no sailor records are added while T1 is executing!
  - Need some mechanism to enforce this assumption. (Predicate locking)
- Example shows that conflict serializability guarantees serializability only if the set of objects is fixed!

Index Locking



- If there is a dense index on the *rating* field using Alternative (2), T1 should lock the index page containing the data entries with *rating* = 1.
  - If there are no records with *rating* = 1, T1 must lock the index page where such a data entry *would* be, if it existed!
- If there is no suitable index, T1 must lock all pages, and lock the file/table to prevent new pages from being added, to ensure that no new records with *rating* = 1 are added.

Predicate Locking

- Grant lock on all records that satisfy some logical predicate, e.g. *age > 2*salary*.
- Index locking is a special case of predicate locking for which an index supports efficient implementation of the predicate lock.
  - What is the predicate in the sailor example?
- In general, predicate locking has a lot of locking overhead.

Locking in B+ Trees

- How can we efficiently lock a particular leaf node?
- One solution: Ignore the tree structure, just lock pages while traversing the tree, following 2PL.
- This has terrible performance!
  - Root node (and many higher level nodes) become bottlenecks because every tree access begins at the root.

Two Useful Observations

- Higher levels of the tree only direct searches for leaf pages.
- For inserts, a node on a path from root to modified leaf must be locked (in X mode, of course), only if a split can propagate up to it from the modified leaf. (Similar point holds w.r.t. deletes.)
- We can exploit these observations to design efficient locking protocols that guarantee serializability *even though they violate 2PL.*

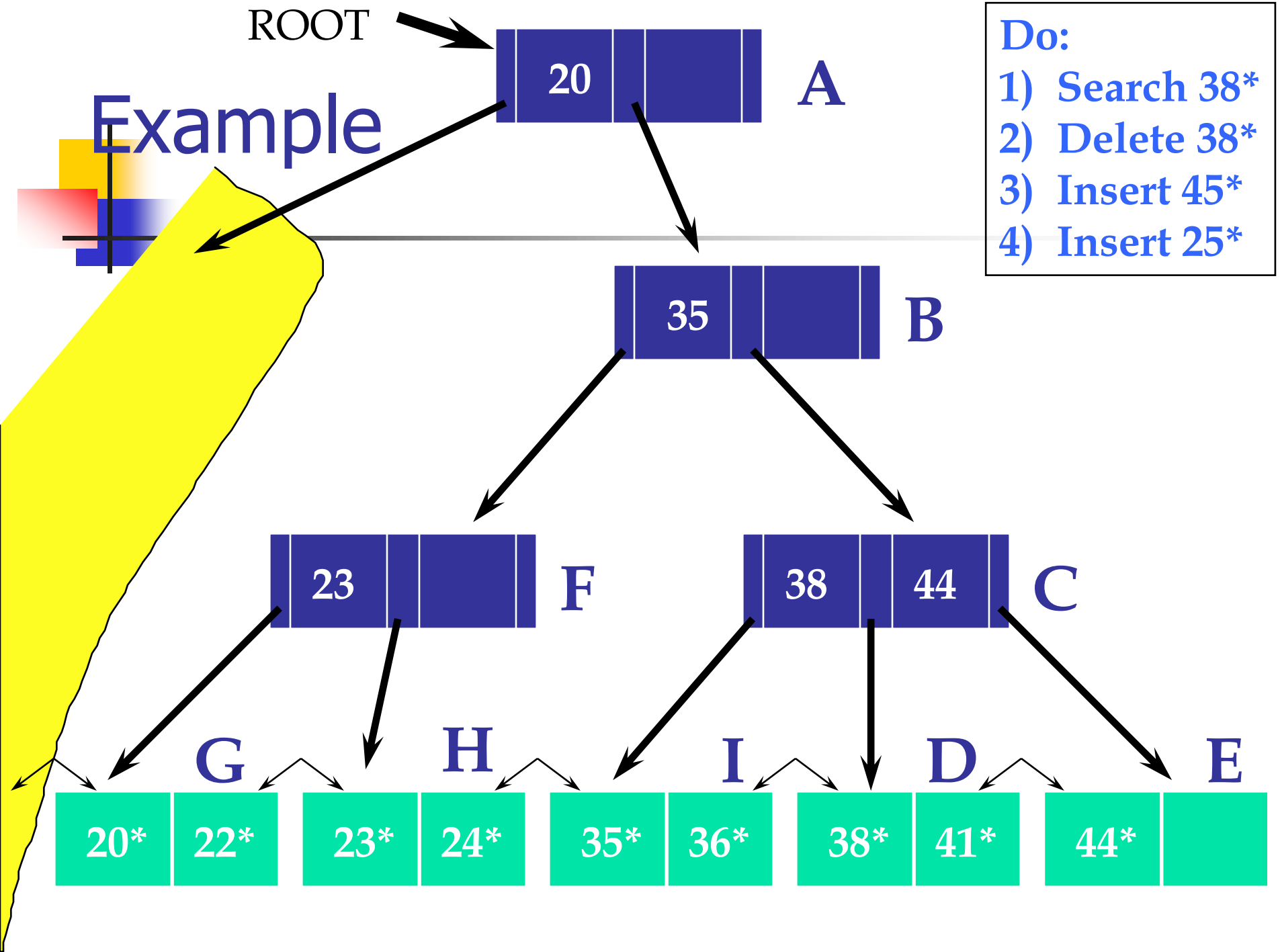
A Simple Tree Locking Algorithm

- Search: Start at root and go down; repeatedly, S lock child then unlock parent.

- Insert/Delete: Start at root and go down, obtaining X locks as needed. Once child is locked, check if it is safe:
  - If child is safe, release all locks on ancestors.

- Safe node: Node such that changes will not propagate up beyond this node.
  - Inserts: Node is not full.
  - Deletes: Node is not half-empty.

Example



Do:
1) Search 38*
2) Delete 38*
3) Insert 45*
4) Insert 25*

Lock Management…

- Lock and unlock requests are handled by the **lock manager**.
- Lock manager maintains a **lock table**
- **Lock table entry** for an object:
  - Number of transactions currently holding a lock
  - Type of lock held (shared or exclusive)
  - Pointer to queue of lock requests
- Locking and unlocking have to be atomic operations
- Lock upgrade: transaction that holds a shared lock can be upgraded to hold an exclusive lock

Lock Management…(contd.)

- DBMS also maintains a **transaction table** which contains a pointer to a list of locks held by the transactions

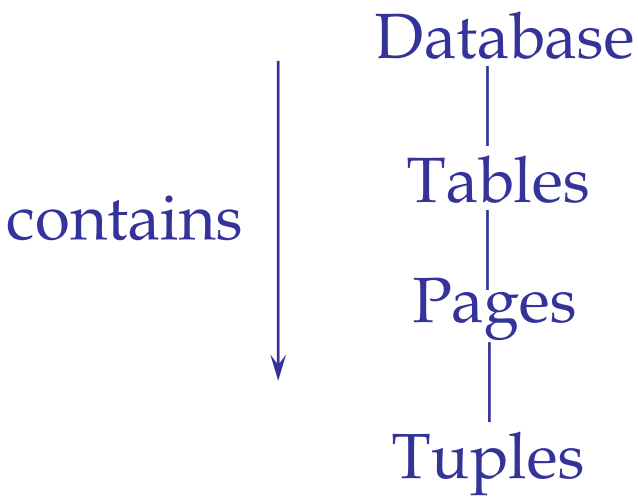
- Transaction requests a lock on an object
  - If shared lock is needed and the request queue is empty and object does not have exclusive lock, then grant & update lock entry
  - If an exclusive lock is needed and the request queue is empty and object is not in shared/exclusive mode, grant & update lock entry
  - Otherwise, add to the request to the request queue

Multiple-Granularity Locking…

- Lock manager finds it difficult to manage locks
- Multiple Granularity: Files, pages, records etc.
  - An X-lock on page means that X-lock of file is not possible
  - Before granting an X-lock on file, need to scan all pages to verify no S or X locks on any page
- Solution: Use multiple-granularity locking scheme

Multiple-Granularity Locking… (contd.)

- Data "containers" are nested:



Solution: New Lock Modes, Protocol

- Allow Xacts to lock at each level, but with a special protocol using new "intention" locks:
  - Before locking an item, Xact must set "intention locks" on all its ancestors.
  - For unlock, go from specific to general (i.e., bottom-up).
  - SIX mode: Like S & IX at the same time.

	--	IS	IX	S	X
--	√	√	√	√	√
IS	√	√	√	√	
IX	√	√	√		
S	√	√		√	
X	√				

Multiple Granularity Lock Protocol

- Each Xact starts from the root of the hierarchy.
- To get S or IS lock on a node, must hold IS or IX on parent node.
  - What if Xact holds SIX on parent? S on parent?
- To get X or IX or SIX on a node, must hold IX or SIX on parent node.
- Must release locks in bottom-up order.

Protocol is correct in that it is equivalent to directly setting locks at the leaf levels of the hierarchy.

Examples

- T1 scans R, and updates a few tuples:
  - T1 gets an SIX lock on R, then repeatedly gets an S lock on tuples of R, and occasionally upgrades to X on the tuples.
- T2 uses an index to read only part of R:
  - T2 gets an IS lock on R, and repeatedly gets an S lock on tuples of R.
- T3 reads all of R:
  - T3 gets an S lock on R.
  - OR, T3 could behave like T2; can use lock escalation to decide which.

Transactions in SQL…

- Access Level: READ ONLY, READ WRITE
- Isolation Level:
  - SERIALIZABLE (Strict 2PL + Index Locking)
  - REPEATABLE READ (Strict 2PL)
    - Phantoms possible
  - READ COMMITTED (S-locks are released b/f end)
    - Phantoms+ Unrepeatable reads possible
  - READ UNCOMMITTED (can only be used with READ ONLY transactions)
    - Phantoms + Unrepeatable reads + Dirty reads possible

Other Concurrency Control Protocols…

- Optimistic Concurrency Control Protocol
- Timestamp-Based Concurrency Control Protocol
- Multiversion Concurrency Control Protocol

Etc.

Summary

- Overheads of lock-based CC protocols
  - Deadlocks
    - Prevention and Detection
  - Phantoms
    - Predicate and Index locking
  - Handling locks
    - Multiple Granularity Locking
- Controlling Isolation level using SQL

Summary (contd.)

- Concurrency control and recovery are among the most important functions provided by a DBMS.

- Users need not worry about concurrency.
  - System automatically inserts lock/unlock requests and schedules actions of different Xacts in such a way as to ensure that the resulting execution is equivalent to executing the Xacts one after the other in some order.

Database Systems

Crash Recovery

Last Lecture

- Transactions and CC
- Any questions?

This lecture…

- Crash Recovery
  - How do you guarantee Atomicity and Durability (with system crashes)?

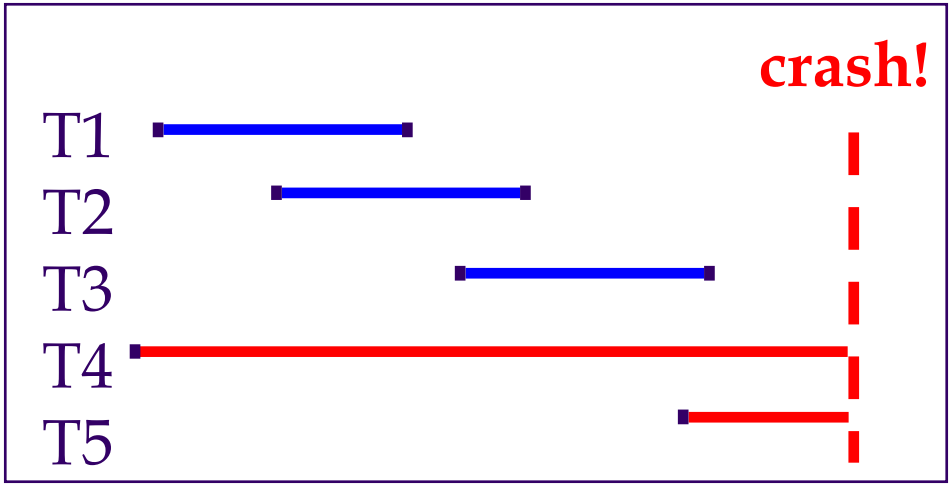
Review: The ACID properties

- **A**tomicity: All actions in the Xact happen, or none happen.
- **C**onsistency: If each Xact is consistent, and the DB starts consistent, it ends up consistent.
- **I**solation: Execution of one Xact is isolated from that of other Xacts.
- **D**urability: If a Xact commits, its effects persist.
- The **Recovery Manager** guarantees Atomicity & Durability.

Motivation

- Atomicity:
  - Transactions may abort ("Rollback").
- Durability:
  - What if DBMS stops running? (Causes?)

- Desired Behavior after system restarts:
  - T1, T2 & T3 should be durable.
  - T4 & T5 should be aborted (effects not seen).



Assumptions

- Concurrency control is in effect.
  - Strict 2PL, in particular.
- Updates are happening “in place”.
  - i.e. data is overwritten on (deleted from) the disk.

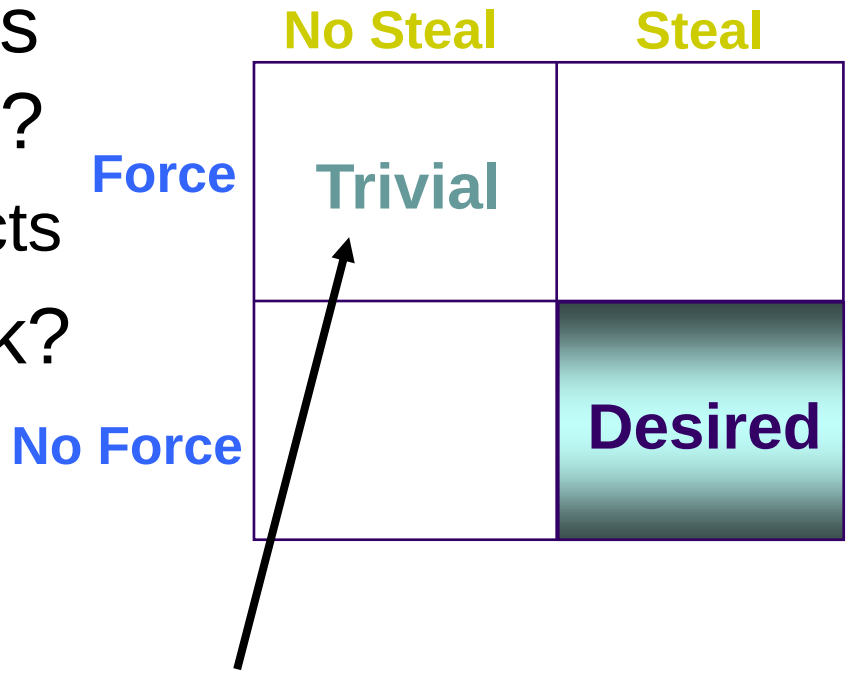
- A simple scheme to guarantee Atomicity & Durability?

Stealing Frames & Forcing Pages…

- Steal
  - Can we write pages to disk of uncommitted transactions?
- Force
  - When a transaction commits, does it require all objects (with changes made during the transaction) be flushed to disk?

Handling the Buffer Pool

- Doesn't allow Steal (No Steal) buffer-pool frames from uncommited Xacts?
  - Not realistic for large Xacts
- Force every write to disk?
  - Poor response time.
  - But provides durability.



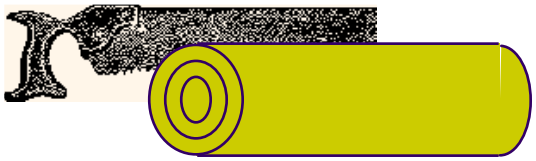
No undoing uncommitted transaction OR re-doing actions of committed transactions after a crash

More on Steal and Force

- **STEAL** (why enforcing Atomicity is hard)
  - *To steal frame F:* Current page in F (say P) is written to disk; some Xact holds lock on P.
    - What if the Xact with the lock on P aborts?
    - Must remember the old value of P at steal time (to support UNDOing the write to page P).

- **NO FORCE** (why enforcing Durability is hard)
  - What if system crashes before a modified page is written to disk?
  - Write as little as possible, in a convenient place, at commit time, to support REDOing modifications.

Basic Idea: Logging



- Record REDO and UNDO information, for every update, in a *log*.
  - Sequential writes to log (put it on a separate disk).
  - Minimal info (diff) written to log, so multiple updates fit in a single log page.
- Log: An ordered list of REDO/UNDO actions
  - Log record contains:

    <XID, pageID, offset, length, old data, new data>
  - and additional control info (which we'll see soon).

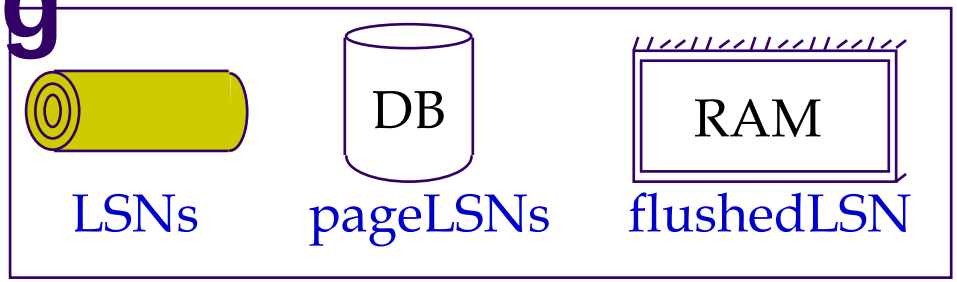
Write-Ahead Logging (WAL)

- The Write-Ahead Logging Protocol:
  - ① Must force the log record for an update *before* the corresponding data page gets to disk.
  - ② Must write all log records for a Xact *before commit*.

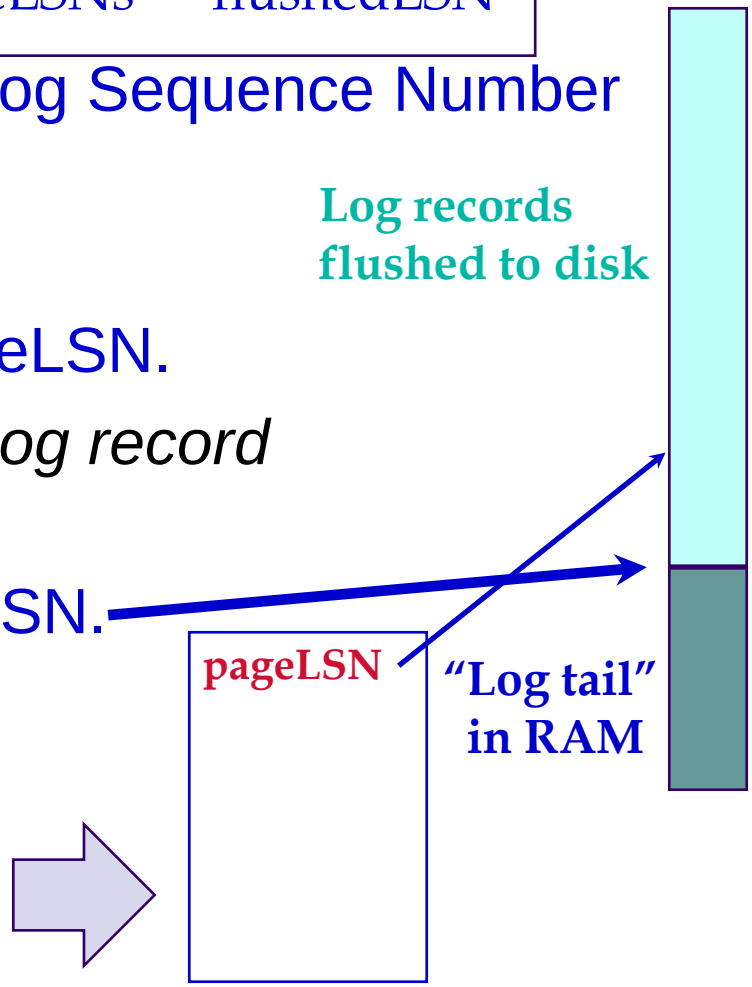
- #1 guarantees Atomicity.
- #2 guarantees Durability.

- Exactly how is logging (and recovery!) done?
  - We'll study the ARIES algorithms.

WAL & the Log



- Each log record has a unique Log Sequence Number (LSN).
  - LSNs always increasing.
- Each *data page* contains a pageLSN.
  - The LSN of the most recent *log record* for an update to that page.
- System keeps track of flushedLSN.
  - The max LSN flushed so far.
- WAL: *Before* a page is written,
  - pageLSN ≤ flushedLSN



Log Records

LogRecord fields:

prevLSN
XID
type
pageID
length
offset
before-image
after-image

update records only

Possible log record types:

- **Update**
- **Commit**
- **Abort**
- **End** (signifies end of commit or abort)
- Compensation Log Records (CLRs)
  - for UNDO actions

Other Log-Related State

- Transaction Table:
  - One entry per active Xact.
  - Contains XID, status (running/commited/aborted), and lastLSN.
- Dirty Page Table:
  - One entry per dirty page in buffer pool.
  - Contains recLSN -- the LSN of the log record which ***first*** caused the page to be dirty.

Normal Execution of an Xact

- Series of reads & writes, followed by commit or abort.
  - We will assume that write is atomic on disk.
    - In practice, additional details to deal with non-atomic writes.
- Strict 2PL.
- STEAL, NO-FORCE buffer management, with Write-Ahead Logging.

Checkpointing

- Periodically, the DBMS creates a checkpoint, in order to minimize the time taken to recover in the event of a system crash. Write to log:
  - begin_checkpoint record: Indicates when chkpt began.
  - end_checkpoint record: Contains current *Xact table* and *dirty page table*. This is a `fuzzy checkpoint':
    - Other Xacts continue to run; so these tables accurate only as of the time of the begin_checkpoint record.
    - No attempt to force dirty pages to disk; effectiveness of checkpoint limited by oldest unwritten change to a dirty page. (So it's a good idea to periodically flush dirty pages to disk!)
  - Store LSN of chkpt record in a safe place (*master* record).

The Big Picture: What's Stored Where

LogRecords

- prevLSN
- XID
- type
- pageID
- length
- offset
- before-image
- after-image

Data pages

each with a pageLSN

master record

Xact Table

- lastLSN
- status

Dirty Page Table

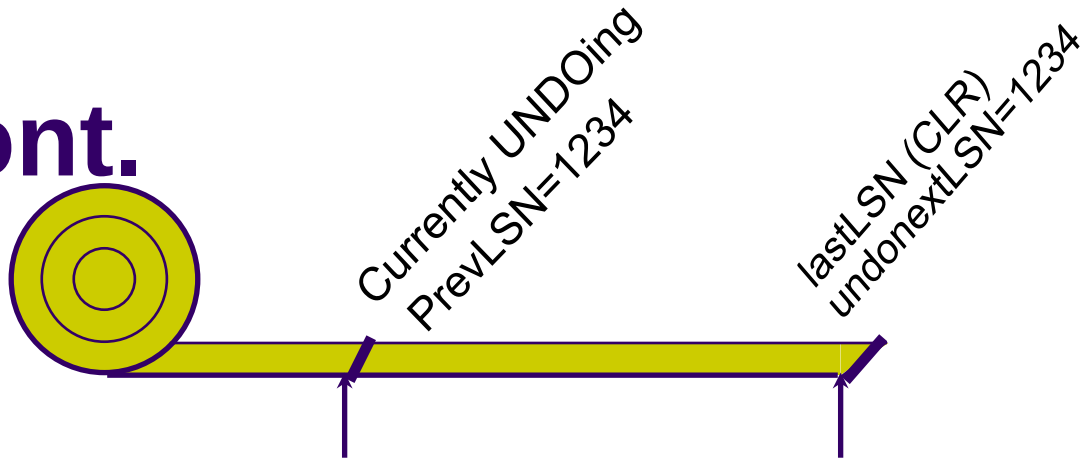
- recLSN

flushedLSN

Simple Transaction Abort

- For now, consider an explicit abort of a Xact.
  - No crash involved.
- We want to “play back” the log in reverse order, UNDOing updates.
  - Get lastLSN of Xact from Xact table.
  - Can follow chain of log records backward via the prevLSN field.
  - Before starting UNDO, write an *Abort* log record.
    - For recovering from crash during UNDO!

Abort, cont.

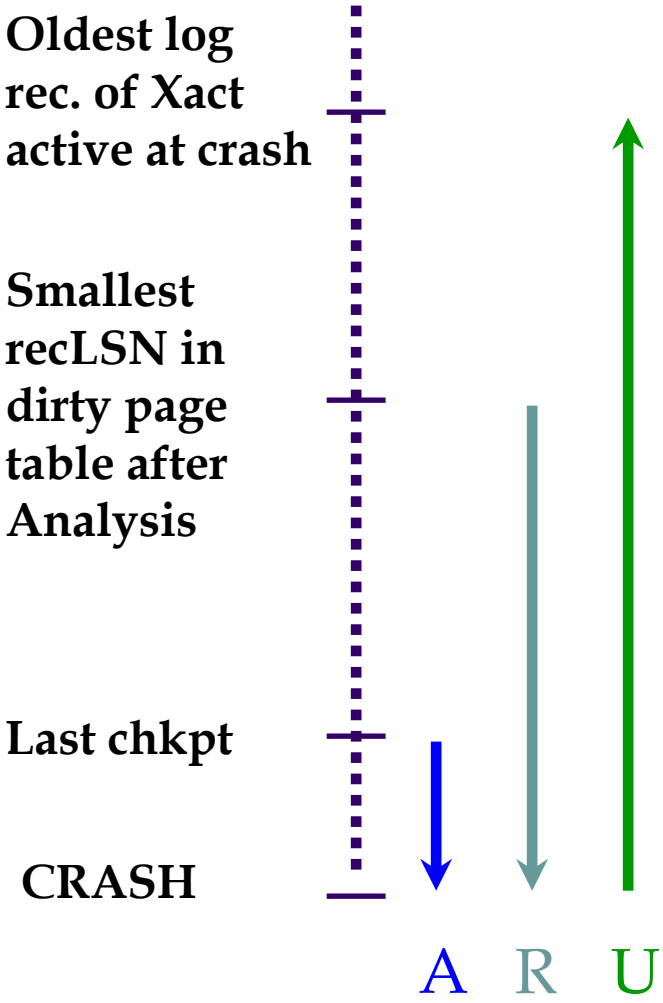


- To perform UNDO, must have a lock on data!
  - No problem!
- Before restoring old value of a page, write a CLR:
  - You continue logging while you UNDO!!
  - CLR has one extra field: undonextLSN
    - Points to the next LSN to undo (i.e. the prevLSN of the record we're currently undoing).
  - CLRs *never* Undone (but they might be Redone when repeating history: guarantees Atomicity!)
- At end of UNDO, write an "end" log record.

Transaction Commit

- Write commit record to log.
- All log records up to Xact's lastLSN are flushed.
  - Guarantees that flushedLSN $\geq$ lastLSN.
  - Note that log flushes are sequential, synchronous writes to disk.
  - Many log records per log page.
- Commit() returns.
- Write end record to log.

Crash Recovery: Big Picture



- Start from a checkpoint (found via master record).
- Three phases. Need to:
  - Figure out which Xacts committed since checkpoint, which failed (Analysis).
  - REDO ***all*** actions.
    - (repeat history)
  - UNDO effects of failed Xacts.

Recovery: The Analysis Phase

- Reconstruct state at checkpoint.
  - via end_checkpoint record.
- Scan log forward from checkpoint.
  - End record: Remove Xact from Xact table.
  - Other records: Add Xact to Xact table, set lastLSN=LSN, change Xact status on commit.
  - Update record: If P not in Dirty Page Table,
    - Add P to D.P.T., set its recLSN=LSN.

Recovery: The REDO Phase

- We *repeat History* to reconstruct state at crash:
  - Reapply *all* updates (even of aborted Xacts!), redo CLRs.
- Scan forward from log rec containing smallest recLSN in D.P.T. For each CLR or update log rec LSN, REDO the action unless:
  - Affected page is not in the Dirty Page Table, or
  - Affected page is in D.P.T., but has recLSN > LSN, or
  - pageLSN (in DB) ≥ LSN.
- To REDO an action:
  - Reapply logged action.
  - Set pageLSN to LSN. No additional logging!
- At the end, write End transaction records to all transactions with Commit status and remove from transaction table

Recovery: The UNDO Phase

ToUndo={ / | / a lastLSN of a “loser” Xact}

Repeat:

- Choose largest LSN among ToUndo.
- If this LSN is a CLR and undonextLSN==NULL
  - Write an End record for this Xact.
- If this LSN is a CLR, and undonextLSN != NULL
  - Add undonextLSN to ToUndo
- Else this LSN is an update. Undo the update, write a CLR, add prevLSN to ToUndo.

Until ToUndo is empty.

Example of Recovery

Xact Table
- lastLSN
- status

Dirty Page Table
- recLSN

flushedLSN

ToUndo

LSN	LOG
00	begin_checkpoint
05	end_checkpoint
10	update: T1 writes P5
20	update T2 writes P3
30	T1 abort
40	CLR: Undo T1 LSN 10
45	T1 End
50	update: T3 writes P1
60	update: T2 writes P5
	CRASH, RESTART

prevLSNs

Example: Crash During Restart!

Xact Table
- lastLSN
- status

Dirty Page Table
- recLSN

flushedLSN

ToUndo

LSN	LOG
00,05	begin_checkpoint, end_checkpoint
10	update: T1 writes P5
20	update T2 writes P3
30	T1 abort
40,45	CLR: Undo T1 LSN 10, T1 End
50	update: T3 writes P1
60	update: T2 writes P5
	CRASH, RESTART
70	CLR: Undo T2 LSN 60
80,85	CLR: Undo T3 LSN 50, T3 end
	CRASH, RESTART
90	CLR: Undo T2 LSN 20, T2 end

undonextLSN

Additional Crash Issues

- What happens if system crashes during Analysis? During REDO?
- How do you limit the amount of work in REDO?
  - Flush asynchronously in the background.
  - Watch “hot spots”!
- How do you limit the amount of work in UNDO?
  - Avoid long-running Xacts.

Summary of Logging/Recovery

- Recovery Manager guarantees Atomicity & Durability.
- Use WAL to allow STEAL/NO-FORCE w/o sacrificing correctness.
- LSNs identify log records; linked into backwards chains per transaction (via prevLSN).
- pageLSN allows comparison of data page and log records.

Summary (contd.)

- Checkpointing: A quick way to limit the amount of log to scan on recovery.
- Recovery works in 3 phases:
  - Analysis: Forward from checkpoint.
  - Redo: Forward from oldest recLSN.
  - Undo: Backward from end to first LSN of oldest Xact alive at crash.
- Upon Undo, write CLRs.
- Redo “repeats history”: Simplifies the logic!